What It Takes to Run a 125B LLM Fast on a 64 GB Mac, and How to Go Faster

Complete report: the main paper and four companion papers as Parts I-V

Kamen Levi

9 November 2026

Contents

		page
Part I	What It Takes to Run a 125B LLM Fast on a 64 GB Mac, and How to Go Faster	3
Part II	Batch-invariant kernels on Apple's M5 GPU and its matrix units	37
Part III	Certifying lossy decoding on the model's known benchmarks: same-scores gates for a 125B mixture-of-experts model on one 64 GB Mac	50
Part IV	Drafting for a 125B mixture-of-experts model that does not fit in RAM: training its own MTP head, one drafter per text type, and the levers that failed	62
Part V	Running a 125B mixture-of-experts model on a 64 GB Mac: the engine package	79

The papers cite each other by tag: [MAIN] = Part I, [KERNELS] = Part II, [EVAL] = Part III, [DRAFTER] = Part IV, [RELEASE] = Part V. Each part keeps its own page numbers; the page numbers above count the pages of this file.

What It Takes to Run a 125B LLM Fast on a 64 GB Mac, and How to Go Faster

How batch-invariant speculative checks, SSD-streamed experts and a certified lossy layer build on each other within a 45 GB memory line, and what each piece adds.

Kamen Levi
Independent researcher

9 November 2026

Abstract

We run a 125B-parameter mixture-of-experts model (Qwen3.8-Flash-Next: 48 layers, 512 routed experts per layer, 10 per token) on one MacBook Pro (M5 Pro, 64 GB), with the routed experts streamed from the internal SSD at about 3.2 bits per weight and the process's lifetime peak at most 45 GB (plus a 7.45 GB expert tier that macOS may reclaim). The pieces are mostly published; this paper is about how they depend on each other and what each adds. Speculative decoding pays one pass of the large model for up to 16 drafted tokens, but it is exact only if that pass computes the same bits as one-token decoding. Batch-invariant kernels on the GPU and its matrix units make it so, and we audit it on every run (96 of 96 code runs, 24 of 24 chat and 24 of 24 math runs token-identical to one-token greedy decoding). Exactness then turns every later optimisation into a test of token equality that takes minutes: the levers of the streamed engine, such as per-layer expert pools and expert reads hidden behind the GPU's work, were each adopted that way. On top of the exact engine, a lossy layer lets drafted rows of code answers skip cold experts and run 3-bit parts, and ships only if it scores the same as the exact reference on the model's benchmarks.

The exact configuration decodes 101.4 tok/s on 12 HumanEval prompts, 70.4 on 12 MBPP, 77.9 on 12 math and 37.1 on 12 chat (IFEval) prompts, at a lifetime peak of 41.7-42.5 GB [K]. The lossy configuration we ship scores the same as the exact reference on the model card's IFBench, GPQA Diamond and LiveCodeBench v6 in both prompt orders, and decodes 164.0, 112.7, 78.9 and 45.2 tok/s on the same sets at 42.4-43.2 GB [K]. Our first token is slow, a median of 2.7-4.6 s on these short prompts: on the same machine and prompts the open engine Strata's takes 0.54-1.01 s, and Strata wins or ties every whole request of up to 256 tokens, although our lossy decode is 2.5-3.5x its decode on code [K]. None of the methods is specific to this model's layers; we tested one model on one machine. Section 12 describes each finding that, to our knowledge, has not been published before, with its evidence, what it gives and its limits, including one idea that did not pay off. We also report negative results, including three headline numbers we corrected or withdrew.

---

1. Introduction

1.1 The setting

- **The model.** Qwen3.8-Flash-Next (arXiv 2608.30320) is an open-weight mixture-of-experts model with a hybrid trunk: 36 linear-attention (Gated DeltaNet, "GDN") layers and 12 full-attention layers, four residual streams joined by low-rank hyper-connection (HC) mixers, one shared expert and 512 routed experts per layer with 10 routed per token, and a multi-token-prediction (MTP) head. It has about 125B parameters in total (section 4.1).
- **The machine.** One MacBook Pro, M5 Pro, 64 GB of unified memory, internal SSD. All numbers are for one user, greedy decoding.
- **The problem.** At MLX 4-bit the routed experts alone are 67.9 GB [K, BM:FINDINGS.md section 1], so they cannot all stay in memory. A token reads about 2.46 GB of dense weights and 1.33 GB of routed experts [K, same]. Decoding one token at a time pays a full pass over the dense trunk per token.
- **Speculative decoding** pays that pass once per check: a small drafter proposes a block of tokens and one pass of the large model verifies up to 16 of them.

- **Why exactness is not free.** A verify pass computes 16 positions in one batched call; one-token decoding computes each in a 1-row call. On stock kernels those are different programs with different rounding: in our first screen all 16 rows differed from one-token decoding at layer 0 (section 3). A greedy decoder then drifts at near-ties, and "speculative decoding is lossless" stops being true.

1.2 The idea: known pieces that need each other

The pieces build on published work; section 8 credits each one and says which part is ours, and the prior art is in Appendix I. Section 12 describes the findings of this work that, to our knowledge, have not been published before. What this paper adds is the order in which they depend on each other, and that they still work, exactly, when combined. Section 7 gives each step's measured effect, and says where a step was never measured alone.

1. **Exact checks.** Speculative decoding verifies up to 16 drafted tokens in one pass, but greedy decoding is unchanged only if that pass computes the same bits as 16 one-token steps. Batch-invariant kernels make it so (section 3; [KERNELS]).
2. **Optimisation by token equality.** With an exact reference, a change to the engine is accepted when every token equals the reference on a handful of prompts, in minutes, with no quality argument (section 3.3). Every lever of the 45 GB engine was adopted that way (section 4.4).
3. **A store that fits.** The routed experts (120.8B parameters) live on the SSD as aligned records with a 3-bit part and a 4th-bit sidecar; each layer keeps a pool of them in memory, and a token's two top-ranked experts read both parts while the other eight read the 3-bit part (sections 4.1-4.2). The check loop is what makes streaming affordable: a check pays the SSD reads for the union of its rows' experts once, and every token it keeps shares them.
4. **Hiding the reads.** Expert ids stay on the GPU, and resident experts compute while missed ones load (section 4.4). The experts' outputs are still summed in a fixed order, so both changes keep every token equal and were accepted as in step 2.
5. **A certified lossy layer.** On drafted rows of code answers the engine skips cold lower-ranked experts and runs the 3-bit part of the rest, and a few reads are made at lower precision everywhere (section 5). The check loop says which rows are drafted; the layer ships only if it scores the same as the exact reference of step 1 on the model's benchmarks.
6. **Prefill from the same store.** A short prompt can build its experts from the copies already in memory and read only what is missing (PF-D, shipped for chat); a long prompt can run each layer over the whole prompt, so that each expert is read about once (built, not shipped; section 4.5).

1.3 What carries to other models

None of the methods depends on this model's linear-attention or hyper-connection layers (section 8). The exact check needs batch-invariant kernels for the model's operations; the store, the loader and the lossy row rules need a mixture of experts with a drafter. The design is meant as an engine for such models in general, but we tested one model on one machine, and that is the limit of what this paper shows.

1.4 Contributions

1. **Exact speculative checks at 125B on Apple silicon (section 3; kernels in [KERNELS]).** A 16-row check that equals 16 one-token steps bit for bit, audited on every measured run, and a rule for changing the reference.
2. **An engine built by token equality (sections 4 and 7).** An SSD-streamed store, cache and loader within a 45 GB memory line, each lever accepted by token equality, and an account of what each piece adds on top of the others and which steps were never measured alone.
3. **A lossy configuration certified on the model's benchmarks (section 5; method in [EVAL]).** The same scores as the exact reference on HumanEval+, MBPP, math and IFEval, and, for the configuration we ship, on the model card's IFBench, GPQA Diamond and LiveCodeBench v6, in both prompt orders.
4. **Methods that carry to other MoE models (section 8), a short comparison with other engines (section 9), and negative results (section 10) and the path to more speed (section 11).**
5. **New findings (section 12).** Findings that, to our knowledge, have not been published before, each with its evidence, what it gives, what is measured and what is still a hypothesis, and how it could be improved.

1.5 What we do not claim

- **Not the fastest engine for this model.** Our first token is 2.7-7.0x slower than Strata's on the same machine, and engines with lower-bit files or on larger Macs report a faster decode or prefill (self-reported; section 9). Our decode leads on code and math, slightly on chat in the lossy mode, and by a wide margin only in the lossy mode on code.
- **Not new pieces.** Every lever in the measured configurations builds on published work, credited in section 8 and Appendix I; the findings that are, to our knowledge, new are in section 12. What we add is making them work together, exactly, at this size on this hardware, and measuring what each adds. To our knowledge no published engine for this model combines all of them; Strata and Sushi combine several (section 9).
- **Not exact to the original 4-bit weights.** "Exact" always names a reference (section 2); ours runs ranks 0-1 of each token's experts at 4 bits and ranks 2-9 at 3 bits.

1.6 The companion papers

[KERNELS] has the batch-invariant kernels and the MLX findings; [EVAL] the certification method and its pitfalls; [DRAFTER] the drafter work, including what failed; [RELEASE] the engine package (GPL-3). The 32, 24 and 12 GB lines are the subject of a forthcoming companion paper ([SMALL-RAM], by 11 November 2026); no number in this paper depends on it. This paper states each companion's results in one sentence where it needs them.

2. Definitions and how we measure

- **Exact (to R).** The output tokens, and the logits row behind every kept token, are bit-identical to one-token greedy decoding under reference R, a fixed set of weights, kernels and call shapes. Changing a kernel changes rounding and makes a new reference, which is accepted once on task scores (section 3.4). Our 45 GB reference is NAX6-N.
- **Lossy, same scores on .** A configuration that changes which computation a row runs, accepted only after its pre-registered same-scores gate against its exact reference on the named known benchmarks, in two prompt orders. Never "same quality".
- **Lossy, gate open** (a gate is running or undecided) and **lossy, ungated** (no gate). Every configuration in this paper carries one of these labels.
- **Speed** comes as a triple for a named prompt set:
  - whole decode = generated tokens / decode wall time (prefill excluded, every check included);
  - whole request = generated tokens / (first token time + decode wall time);
  - first token = request start to the first generated token, including prefill and setup (median and range);
  - plus the lifetime peak memory footprint of the process (ri_lifetime_max_phys_footprint, which counts the GPU's buffers), in GB of `10^9` bytes; the 45 GB line is 41.9 GiB [K, ES:evalsuite/decode.py, footprint_peak_gb];
  - not counted in it: a 7.45 GB volatile expert tier (64 / 32 slots per base / sidecar pool), purgeable memory that macOS reclaims before it compresses or swaps; every run log behind Table 6.1 states it [K, e.g. BM:data/rel/rli_160.txt line 7].
- **Aggregation.** Pooled over prompts and passes (total tokens / total time). The engine's run reports quote a geometric mean over prompts ("geo"), 0-8 % higher on our sets; both are in Appendix E.
- **Workloads.** 12 HumanEval and 12 MBPP prompts (the same ids in every run), 12 IFEval prompts as our chat set and 12 math (MATH-500) prompts; greedy, at most 256 new tokens, EOS stop. HumanEval flatters: 67.8 % of the generated tokens on our 12 HumanEval prompts copy the prompt's own signature and docstring [K, F:lossy350-thinking-hour], which drafting accepts cheaply. We never quote HumanEval without MBPP or chat beside it.
- **The machine during runs.** Other jobs were stopped through a GPU lock ("quiet"); a run's report lists pauses for heat and drops any request that overlaps one.
- **Marks.** [K] measured, [P] priced (arithmetic or a CPU replay on measured data), [G] judgement. Every number names its source (a run record, report or result file); Appendix C maps the keys to paths.
- **The memory line.** "45 GB" means the run's lifetime peak is at most 45.0 GB. It is the engine's own footprint on a 64 GB machine, not a limit the operating system enforces.

3. Exactness (details in [KERNELS])

3.1 References

reference	what it is	status
REF2	batch-invariant kernel set: every matmul, HC read and attention call padded to 16 rows, routed experts on one 16-row tile kernel, a load-time self-test	adopted 28 Oct (T1210)
REF2b	REF2 + attention key buckets past 512 keys, attention classes below 128 keys, the commit phase and slot call	adopted 31 Oct (T1219)
REF2-T	REF2b on rank-tiered experts: ranks 0-1 at 4 bits, ranks 2-9 at 3 bits	1 Nov (T1214, T307)
NAX6-N	REF2-T with the routed experts on a matrix-unit kernel over records in the kernel's native layout	certified 5 Nov; the 45 GB exact reference (EV:RESULT_nax6_2026-11-05)

The first screen compared 16 one-token steps with one 16-row verify on identical caches, layer by layer: with the shipped kernels all 16 rows diverged at layer 0, because the framework picks a different matmul kernel by row count; attention, long context (past 512 keys) and short context (below 128 keys) broke next [K, LT:'R-40 REF2'; T1225; T1208; T1218]. The fixes and the kernels are [KERNELS]'s subject.

3.2 The audits

Each audit decodes every prompt twice, with the speculative loop and one token at a time, and compares every kept row.

reference and arm	prompts	result	source
REF2b	36 full-length (12 code, 12 math, 12 IFEval)	16,305 / 16,305 kept rows bitwise; 15 / 15 prompts past 512 keys, 6 / 6 past 1,024	EV:RESULT_r56b_2026-10-31
NAX6-N + out-of-order experts	same 36	15,690 / 15,690 kept rows bitwise; out-of-order path engaged on every tier call	EV:RESULT_nax6_2026-11-05, Part 2
the exact default (E, EB) on code	12 HumanEval + 12 MBPP, 2 arms x 2 passes	96 / 96 runs token-identical to one-token decoding	F:2026-11-07_1735
EB on chat and math	12 IFEval + 12 math, 2 passes	24 / 24 and 24 / 24	F:2026-11-07_2330
E + coherent reads (+ the 1-row check) on chat	12 IFEval	12 / 12 in two runs (37.8 / 37.9 tok/s) and 12 / 12 (38.3); without coherent reads 3 / 12	F:2026-11-08_0750
layer-major prefill	3 agent prompts, 1.6K-7.9K tokens	decode tokens equal; logits hash equal on 8 / 8, 28 / 28, 22 / 22 checks	F:2026-11-08_0800

- **The chat failure and its fix.** A one-call path for chat checks made exact chat nondeterministic (3 of 12 prompts equal). Coherent reads in the expert kernel restored 12 of 12 at no cost (F:2026-11-08_0750); [KERNELS] has the mechanism.
- **The audit is part of every run.** Each exact row of section 6 carries its own token-identity count. It is how two bugs were caught that speed numbers alone hide: a fast path that never engaged (T18) and a kernel that was silently bypassed (F:flat2, caveat).

3.3 What exactness buys

Once a reference exists, an optimisation is accepted by token equality on a handful of prompts, in minutes, with no quality argument. Every lever of section 4.4 was accepted this way. It has a price: the batch-invariant one-token path ran at x0.75 of stock kernels (11.65 vs 15.50 tok/s over 210 rows, row time with prefill included; information only) [K, EV:RESULT_ref2_2026-10-28; [KERNELS] R31].

3.4 Changing the reference: certified once, on scores

A new reference moves bits, so it is judged on tasks, once, by a rule fixed in advance ("decision 4"): on each of three sets (math, n 50; IFEval, n 100, strict prompt level; HumanEval+, n 60, pass@1) the paired score difference must be above -5 points and the exact McNemar p at least 0.05.

new reference vs old	math	IFEval	HumanEval+	verdict	source
REF2 vs the previous reference	+8.0	-3.0	0.0	PASS	EV:RESULT_ref2_2026-10-28
REF2b vs the previous reference	+6.0	-3.0	0.0	PASS	T1219
REF2-T vs REF2b	0.0	+3.0	+3.3	PASS	T307
NAX6 vs REF2-T	0.0	-3.0 (p 0.375)	-3.3 (p 0.50)	PASS	EV:RESULT_nax6_ 2026-11-05

These are same-score tests on 210 items, not proofs of equal quality: pooled over the NAX6 sets, NAX6 alone is right on 2 items and REF2-T alone on 7 (two-sided p 0.18). A further kernel bundle (REF2c-T) failed the rule on IFEval (-5.0) and was not adopted. The gaps chain: the certified lossy config of section 5 is -8.33 points on the HumanEval+ core set against REF2-T, two references back (forward order, p 0.0625), which no gate of its own measured [K, F:2026-11-08_0455; information].

4. The 45 GB system

4.1 Model, store and hardware

part	parameters	notes
routed experts	120.8 B	48 layers x 512 experts, top-10; one expert = gate/up [1280 x 2560] + down [2560 x 640]
linear attention (GDN)	2.09 B	36 of 48 layers
full attention + indexer	0.62 B	12 layers, 24 query heads, 2 key/value heads, head dim 256
hyper-connections	0.64 B	4 residual streams, low-rank mixers
embedding / output head / shared expert / router	0.64 / 0.64 / 0.24 / 0.06 B	one shared expert per layer
MTP head	2.61 B	one layer with its own 512 experts
n-gram tables	51.2 B	layer 1 only; 16 rows read per token, from the SSD

Source: BM:FINDINGS.md section 1 (config and safetensors headers). **Active per token [P, arithmetic on this table]:** 10 of 512 routed experts in each of 48 layers (480 x 4.92 M = 2.36 B), attention 2.71 B, hyper-connections 0.64 B, the shared expert and router 0.30 B: 6.0 B, or 6.65 B with the output head. The 125.7 B total counts the rows above except the MTP head, the vision tower and the n-gram tables (16 rows of 160 values read per token); with them the checkpoint stores 180 B, the size some engines quote for this model (Kapitsa, Mference; Appendix H). Our engine runs this architecture through the model code of MTPLX by Youssof Altoukhi (Apache-2.0; github.com/youssofal/mtplx); the store, the kernels and the check loop are ours.

- **Records.** Each expert is one contiguous, 16 KB-aligned record, read with one call that bypasses the page cache (F_NOCACHE): an MSB part (the 3 most significant bits of every weight with scales and biases) and an LSB sidecar (the 4th bit). The 4-bit weights are rebuilt bit-exactly from the two parts; the store is 68.5 GB on disk (48 layer files; 2.79 MB per expert record) [K, BM:models/bare/records3, the converter's output; not in the public snapshot]. For the matrix-unit kernel the records are stored in the kernel's own fragment order, with scales packed to 11 bits (bitwise; T527, T334).
- **Rank tiers.** A token's two highest-scoring experts (ranks 0-1) run at 4 bits and read both parts; ranks 2-9 run from the MSB part alone. Expert bytes fall about 20 % (T307).
- **The SSD.** Cold 2.77 MB reads reach 14.0 GB/s at queue depth 16 [K, BM:FINDINGS.md section 2]. In the engine the SSD time per check follows 10-16 ms plus 0.15-0.17 ms per missed expert [K, T288].

4.2 Expert cache and loader

- Each layer has a pool of expert slots in GPU memory; C 256 means 256 slots per layer on average, with per-layer sizes at equal memory (-17.2 % SSD bytes per kept token and +8.3 % tok/s, exact; T12, adapted from AdapMoE). Eviction is LRU.
- All misses of a layer load in parallel, aligned, straight into slot memory (T7; M-29). Part of the pool is a volatile tier the operating system may reclaim (T1088).
- **The wired limit.** MLX's wired-memory collector unwired the expert pools during prefill, and each prompt's first check re-wired about 31.8 GB, blocking for about 250 ms. One call after load (mx.set_ wired_limit, published practice in mlx-lm) removes it, bitwise in the exact mode: +25.1 % whole decode, pooled, on the lossy config (144.1 -> 180.1 tok/s, 12 HumanEval, two passes each; the notes

give geo means, 142.2 -> 181.2) [K, BM:data/r57/s6{b,w}{1,2}_256_runs.json; F:2026-11-07_0900, F:2026-11-07_0945].

4.3 The check loop

1. **Draft.** A block drafter (our fine-tune of DFlash2 by Inco AI, Apache-2.0; [DRAFTER], Appendix I) proposes up to 16 positions in one pass; chat uses the model's own MTP head, chosen by task (T782). Drafts are kept while the running product of the drafter's confidences stays at or above 0.3 (T309).
2. **Verify.** One padded 16-row pass of the 125B model checks the drafts; they are accepted up to the first mismatch, and the row after the last accepted draft gives one more token (the bonus row).
3. **Commit.** The recurrent and attention states advance by the kept rows only.

Tokens kept per check decide the speed and depend on the workload: 10.7 per check on HumanEval, 6.0-6.1 on MBPP, 6.5 on math and 2.5 on chat (kept tokens over checks, pooled) [K, F:2026-11-07_1735; F:2026-11-07_2355]. Chat checks are short because the drafter is unsure, and their cost is the low-row check's fixed cost (about 51 ms exact, 40 ms lossy at 1-2 rows, against a memory floor near 15-20 ms) [K, F:2026-11-07_2355].

4.4 The exact default and its levers

Every lever was adopted on "every token equal to the reference" plus a speed line written before the run. The first five rows give each lever's effect as measured then, in its own run's metric (often a geo mean or the steady rate, before the 7 Nov correction in section 10); they do not compare with Table 6.1.

lever	what it does	measured effect	source
sync-free expert gate	expert ids stay GPU data; the GPU waits on slots the loader publishes, no CPU-GPU sync per layer	warm check + commit 232.6 -> 117.3 ms; +35 %	T489, T476
confidence cut at 0.3	section 4.3	+19 %	T309
out-of-order experts	resident experts compute first, missed ones as their reads land, in a fixed bitwise order	+13 to +18.7 %	T14, T490
per-layer pool sizes	section 4.2	+8.3 %	M-17
drafter by task	MTP head on chat	chat +24.5 % (time-weighted) on 19 fresh chat prompts	T782
SSD keepalive	section 8.1	about -3 ms per check	BM:docs/NIGHT_1104.md:37
matrix-unit experts on native records (NAX6-N)	section 3.1	+6.2 % whole decode, pooled (44.74 vs 42.12 tok/s, 13 prompts, C 192; +6.7 % on the steady rate, information only)	LP:data/natfri/natfri_pooled.txt; LL, 'NATIVE-FRI run 2'
cache keep	the loop no longer clears the kernels' per-weight caches before every check	-7.2 to -8.0 ms per check; x1.06 whole decode on the lossy config, pooled (x1.10 on the steady rate, information only); tokens equal	F:2026-11-07_0144; BM:data/r57/sc{b,k}{1,2}_256_runs.json
wired limit	section 4.2	removes a ~250 ms first-check stall per prompt	F:2026-11-07_0900
the exact-safe bundle (EB)	a bundle of kernel changes (seams a-e, DQ1; [KERNELS]), bitwise	+3.7 % HumanEval, +2.9 % MBPP (pooled)	F:2026-11-07_1735; Appendix E
coherent reads	GPU reads of loader-written memory made coherent	exact chat 3 / 12 -> 12 / 12, no cost	F:2026-11-08_0750
the 1-row check	a 1-row form of the check	bitwise (676 / 676 logits hashes); +1 % exact chat, +4.2 % lossy chat	F:2026-11-08_0750

4.5 Prefill

Today's prefill runs the prompt in 1,024-row chunks and streams each chunk's experts from the SSD, so a long prompt reads the expert store several times (430 GB for a 7.9K-token prompt). Layer-major prefill runs each layer over the whole prompt and reads each expert about once (68 GB), exactly: 1.9-3.4x faster at 1.6K-7.9K tokens (section 6.2) [K, F:2026-11-08_0800]. On short prompts the first token is still 2.7-4.6 s (section 6.3), except on chat with PF-D.

PF-D builds a short prompt's routed experts from the copies already in memory and reads only the missing parts from the SSD, instead of re-reading every routed expert (20.4 -> 7.5-9.6 GB per chat prompt [K]); its version 4 also reads the next layer's missing experts during the current layer. Both follow Strata's prompt-path

design (read only what memory lacks; read the next layer's misses during this layer); no Strata code is included ([RELEASE] section 7). PF-D is exact: on the shipped chat configuration its tokens (12 / 12) and logits hashes (721 / 721) equal the same configuration without it, and a check arm on the exact configuration (16 tokens per prompt) found all 87,912 staged rows equal to the reference rebuild and its tokens equal to one-token decoding on 12 / 12 prompts [K, BM:data/pfd/pfd_report_0009.txt]. It ships on for chat at 45 GB only (Table 6.1c); every other row of section 6 was measured without it, and it was not run at 32 GB.

5. The lossy configuration and its certificate (method in [EVAL])

5.1 What it changes

The certified configuration (snapshot d8eb2e4; its full switch list is `BM:data/stack/cert_env.txt`) keeps the check loop and changes what some rows compute. Grouped by effect:

- **Lossy on code sets only (HumanEval, MBPP):** QOM-KEEP: on a drafted row, ranks 0-1 always run; a rank 2-9 expert runs only if the check loads it anyway or it is already resident, and otherwise drops out at weight 0, with no rerouting; the kept experts of drafted rows run from the 3-bit part (U3). Section 8.2 has the evidence.
- **Lossy everywhere (bits change on every set):** 8-bit hyper-connection mixers read in one pass on the matrix units (HC8); a split of K for narrow dense matrix-vector calls (DN-KS, rounding-level); single-call attention; small weights in the native layout; the matrix-unit expert form of the above. Single-call attention is bitwise below 512 keys.
- **Acceptance:** exact everywhere, except inside '#' comments of code answers, where a draft ranked first or second with at least a quarter of the top token's probability is kept (section 8.3).
- **Drafting:** one extra draft position so that the bonus row is gated too (DRAFT17); 17-32-row drafter projections on the matrix units; a drafter head over a reduced vocabulary; a copy check that continues a verbatim copy of the prompt in the draft (2,888 copied tokens accepted on 12 HumanEval prompts; F:2026-11-07_0025). It only drafts: the longest 8-16-token match in the committed context replaces drafts while a per-session duel says the copy wins at least 60 % of its disagreements with the drafter; the check still decides every token; a row whose draft the copy changed never counts as sure for the lossy row rules; it is off on math [K, code read: BM:scripts/r57_screen.py, R57_COPY=g2].
- **Exact plumbing:** lazy GDN commit, submitting work every two layers, the cache keep and the wired limit.

5.2 The certificate [K]

Against NAX6-N, forward and reverse prompt order (EV:RESULT_cert, 2cfed64; results/cert/CERT_VERDICT.txt):

set	items	forward	reverse
HumanEval+	164	90.2 vs 92.7 % (-2.44, p 0.125)	92.7 vs 92.7 % (0.00)
math	100	86.0 vs 86.0 %	84.0 vs 86.0 % (-2.00, p 0.625)
MBPP	100	76.0 vs 79.0 % (-3.00, p 0.375)	81.0 vs 79.0 % (+2.00)
IFEval (chat)	100	86.0 vs 87.0 % (-1.00)	85.0 vs 87.0 % (-2.00, p 0.754)
pooled: HumanEval+, math, MBPP (728 pairs; IFEval not pooled)		-0.96 points, p 0.265: PASS (bar: above -2 points and p >= 0.05)	
long code answers	41 rows, 55,966 tokens	0 blunders: PASS	

- Every bar was registered before any row ran (PREREG_cert and amendments 2-3).
- **What it does not cover.** Multilingual benchmarks were not part of it; the claim is "same scores on these four English sets". It covers this configuration (CERT) only. L, which adds the exact-safe bundle (Table 6.1c), was never gated by itself; the configuration we ship, L + coherent reads, passed its own gate on the model card's benchmarks on 9 Nov (Table 6.1b).
- **Chat is lossy in this configuration,** although chat uses exact acceptance: only 1 of 12 IFEval rows (CERT; L: 6 of 24, N45) equal one-token decoding, because the "lossy everywhere" parts change bits on every set [K, F:2026-11-08_0455; F:2026-11-07_2330]. A bisect on chat, one part at a time on the exact base, found two besides the 8-bit hyper-connection read (lossy by design): the one-call expert path (3 and 4 of 12 equal; nondeterministic until coherent reads) and the native layout for small weights (3 of 12); every other part kept 12 of 12 [K, BM:data/chs/chs3_report.txt].

5.3 The chat lever and its card gate

M1-B extends QOM's keep rule to the MTP head's chat drafts. With coherent reads it decodes 49.8 tok/s on chat, +13 % over L's chat runs (Table 6.1c; 44.2 tok/s in CHS4, an earlier run; expert misses per check about 92 -> 25) [K, Appendix E; BM:data/chs/chs4_report.txt; F:2026-11-08_0750]. Its IFEval gate passed (86 vs 87 forward, 83 vs 87 reverse; EV:RESULT_chat_m1b_2026-11-08). On a multilingual chat set it produced 334 low-probability tokens per 10,000 (a token-level line, information only) [K, same]. Its release gate is the card gate: the model card's benchmarks (IFBench, GPQA Diamond, LiveCodeBench v6) against the exact reference in both orders. It passed on 9 November (card arm m1b, 6 of 6 lines): M1-B + COH is "lossy, same scores on IFEval, IFBench, GPQA Diamond and LiveCodeBench v6 (both orders)" [K, EV:RESULT_card_m1b_2026-11-09; [EVAL] section 4.5].

---

6. Results

6.1 The 45 GB line: whole decode, whole request, first token

Setup for every row: 12 prompts per set (HumanEval and MBPP: the S5 ids; IFEval, our chat set, and math (MATH-500): the N45 ids), greedy, at most 256 new tokens, EOS stop, expert cache C 256, quiet machine (GPU lock), two passes unless noted. Whole decode and whole request are pooled over prompts and passes; first token is the median over prompts and passes (min-max) and includes prefill and setup. Every row is re-derived from the raw run records (`BM:data/r57/<tag>_256_runs.json`) by `MS:papers/main_table.py`; Appendix E lists the run reports' geometric means beside it.

Table 6.1a. EXACT: token-identical to one-token greedy decoding under NAX6-N [K].

set	config (run, tags)	whole decode tok/s	whole request tok/s	first token s	lifetime peak GB	runs == one-token decoding	source
12 HumanEval	EB = exact default + the exact-safe bundle (E5; e5hb1-2)	101.4	32.6	4.41 (3.85-6.23)	42.2 (per set)	24 / 24	F:2026-11-07_1735; BM:data/r57/ e5hb{1,2}
12 HumanEval	E = exact default (E5; e5h1-2)	97.8	31.7	4.59 (3.72-6.26)	42.2 (per set)	24 / 24	same; e5h{1,2}
12 MBPP	EB (E5; e5mb1-2)	70.4	21.7	3.80 (3.32-4.85)	41.9 (per set)	24 / 24	same; e5mb{1,2}
12 MBPP	E (E5; e5m1-2)	68.4	21.8	3.73 (3.26-5.60)	41.9 (per set)	24 / 24	same; e5m{1,2}
12 IFEval (chat)	EB (N45; nie1-2)	37.1	20.8	2.89 (2.23-3.88)	41.7	24 / 24	F:2026-11-07_2330; BM:data/n45/n45_ report.txt
12 IFEval (chat)	E + coherent reads + 1-row check (CHS4; yes1, one pass)	38.3	21.3	2.82 (2.34-3.71)	41.7	12 / 12	F:2026-11-08_0750; BM:data/chs/chs4_ report.txt
12 math	EB (N45; nme1-2)	77.9	38.9	2.73 (2.52-6.75)	42.5	24 / 24	F:2026-11-07_2330

- E = NAX6-N kernels + the exact levers (section 4) + cache keep + the wired limit; EB adds a bundle of kernel changes that are bitwise in this mode. "Exact" is to NAX6-N, whose experts run ranks 0-1 at 4 bits and ranks 2-9 at 3 bits.
- E5 prints the lifetime peak per set (both arms together), not per arm.
- Exact chat needed one fix: without coherent reads the one-call path matched one-token decoding on 3 of 12 prompts; with them 12 / 12 in two runs, at no cost (37.8 / 37.9 tok/s) [K, F:2026-11-08_0750].

S5's run records carry no first-token field, so Table 6.1b takes whole request and first token for the certified flags from S5' (same flags, a later snapshot, same sets).

The three lossy configurations at 45 GB. CERT is the certified configuration "v1' + wired" (BM:data/stack/cert_env.txt; section 5). L adds the exact-safe bundle, six switches that are bitwise in the exact mode (BM:data/stack/s5p_env.txt). L + COH adds coherent reads (BM:data/stack/lcoh_env.txt); it is the package's lossy configuration ([RELEASE] section 2). A gate covers only the configuration it tested, so each carries its own label.

Table 6.1b. LOSSY with a passed gate [K]: the configuration we ship (L + COH) and the certified one (CERT).

set	config, run (tags)	whole decode	whole request	first token s	lifetime peak GB	gate	source
12 HumanEval	L + COH as shipped, snapshot rel1 (rlh, one pass)	164.0	36.2	4.60 (3.78-6.07)	42.9	same scores as NAX6-N on IFBench, GPQA Diamond and LiveCodeBench v6, both orders (card arm lcoh)	BM:data/rel/rel_rows_report.txt (53dfff5); BM:data/r57/rlh_256_runs.json
12 MBPP	L + COH (rlm, one pass)	112.7	25.3	3.72 (3.35-4.88)	42.6	same	same; rlm
12 IFEval (chat)	L + COH (rli, rlr: two processes)	45.2	22.4	3.01 (2.41-3.81)	42.4	same	same; rli, rlr
12 math	L + COH (rlt, one pass)	78.9	37.9	3.20 (2.71-6.80)	43.2	same	same; rlt
12 HumanEval	CERT: S5, the certified build d8eb2e4 (s5v1-2)	175.3	not recorded (see S5')	not recorded	42.9	same scores as NAX6-N on HumanEval+, MBPP, math and IFEval, both orders (the certificate)	F:2026-11-07_1430; BM:data/r57/s5v{1,2}
12 HumanEval	CERT: S5', same flags, snapshot 6db4c24 (spb1-2)	171.0	36.0	4.56 (4.01-6.40)	42.9	same	BM:data/s5p/s5p_report.txt; spb{1,2}
12 MBPP	CERT: S5 (s5mv1-2)	111.1	not recorded (see S5')	not recorded	42.6	same	F:2026-11-07_1430
12 MBPP	CERT: S5' (spmb1-2)	114.5	25.8	3.79 (3.43-4.95)	42.6	same	BM:data/s5p/s5p_report.txt

- **The card gate (L + COH)** [K, EV:RESULT_card_lcoh_2026-11-09]: the model card's IFBench (150 keys), GPQA Diamond (100) and LiveCodeBench v6 (131) against NAX6-N in forward and reverse order: deltas -3.33 / -3.33, +3.00 / +3.00 and -1.53 / -3.05 points, exact McNemar p 0.29-0.69; the rule is the certificate's ([EVAL] section 4.5). These are not the four sets of the speed rows.
- **The certificate (CERT)** (details in [EVAL]): HumanEval+ (164), math (100), MBPP (100) and IFEval (100), forward and reverse order, each set within -5 points with exact McNemar p >= 0.05, and HumanEval+, math and MBPP pooled over 728 pairs -0.96 points (p 0.265); 0 blunders in 41 long code answers (55,966 tokens) [K, EV:RESULT_cert (2cfed64); results/cert/CERT_VERDICT.txt; F:2026-11-08_0455]. Multilingual known benchmarks were not part of it.
- **CERT's speed rows are code only.** The IFEval and math rows this table carried until 9 Nov (N45), and the chat runs we compared M1-B with (CHS4), ran L, not CERT [K, BM:scripts/n45.sh l30-31; BM:scripts/chsb.sh l34]; they are in Table 6.1c now.
- **L + COH against L** (Table 6.1c; other runs, not an A/B): whole decode -9.5 % on HumanEval, -5.9 % on MBPP and -9.2 % on math, +4.1 % on IFEval; the cause was not measured [K]. Coherent reads fix the one-call path's stale-slot reads (Table 6.1a's note); without them, chat outputs differ from run to run [K, F:2026-11-08_0750].
- Lossy runs need not repeat token for token: the two S5' passes (CERT) give the same tokens on 8 of 12 HumanEval prompts, because several lossy parts depend on the cache state [K, spb1 / spb2 token files]; this is why the gates use two prompt orders. L + COH repeated its IFEval run token for token (12 / 12, separate processes) and equals one-token decoding on 5 / 12 HumanEval, 5 / 12 MBPP, 2 / 12 IFEval and 5 / 12 math prompts [K, BM:data/rel/rel_rows_report.txt]. We call both "lossy, same scores", never exact.

Table 6.1c. LOSSY: L by itself (not gated) and the chat configurations (status per row) [K].

set	config (run, tags)	whole decode	whole request	first token s	lifetime peak GB	status	source
12 HumanEval	L = CERT + the exact-safe bundle (S5'; spx1-2)	181.3	36.6	4.51 (3.96-6.25)	42.9	not gated by itself: the certificate covers CERT and the card gate L + COH; the bundle is bitwise in the exact mode	BM:data/s5p/s5p_report.txt
12 MBPP	L (spmx1-2)	119.8	25.3	3.76 (3.37-5.40)	42.6	same	same
12 IFEval (chat)	L (N45; nil1-2)	43.4	21.1	3.00 (2.43-3.93)	42.4	same	F:2026-11-07_2330; BM:scripts/n45.sh l30-31
12 math	L (N45; nml1-2)	86.9	39.6	2.96 (2.71-6.98)	43.1	same	same

set	config (run, tags)	whole decode	whole request	first token s	lifetime peak GB	status	source
12 IFEval (chat)	M1-B + coherent reads (CHS5; z0, z0b)	49.8	24.4	2.92 (2.34-3.81)	42.4	same scores on IFEval (86 vs 87 forward, 83 vs 87 reverse) and on the card's IFBench, GPQA Diamond and LCB v6, both orders (arm m1b, EV:RESULT_ card_m1b_ 2026-11-09)	F:2026-11-08_0750; EV:RESULT_ chat_m1b_ 2026-11-08; BM:data/chs/ chat_ttft.txt
12 IFEval (chat)	+ the 1-row check (CHS5; zs, one pass)	51.9	24.8	2.98 (2.51-3.65)	42.4	as M1-B + COH; its tokens equal the first chat row's (z0) on 12 / 12	BM:data/chs/ chs5_report.txt; BM:data/r57/ {zs,z0}_256_ tokens.json
12 IFEval (chat)	chat as shipped: zs + PF-D v4 + read-ahead (section 4.5; pdz4a, one pass)	50.7	28.8	2.26 (1.55-2.59)	43.7	as zs (M1-B + COH); PF-D is exact: tokens 12 / 12 and logits hashes 721 / 721 equal its control without PF-D (pdz0, same prompts: 50.3 / 24.6 / 2.91 (2.34-3.74), 42.4 GB)	BM:data/pfd/pfd_ report_0009.txt (snapshot 36ee120); pdz0: BM:data/ pfd/pfd_report_ 2355.txt (366c452); records BM:data/ r57/pdz{4a,0}_ 256_runs.json
12 IFEval (chat)	+ bundle / HC8_G + rank cut 8 + R1 (CHS5; zsgcr, one pass)	56.0	24.6	2.94 (2.55-3.74)	42.4	ungated	same

- L is lossy on chat and math too: 6 of 24 IFEval and 7 of 24 math runs equal one-token decoding, although those sets use exact acceptance [K, F:2026-11-07_2330].
- The PF-D rows are one pass each, from two runs a quarter of an hour apart. Without the read-ahead (PF-D v3) the median first token was 2.25-2.41 s in three arms over the two runs, so the read-ahead's own share is within the run-to-run spread; it reads more (9.6 against 7.5 GB per prompt) because 19-31 % of what it stages goes unused [K, same reports]. A lossy form (experts whose 4th-bit part is not in memory run at 3 bits) reaches 1.83 s and 30.5 tok/s whole request but changes tokens on 10 of 12 prompts; it is ungated and ships off [K, pdzfa in the same report].

The memory line has two known exceptions [K]. On three long agent prompts (1.6K-7.9K tokens) the certified lossy config peaks at 45.1 GB and the exact config at 44.5 GB (F:2026-11-07_2330, FLAG 2); the layer-major exact prefill of section 6.2 peaks at 45.10 GB (F:2026-11-08_0800). Both need a trim (a smaller C or an earlier scratch release) before they count at 45 GB. No trimmed run was made before the release: on such prompts the engine exceeds the 45 GB line by up to 0.1 GB [K]. Every row of Tables 6.1a-c is at or under 43.7 GB (the PF-D row; every other row at or under 43.2 GB).

6.2 Prefill on long prompts (exact)

prompt (agent set)	today's prefill, 1,024-row chunks: first token (prefill tok/s)	layer-major prefill (exact): first token (tok/s)	SSD read per prompt
1,565 tokens	15.9 s (98) [N45]; 15.2 s (103) [PFX]	8.0 s (195)	109 -> 64 GB
4,730 tokens	40.1 s (118); 37.3 s (127)	12.9 s (368)	265 -> 67 GB
7,932 tokens	64.1 s (124); 61.9 s (128)	18.0 s (441)	430 -> 68 GB

- Layer-major prefill runs each layer over the whole prompt and reads each expert about once; its decode tokens and logits hashes equal today's on every check (8 / 8, 28 / 28, 22 / 22) [K, F:2026-11-08_0800; BM:data/pfx/pfx_report.txt]. Peak: 45.10 GB (see above).
- The certified lossy config prefills like the exact one (16.3 / 41.2 / 66.6 s) [K, N45].
- Larger chunks alone (2,048-8,192 rows) change the logits and are out (section 10). A faster prefill form (PF-B: one MoE call per layer, with read-ahead) is exact since its gate fix: tokens and logits hashes equal the layer-major prefill's (8 / 8, 28 / 28, 22 / 22) at x1.12-1.21 its speed (500 tok/s at 7.9K), but it peaks at 45.8-46.2 GB, over the line, so it does not ship [K, BM:data/pfd/pfd.log part B; [KERNELS] section 5.2].

6.3 Where a request's time goes

- On the four 12-prompt sets the first token takes 43-80 % of a whole request's wall time [P on K: Table 6.1, e.g. HumanEval lossy as shipped 218 tokens per task at 164.0 tok/s = 1.33 s of decode against 6.03 s per request; exact chat 3.79 of 6.77 s is decode]. Decode is no longer the bottleneck at these answer lengths; the first token is.
- Why it is slow on short prompts: a chat prompt's prefill reads about 20 GB of experts, because every prompt token routes 10 experts in each of 48 layers, and it re-reads experts that are already resident; first token = 0.32 s + GB / 7.33 GB/s [P: a least-squares fit on the 12 CT1 rows of BM:data/pfb/pfb_report.txt; section 11].
- PF-D (section 4.5) cuts the shipped chat configuration's read from 20.4 to 9.6 GB per prompt and its first token from 2.91 to 2.26 s; whole request rises from 24.6 to 28.8 tok/s and decode is unchanged (50.3 / 50.7) [K, Table 6.1c]. The fit above gives 1.63 s for 9.6 GB, so with PF-D the reads alone no longer set the first token [P on K].
- What it costs a user is section 9: against an engine with a 0.5-1.0 s first token, a decode up to 3.5x faster (on code) does not pay back within 256 tokens.

7. How the pieces build on each other

Steps 1-5 are in the order in which each needs the ones before it; steps 6 and 7 apply to any configuration. Each effect was measured when its piece was adopted, on the engine of that day and in that run's own metric (section 4.4), so the effects do not add up to Table 6.1, and we give no waterfall. No run removes one piece at a time from the final configuration.

step	piece	what it does	what later steps need from it	measured effect	source
1	batch-invariant kernels (REF2 to NAX6-N; section 3)	a 16-row check computes the same bits as 16 one-token steps	a reference against which any later change is a token-equality test	exactness, audited on every run (section 3.2); the one-token path ran at x0.75 of stock kernels (information only)	EV:RESULT_ref2_2026-10-28; EV:RESULT_nax6_2026-11-05
2	the check loop: a block drafter cut at confidence 0.3, the MTP head on chat (section 4.3)	one pass of the 125B model keeps 2.5-10.7 tokens per check, by task	a pass whose dense and SSD costs every kept token shares; drafted rows that step 5 can treat differently	confidence cut +19 %; MTP head on chat +24.5 % (time-weighted, 19 chat prompts); no A/B against one-token decoding on the final engine	T309; T782; F:2026-11-07_1735; F:2026-11-07_2355
3	the streamed store: aligned records, per-layer pools, rank tiers (sections 4.1-4.2)	the experts fit a 45 GB line; a check reads each missing expert once for all its rows	a loader whose reads can overlap the GPU's work	per-layer pool sizes -17.2 % SSD bytes per kept token, +8.3 % tok/s, exact; rank tiers about -20 % expert bytes (a new reference, certified on scores)	T12; T307
4	hiding the reads: the sync-free expert gate, out-of-order experts in a fixed order (section 4.4)	the GPU computes resident experts while missed ones land, with no CPU-GPU sync per layer	a loop whose per-check fixed costs then show (step 6)	warm check + commit 232.6 -> 117.3 ms (+35 %); out-of-order +13 to +18.7 %; both exact	T489, T476; T14, T490
5	the lossy layer (CERT; section 5)	drafted rows of code answers skip cold lower-ranked experts and run 3-bit parts; some reads at lower precision everywhere	-	CERT 171-175 tok/s on HumanEval and 111-114 on MBPP, against 101.4 and 70.4 for the exact default; the two differ in more than the lossy parts (section 5.1), so this is not an A/B of the layer	Tables 6.1a and 6.1b; Appendix E

step	piece	what it does	what later steps need from it	measured effect	source
6	per-check fixed costs: cache keep, wired limit, matrix-unit experts, the exact-safe bundle, coherent reads, the 1-row check (section 4.4)	removes per-check clears and a ~250 ms first-check stall; faster kernels; coherent reads make exact chat deterministic	-	cache keep -7.2 to -8.0 ms per check; wired limit +25.1 % whole decode (lossy configuration); matrix-unit experts on native records (NAX6-N, later the reference) +6.2 %; the bundle +3.7 % and +2.9 % on code; coherent reads 3 / 12 -> 12 / 12 exact chat prompts; the 1-row check +1 % exact chat, +4.2 % lossy chat	section 4.4
7	prefill from the same store (section 4.5)	PF-D builds a short prompt's experts from the copies in memory; layer-major prefill reads each expert about once	-	PF-D: chat as shipped, first token 2.91 -> 2.26 s, exact; layer-major 1.9-3.4x at 1.6K-7.9K tokens, exact (built, not shipped)	BM:data/pfd/pfd_report_0009.txt; F:2026-11-08_0800

- **Why exactness comes first.** The levers of steps 2-4 and 6 were each accepted by token equality, in minutes; rank tiers, which change bits on purpose, were certified once as a new reference. Without the reference of step 1, each lever would have needed a benchmark gate, as the lossy layer does (sections 3.3-3.4).
- **Why the check loop makes the store affordable.** A check's SSD time follows 10-16 ms plus 0.15-0.17 ms per missed expert [K, T288], paid once per check and shared by the tokens it keeps: 10.7 per check on HumanEval, 6.0-6.1 on MBPP, 6.5 on math and 2.5 on chat [K, F:2026-11-07_1735; F:2026-11-07_2355]. Chat keeps few tokens per check, so its cost is the check's fixed cost (about 51 ms exact at 1-2 rows) [K, F:2026-11-07_2355], which is why the per-check costs of step 6 matter most on chat.
- **Why the lossy layer needs the others.** It decides per drafted row (step 2), reads the 3-bit part that the store already keeps separately (step 3), and is certified against the exact reference (step 1). It pays most on code, where its expert rules apply: the lossy configuration we ship decodes 164.0 tok/s on HumanEval and 112.7 on MBPP against 101.4 and 70.4 for the exact default, 78.9 against 77.9 on math, where those rules are off, and 45.2 against 37.1 on chat [K, Tables 6.1a and 6.1b].
- **Where pieces do not compose.** Exact pieces compose by construction, and the combination is audited on every run (section 3.2). Lossy pieces need not: on chat, relaxed acceptance alone scored IFEval 87 and 8-bit hyper-connection mixers alone 89, both together 84, against 90 for the reference (section 10.6). This is why the lossy layer is gated as one configuration, not part by part.

8. Methods that carry to other models

The engine is not tied to this model. We tested one model, on one machine, so whether the methods below carry is a property of the design, not a measurement.

The second column names the published work each method builds on, with our part where it matters. "Carries": MD = any MoE with a drafter, SD = any speculative-decoding LLM, MAC = Apple's SSD or Macs, AGPU = MoE on Apple's GPU. None of these is specific to this model's GDN or HC layers (INV).

method	builds on	status here	carries	evidence
batch-invariant 16-row kernel set + key buckets	batch invariance (He and Thinking Machines Lab, 2025); ours: the 16-row form on Apple's GPU and matrix units, with key buckets	live, exact	SD, AGPU	section 3; [KERNELS]

method	builds on	status here	carries	evidence
SSD wake penalty + keepalive	NVMe idle power states; keeping drives awake with periodic I/O; ours: the penalty on Apple's internal SSD at sub-millisecond gaps and the read spacing that removes it (section 12.1)	live, exact	MAC	section 8.1
wake-ahead ping	waking a device ahead of predicted I/O (HotOS '15, "Software Engagement with Sleeping CPUs"); ours: one 4 KB read about 200 us before a burst, timed on this SSD	passed in microbenchmarks, not live	MAC	T47, T98
aligned zero-copy reads into GPU slots	direct I/O into device-visible buffers (F_NOCACHE / O_DIRECT; GPUDirect Storage); ours: the measurements into Metal slots on this SSD	live in the store	MAC	NOV, M-29
loader-to-GPU handoff by a coherent spin	MLX's polling-based fast CPU-GPU synchronization fence (MLX pull request #1773, 2025); ours: fresh lines per pass, per-layer events and residency published a pass ahead, between the SSD loader and the expert kernel	live, exact	MAC, AGPU	T76
OS-reclaimable (volatile) expert tier	macOS purgeable memory; ours: holding part of the expert pool in it	live, exact	MAC	T1088
rank-tiered records (4 bits for ranks 0-1, 3 for 2-9)	mixed precision across experts (HOBBIT; MC-MoE); ours: one record split into a 3-bit part and a 4th-bit sidecar, read by rank	live, own reference	MD	T307
out-of-order expert execution in a fixed bitwise order	overlapping expert compute with expert loading (Pre-gated MoE; MoE-Infinity); ours: a fixed summation order that keeps it bit-exact	live, exact	MD	T14
cold ranks half a layer late	KTransformers (used as published)	live at 12 GB	MD	T29
QOM: drafted rows skip the cold experts that would miss	cache-aware expert skipping (Mixture of Cache-Conditional Experts; MoE-Spec); ours: only on drafted rows, only the experts that would miss, no rerouting	certified config	MD	NOV
3-bit kept experts on drafted rows (U3; "drafter-sure rows" in its tested form)	lower precision for experts on demand (SliceMoE); ours: choosing it by drafted rows	certified config (code sets)	MD	section 8.2
union rule keyed on drafter certainty (Q1-UNION)	OEA; PSR; ours: the union keyed on drafter certainty	RELAX B	MD	T103, T993
relaxed acceptance only inside code comments	relaxed acceptance (Appendix I); ours: restricting it to code comments	certified config (code answers)	SD	section 8.3
copy gate: rows inside a verbatim copy of the prompt count as sure	SAM-Decoding; ours: rows inside the copy count as sure for the lossy row rules	RELAX B	SD, MD	EV:PREREG_ relax_2026-11-02
one extra draft position for the bonus row	the bonus token of speculative decoding (Leviathan et al.); ours: gating it with the next segment's first draft	certified config	MD	F:draft17
per-shape K split for narrow matrix-vector calls (DN-KS)	split-K (standard GEMM practice); ours: the per-shape choice on the M5	certified config	AGPU	[KERNELS]
the request's own prompt footprint as the "cold" table	cache-aware expert skipping (as for QOM); ours: the request's own prompt routing as the table	passed on code, not live	MD	NOV; T312

method	builds on	status here	carries	evidence
matrix-unit kernels for padded expert tiles	padded expert GEMMs for few rows (MonoMoE; ONNX Runtime's MoE kernels); ours: the form on the M5's matrix units, bit-exact	live	AGPU	NOV

One idea, reusing the previous position's MoE output, was killed on bytes (section 12.3).

8.1 The SSD wake penalty and the keepalive

- **Builds on.** An NVMe drive enters an idle power state after an idle time and pays a published exit latency on the next request (Microsoft's NVMe power-management guidance for StorNVMe); keeping a drive awake with periodic I/O is old practice (for example Torpid, which writes a small file every 2.5 s); waking a device ahead of predicted I/O is published (HotOS '15). What we found on this MacBook Pro is in section 12.1.
- **Why it works [G].** The penalty depends on the idle gap and not on CPU load (below), so it sits in the SSD path, consistent with an idle timer in the drive or its link. A read every 270 us or sooner keeps that timer from expiring; a single read about 200 us before a burst pays the wake early, off the critical path.
- **Finding.** On this machine's internal SSD, the first read after an idle gap is slower. After gaps of 0.25-1 ms it costs +50-65 us per burst; after 20 ms, +100-120 us [K, T93]. A streamed MoE produces exactly such gaps: one burst of expert reads per layer, with GPU work between bursts. If every layer pays, that is about 3 ms per check at 45 GB [P, T93].
- **A per-call decomposition** puts the fixed cost of one loader call at 187 us of the engine's 283 us: first byte about 119 us, wake 44, dispatch 8, and about 96 us outside the I/O path [K, T97].
- **Fix.** A trickle of 4 KB reads keeps the drive awake. A real spacing of at most 270 us removes the whole +44 us; 400 us or more leaves half [K, T44]. A busy CPU core does nothing, so the effect is in the SSD path [K, T93].
- **In the engine:** a 50 us keepalive, about -3 ms per check on clean prompts [K, BM:docs/NIGHT_1104.md:37], adopted in the exact default as part of the +12.1 % bundle (T10). No isolated A/B of the keepalive exists on the final engine (the engine's owner, 6 Nov), so its share of the bundle is known only from that figure.
- **A cheaper form** is a single 4 KB read about 200 us before a predicted burst. It removes 97.5 % of the penalty in microbenchmarks (100 us ahead is too late, 400 us too early), and 87-94 % at the engine's real gaps [K, T47, T98]. It is not live.
- **What does not work:** pinging the record's own first block to pre-warm it (+0.6 to +3.2 us, T202); splitting one record into parallel reads (each piece pays its own request latency, T199); the page cache (+49 us per call, T201).

8.2 Three-bit top-ranked experts on rows the drafter is sure of (as shipped: on every drafted row)

- **Background.** Ranks 0-1 at 4 bits are load-bearing: all ranks at 3 bits fails both the divergence gate and the same-scores test (`TRIED_SHORT.md`; EV:RESULT_relax_2026-11-05, T3ALL: HumanEval+ 88.3 vs 95.0).
- **The signal.** On rows that verify a draft the drafter was confident about, the LSB sidecar is not read for experts that only such rows use. The rationale [G]: on those rows the large model's top token rarely depends on the 4th bit of its top experts. The evidence is the flip rates below.
- **Evidence [K, decision map].** With the union rule at certainty 0.9: 1.19 / 5.06 / 3.24 % flips on full 16-row checks, under the lines (T103). On the QOM form at certainty 0.5: 0.65 / 4.42 / 3.49 %, PASS on all three; at certainty 0 (the shipped setting): 0.71 / 4.45 / 4.14 %, so code and IFEval pass and multilingual chat is BETWEEN (T106). SSD megabytes per kept token: -46.3 % against -34.9 % for the union rule alone [P, T103].
- **As shipped.** The certainty threshold is 0, so every drafted row counts as sure and every kept expert of a drafted row runs at 3 bits; the bonus row stays exact unless it has a draft. The drafter's certainty enters only through the cut that decides which rows are drafted at all (section 4.3). The "certainty 0" numbers above are the shipped setting's.
- **Status.** Part of the certified configuration on the code sets (section 5): same scores as NAX6-N on HumanEval+ and MBPP in both orders [K, F:2026-11-08_0455].

- **Prior art (as recorded).** Per-token mixed precision exists keyed on the router's rank, and per-expert bit widths keyed on importance; none found keys precision on the drafter's certainty (NOV, row "3-bit records on drafter-sure rows").

8.3 Relaxed acceptance restricted to code comments, and its failure when widened

- **The observation.** In code answers, comment text is a small share of positions and a large share of the drafter's misses: 13.5 % of positions and 44.7 % of misses on one walk (T1350); on another, comments carry 52 % of the first misses in a check [P on K, L350 section 1]. A wrong comment word does not change what the code does.
- **The rule** (EV:PREREG_relax_2026-11-02). On a row whose draft sits inside a '#' comment, the draft is kept if it ranks first or second on its row and has at least a quarter of the top token's probability. Everywhere else acceptance is exact.
  - Only inside the answer's code fence; never in its first two lines; never in docstrings, strings or code.
  - Never on a comment that is or could become a tool directive (type comments, linter pragmas, an encoding line, a shebang).
  - The row must already be inside the comment before its draft, and a draft that starts with a newline stays exact.
  - The span is found by an incremental scan of the answer's text, token by token, so it needs no lookahead.
- **In the engine it is safe so far, and small.** As part of RELAX B it passed decision 4 and the blunder bar on code (0 blunders in 25,656 new HumanEval+ tokens), but it adds only 1.8 % to tokens kept per check on HumanEval+ [K, EV:RESULT_relax_2026-11-05]. A replay had promised x1.07 to x1.16 (T1350). The live confidence cut explains the gap: checks that would have continued through a comment are already cut short [P, L350 section 8.2]. In the certified configuration the rule (rank 1-2, at least a quarter of the top probability) is on, and it fired 0 times on the 12 HumanEval prompts of the stack's run [K, F:2026-11-07_0025].
- **Widened, it fails.** "CD16" keeps the drafter's whole comment and docstring text without checking it, and checks only code. On 12 MBPP tasks it kept x1.45 tokens per check and ran +23.7 % faster, and the tasks' tests passed on 8 of 12 against 11 of 12 without it, in both passes: 3 flips against a line of 2 [K, T855; F:45gb-fast-stack]. The answers show comment loops and code that drifts after the drafter's own comments. HumanEval did not show it, because its docstrings copy the prompt. CD16 was dropped the same day, with the headline number that contained it (section 10.3).
- **The lesson for others:** text that "does not matter" still conditions everything after it. Relaxing the choice among the model's own near-top candidates is a different operation from accepting text the model never scored.
- **Tag.** Relaxed and lossy verification is published; the comment-only span was found in neither of two web checks (NOV; INV section 1): adapted.

---

9. Comparison with other engines and prior work

Appendix G has our runs of Strata on this machine, Appendix H the published runs on this model, and the prior art is in Appendix I.

- **Same machine, same prompts [K; Appendix G].** Strata 0.1.58-metal (MIT; github.com/Niko1221/Strata), with its 3-bit IQ3_XXS file, experts streamed from the SSD and its MTP layer, decodes 40.6-47.1 tok/s at a lifetime peak of 40.8-41.2 GB, with its first token in 0.54-1.01 s. Against Strata's arm of the same kind, our lossy decode is x2.5-3.5 its decode on code, x1.7 on math and x1.08 on chat; our exact decode is x1.6-2.2 on code and x1.7 on math, and behind on chat (37.1-38.3 vs 40.6). Our first token is 2.7-7.0x slower, so Strata wins or ties every whole request of up to 256 tokens; ours passes it at about 245-260 tokens on code and math, and at about 340 on chat as shipped [P].
- **Published runs [self-reported; Appendix H].** On our machine class, Splash reports 95 tok/s decode and a 1,312 tok/s prefill with a 2.4-bit file. On a larger Mac (M5 Max, 128 GB) with every weight resident, Sushi reports 138-169 tok/s decode with 2.75-bit weights, and MTPLX 79-126 with 4-bit weights. None of them used our prompts.
- **Prior art [Appendix I].** Every piece of section 7 has published roots: batch invariance (Thinking Machines Lab; vLLM and SGLang, on CUDA), drafters and verification, SSD-streamed MoE inference, expert caching and prefetch, mixed precision for experts, fewer experts under speculation, and relaxed

or lossy verification. To our knowledge no published engine for this model combines exact speculative checks, an SSD-streamed rank-tiered store and a certified lossy layer; Strata and Sushi combine several of the pieces.

10. Negative results that teach something

10.0 Headlines we corrected or withdrew

headline as first written	what it was	correction	source
"45 GB FAST v1: 180.3 tok/s" (and 144.9 for the stack before it)	the registered "steady" metric: it timed part of each check and skipped each prompt's first check (~300 ms) and 2.35-4.7 ms of host work per check	142.8 tok/s whole decode, pooled (the note's 142.1 is a geo mean); steady / whole = 1.09-1.27, growing as checks get faster. Since 7 Nov every headline is whole decode	F:2026-11-07_0500; BM:data/r57/s1n{1,2}_256_runs.json
"45 GB FAST stack: 157.9 tok/s"	included CD16, which accepted the drafter's comments unchecked and failed MBPP	withdrawn (section 10.3)	F:2026-11-06_1550 (WITHDRAWN)
"12 GB exact: 40.8 tok/s in 11.35 GB"	the measured arm was a lossy mode; the verified loop runs about 18 tok/s	withdrawn (section 10.5)	F:2026-11-04_1439 (corrected)
"chat runs exact in the lossy config"	the lossy parts that change bits run on every set	chat is lossy, same scores (section 5.2)	F:2026-11-07_2330
our first Strata comparison (whole request, bit tiers)	mixed aggregations; "our exact mode reads 4 bits"	pooled from raw rows; both our modes use the same tiers	F:2026-11-08_1630 (corrected the same day)

10.0b Recent negative results

- **Larger prefill chunks are not exact.** 2,048-8,192-row chunks change the logits (hash equal on 0 of N checks), change decode tokens at 4.7K and 7.9K tokens, and peak at 46.5-53.8 GB [K, F:2026-11-08_0800]. Part of that run was also hit by an MLX bug (sorted gather_qmm past 32,767 rows when M is not a multiple of 64; MLX #4632; [KERNELS]); the 2,048-row arm was not hit and still changes the logits.
- **One MoE call per layer in prefill (PF-B) was not exact at first** (0 of 7-28 logits hashes equal, +15-22 %) [K, BM:data/pfb/pfb_report.txt]: the shared expert's one-output gate splits its sums by row count ([KERNELS] section 5.2). Kept per chunk, that gate makes PF-B exact (8 / 8, 28 / 28, 22 / 22 logits hashes, x1.12-1.21), but it peaks at 45.8-46.2 GB, over the line, and does not ship [K, BM:data/pfd/pfd.log part B].
- **Never waiting for cold experts, third form:** +6.2 % whole decode, pooled (141.4 -> 150.2 tok/s), against a +8 % line: BETWEEN, not adopted [K, BM:data/r57/s1b{b,n}{1,2}_256_runs.json; F:2026-11-07_0550].
- **Gaps chain across references.** Each step passed its own gate (NAX6 -3.3 points against REF2-T, the lossy config -3.3 against NAX6 on 60 HumanEval+ tasks), and the lossy config ends -8.33 points against REF2-T on the HumanEval+ core set [K, F:2026-11-07_0550, F:2026-11-08_0455]. Gates against the immediate predecessor do not bound drift over several steps.
- **Lossy runs are not reproducible token for token** (8 of 12 prompts equal between two passes; section 6.1); a lossy configuration has to be certified under two prompt orders.

10.1 Never waiting for the SSD: passes the quality screen, loses in the engine

- **Idea.** A missing top-2 expert is read in the background and its term is added one layer late, so the GPU never waits (the deferral family, here for the 45 GB FAST mode).
- **Quality screen: PASS.** Code flips 0.68 % against 0.74 % for its twin at a 208-slot pool (T50).
- **Engine: KILL of the first form.** -2.8 % (153.5 vs 157.9 tok/s in the same run, on the steady metric then registered: section 10.0), although kept tokens rose 4.6 %. The check grew from 68.8 to 75.3 ms (T209).
- **Why.** The form did not cut the wait: the gate service went from 14.0 to 15.6 ms per check, and about 6.5 ms of late work was added in the per-layer pipeline (T209c; the engine's owner, 6 Nov). Its own calls still waited on the loader's release for cold optional experts and cold sidecars (LL, 'NEVER-WAIT v3').
- **Why the price was wrong.** The prediction (+10-15 %) treated 13.5-15 ms of gate service per check as removable waiting. It is a spin bound that out-of-order execution already overlaps with compute.

- **Status.** A revised form that drops the cold optional reads is built and bit-checked. It needs its own quality arm before a speed test, so the family is not closed.
- **Note.** Both arms ran on a stack that still contained CD16 (section 10.3). The comparison is within one run.

10.2 Fewer expert groups on sure rows ("split"): the tile price was 2-3x too high

- **Idea.** On rows the drafter is sure of, compute only ranks 0-1 plus experts the check loads anyway.
- **Quality screen.** PASS on code (0.92 % and 1.31 % flips at certainty 0.99 and 0.9, line 1.34 %), KILL on chat (T553, T552).
- **Engine.** Computed expert groups fell to x0.63 and x0.49, yet check + commit fell only 6 and 5 %, and kept tokens fell 4.2 and 1.7 %: net about 0 to +1.5 % against a priced +11-19 % (T610; F:flat2). On the matrix-unit kernel: -1.3 %, with kept tokens -5.7 % (T744).
- **Lesson.** On this GPU the expert time is mostly per-row work, not per-group reads (11.1 ms at 2 rows, 57.0 at 16, all resident). Counting tiles, groups or bytes overpredicted the gain two to three times. A lossy cut also lowers acceptance, and that loss must be in the price.

10.3 Wholesale acceptance of comments (CD16), and a withdrawn 157.9 tok/s

- The stack measured 157.9 tok/s on 12 HumanEval prompts (passes 157.2 / 158.1) and was written up as a headline. It included CD16. CD16 then failed quality on MBPP (section 8.3), and the number was withdrawn before publication (F:45gb-fast-stack, WITHDRAWN).
- **Lesson.** A lever must be gated on a workload that can expose it. HumanEval could not: its docstrings are copies of the prompt, so accepting them unchecked changes nothing. A guarded form (T16) scored BETWEEN on 12 MBPP prompts (one flipped task) and was not adopted [K, F:2026-11-07_0025].

10.4 The 12 GB FAST mode failed same-scores

- **The mode.** A cheap first pass with only the top-2 experts certifies a token when its top probability is at least theta; other rows are recomputed with the full computation.
- **Result.** At theta 0.8 on code and 0.95 on chat: HumanEval+ 80.0 vs 91.7 (0 / 7, p 0.02) and IFEval 84.0 vs 90.0 FAIL; math passes; memory passes (at most 10.84 GB) (EV:RESULT_t0_2026-11-05). Truncations and loops rose from 13 / 2 to 23 / 7.
- **What it teaches.**
  - The decision map had predicted it: the same setting flips 1.63 % of code rows, over the 1.34 % line (T54).
  - The certificate itself is token-accurate (0-1 divergences in about 3,400 tokens at every theta). The damage comes through the context: the cheap pass also writes the attention and recurrent state that later tokens read (T1265). A certificate on the token says nothing about the state it leaves behind.
- **Status.** Since then theta 0.8 and its theta 0.9 fallback failed HumanEval+ in both prompt orders (83.3 and 86.7 vs 95.0 %; EV:RESULT_t80ck1u_2026-11-07). The code-valid point (theta 0.95, map 1.22 %), modelled at 35.9 tok/s [P, a calibrated replay], measured 27.9 tok/s whole decode, pooled, on 12 HumanEval + 12 MBPP [K, F12R:tab12f]. The mode is labelled "lossy, failed its gate".

10.5 A withdrawn "12 GB exact at 40.8 tok/s"

- A public note claimed 40.8 tok/s, bit-exact, in 11.35 GB. A re-check found that the measured arm was a FAST mode: it commits rows certified by the cheap pass without the full check. It matched the verified loop on 5 of 16 prompts, and the quality screen scores its certificate (top-2 experts, theta 0.8) a code KILL at 1.63 %. The verified loop runs 18.3 tok/s whole decode, pooled, on 12 HumanEval + 12 MBPP (whole request 13.1, first token 4.29 s, lifetime peak 11.14 GB) [K, F12R:tab12e]. The note now carries a dated correction (F:12gb-exact-40tps; F12).
- The same week showed that the verified loop itself is not bit-identical to one-token decoding [K, F12 'What the 12 GB exact loop (arm ex) is exact to']. The word "exact" had been used for three different properties: bit-identical to one-token decoding, verified by the full model, and reproducible.
- **Lesson.** An arm's name is not its definition. The exactness check had been run on 4 of the 16 prompts. An audit over every prompt, as in section 3.2, would likely have caught it before the note went out [G].

10.6 Agent turns, and two lossy parts that compound on chat

- **Agent turns.** Plain FAST wrote 85 low-probability tokens in 19,749 on 17 long agent turns (about 43 per 10,000); 75 of them are inside the written file content. Short edits are clean; long file writes carry the failure (EV:RESULT_agentfast_2026-11-06).
- **Cause.** With QOM-KEEP off, blunders fall from 21 to 4 (forward order) and 18 to 0 (reverse): the cache-dependent keep carries them. Other parts do not move the count (EV:RESULT_agentbisect_2026-11-06).
- **Run-to-run dependence is wider than one lever.** With QOM-KEEP off, the two prompt orders still differ within the first tokens. The check logs show the drafted row counts differing about 27 tokens before the committed tokens do, so the drafts carry history. If any FAST kernel's bits depend on a check's row count, even exact acceptance then depends on the drafts (same source, addendum). The exact base must be identical in both orders. It is: every exact run equals one-token decoding, in any order (section 3.2).
- **Chat.** Relaxed acceptance alone scores IFEval 87, 8-bit HC mixers alone 89, both together 84 against 90 for the reference: a failure neither shows alone (EV:RESULT_relax_drift_2026-11-06, section 3).

10.7 A data-aware 2.4-bit trellis store did 1.6-2.5x the damage its error predicted

- Priced from its weight-error ratio against 3-bit truncation, the store measured much worse on the decision map: 1.45 / 9.54 / 10.44 % flips against lines of 1.34 / 5.42 / 3.69 (F:trellis_mse_miss).
- Two independent derivations agree that these errors cost about 1.7-3.5x more KL divergence per unit of expert-output error than truncation errors do. The mechanism is open.
- **Lesson.** For this model, weight or output error cannot rank a data-aware code against plain truncation without a divergence screen on real prompts.

10.8 A fixed cost that did not fall, and a timing method that was wrong

- With every lossy lever of 6 Nov morning on, the fixed cost per check was still 53.7 ms, against a KILL line of 36 ms ([KERNELS] 6.5). 350 tok/s on code at 12 kept tokens would need about 25 ms [P, F:flat2].
- An earlier fit had put a dependent dispatch at 12-20 us, which pointed at fewer, fused kernels. The direct measurement is 1.6 us ([KERNELS] 6.3); the loss was in narrow kernels. Two rebuilt kernels from that programme (a row-parallel GDN step and a replacement HC kernel) were bit-correct and slower than what they replaced (LL, 'GDN-CHUNK', 'HC-K2C').
- A split that was thought live was inert for a night because another fast path bypassed it (F:flat2, caveat). The engaged-path counters of section 3.2 exist for this reason.

10.9 From the ledger: ideas an outside reader would likely propose

One line each; numbers and sources in `TR:TRIED_SHORT.md` and the full ledger.

- **Look-ahead expert prefetch:** pays at 24 GB (+9-10 %), KILL at 45 GB (wrong guesses evict).
- **mmap or the page cache instead of direct reads:** KILL (mmap about 5x slower).
- **Cache-aware routing or substituting resident experts:** KILL; not exact, and small. Expert independence (Appendix F.6) explains why.
- **Fewer experts per token (top-8):** KILL at the strict gate, with or without renormalisation.
- **All experts at 3 bits; any 2-bit tier:** KILL.
- **Draft trees:** KILL; the best tree is -10 to +2 % against the best chain at equal cost.
- **Wider (32-row) checks:** KILL at today's per-row cost (-1.3 %); they pay only below about 0.9 ms per row (KP).
- **Self-speculation, layer skip, early exit:** KILL; exiting after layer 40 agrees with the full model on 5.7 % of code positions.
- **Exact speculative sampling instead of greedy matching:** KILL; acceptance +0.003 to +0.014.
- **Drafter retrains:** on-policy data +0.13 kept per doubling; 2x capacity +0.18; one reseed alone moves code by 0.16. Only the data slope is above noise (KP).
- **Reusing the previous position's MoE output:** the one fully new idea; KILL on bytes (union only x0.91) (NOV).
- **Matrix units for experts at short checks:** KILL at the 1 Nov operating point (+1.8 %); adopted later, once checks were longer and the records were in the kernel's native layout (T675, T527).
- **Compiling the verify graph:** KILL (-0.7 ms; the graph build was not the cost then).

- **Several streams in one pass:** aggregate throughput only; per stream it is slower than solo (F:union-law).

10.10 How far ahead can a forecast help the expert cache? (CPU replays; measurements, no method claim)

A forecast made 8 checks ahead of the experts each layer will need could steer the expert cache. We measured its ceilings on recorded traces: 176 checks of six prompts, replayed on the CPU through the engine's cache at the 45 GB line (K 266 slots per layer) and the 24 GB line (K 147) [K, FAR:far_ceiling.txt; F:2026-11-04_2210_far-ahead-foresight]. No forecaster was built into the engine, and the GPU test that would score a real far draft did not run.

- **The kept path is not enough.** A perfect forecast of only the rows the checks keep cuts misses by 20.3 % against LRU at K 266 (26.3 % at K 147), at recall 0.68 (0.76): the rows the checks reject hold the other 32 % of the reuse [K, FAR:far_ceiling.txt].
- **Misses are not stalls.** Perfect foresight of every row of the next 8 checks cuts misses by 33.7 % but the layers that wait on the SSD by only 5.4 % (Belady's optimum: 55.3 % and 19.3 %) [K, same file].
- **Tokens do not predict deep-layer routes; states do better.** From the token alone, per-row recall@10 of a row's experts is 0.24-0.28 in layers 8-47 (a table built from the other prompts; about 0.40 from the same token's previous occurrence), and the best honest token map cuts misses by 6.0 % [K, same file]. A ridge map from the 125B's own layer-1 MoE inputs of the future rows, standing in for a far drafter's states, reaches recall 0.70 / precision 0.64 and 14.6 % fewer misses at K 266 (stalled layers -1.4 %), and 11.9 % at K 147 [K, FAR:far_ridge.txt]. Both are the best of 16 settings, each fitted leave-one-prompt-out but picked on the same replay; the 16 span 7.9-14.6 % and 6.2-11.9 %.
- **Prior art.** Draft-fed expert forecasts in the literature look ahead one round (SP-MoE, 2510.10302) or at most 32 tokens (Specter, 2511.14102). The state-to-route map itself is adapted from SP-MoE and DraftExpert (2607.24434).

---

11. The path to more speed

Everything we tried is in the public ledger (FableGod/TRIED): one line per attempt, with its verdict, its source and the wall it hit. Below: the gain of each change outside the shipped configuration (11.1), the walls (11.2), and the path for chat and code per memory line (11.3). **Sure** means measured [K] on this engine, in a run that had the change in it; a lossy change is sure as speed, and its quality only once its gate passes. **Bet** means priced [P] from measured parts, or a judgement [G]. Gains multiply only where they were measured in one run.

Today, as shipped at 45 GB (whole decode, 12 prompts per set): HumanEval 164.0, MBPP 112.7, math 78.9 and chat 50.7 tok/s, against 101.4 / 70.4 / 77.9 / 37.1 for the exact default [K, Tables 6.1a-6.1c].

11.1 If this is done, this is the gain

change	line, workload	gain (whole decode unless noted)	mark	state
L by itself (CERT + the exact-safe bundle, without COH)	45 GB, HumanEval / MBPP	+6.0 % / +4.7 % over CERT, pooled, one series (181.3 / 119.8); the shipped 164.0 / 112.7 is another snapshot	[K, BM:data/s5p/s5p_report.txt]	built; not gated by itself
the chat stack (zs + bundle / HC8_G + rank cut 8 + R1)	45 GB, 12 IFEval	56.0, +12.8 % over the run's first arm; cycle 44.1 ms	[K, BM:data/chs/chs5_report.txt]	built; not gated
zs32 (M1-B + COH + SB1)	32 GB, 12 IFEval	51.3, against 33.8 for the exact e32 and 39.0 for the shipped lossy-32	[K, BM:data/chs/c32_report.txt; BM:data/rel/rel_rows_report.txt]	built; not gated
never waiting for cold experts	45 GB, HumanEval	+6.2 % pooled, under its +8 % line (BETWEEN)	[K, BM:data/r57/s1b{b,n}{1,2}_256_runs.json]	built; not shipped

change	line, workload	gain (whole decode unless noted)	mark	state
PF-D beyond chat; a better next-layer predictor	45 GB, first token	chat 2.91 -> 2.26 s exact (shipped); lossy form 1.83 s; reading only the missing experts ~1.6 s; 19-31 % of what v4 stages goes unused	[K, Table 6.1c]; 1.6 s [P, BM:data/pfb/pfb_ report.txt]	built for chat; not run on other configurations or at 32 GB
exact layer-major prefill	45 GB, 1.6K-7.9K-token prompts	prefill 1.9-3.4x	[K, F:2026-11-08_0800]	built; needs a 0.1 GB memory trim
one MoE call per layer in prefill (PF-B)	same	a further 12-21 %, exact; peaks 45.8-46.2 GB	[K, BM:data/pfd/pfd.log part B]	over the line
prefill reads overlapped with compute	same	~590 / 750 tok/s prefill at 4.7K / 7.9K	[P, F:2026-11-08_0800]	needs building
two-row verify calls on chat (C2c)	12 GB, 12 IFEval	priced x1.10-1.15 over P2'; measured x1.053 pooled (14.46 vs 13.73, share lock); code x0.881 against A' of an earlier run	[P, F12C]; [K, F12L:chw19_run.log]	screened: chat marginal, code killed
less Python on the critical path	12 GB	at most x1.09	[P, F12P]	test designed, not run
the prompt pass at verify-call bandwidth	12 GB, first token	2.906 s for 63 tokens (21,986 MB at 7.6 GB/s) -> ~1.8 s at ~12 GB/s	[K, F12H]; [P]	needs building

11.2 The limits

Per wall: today's limit; the size of its ledger section (FableGod/TRIED/TRIED.md, one wall per line, read on 9 Nov

2026) and the best result; what breaking it unlocks, and which wall then binds; what could break it.

wall	today's limit	ledger lines; best result	unlocks; then binds	what could break it
fixed cost of a check	chat ~51 ms exact / ~40 ms lossy at 1-2 rows, against a memory floor of ~15-20 ms [K; floor P; F:2026-11-07_ 2355]; the fixed part is ~75-85 % of a chat check [P, [DRAFTER] App. C row 23]	315; warm check + commit 232.6 -> 117.3 ms (T489, T476); chat cycle 48.5 -> 44.1 ms (CHS5) [K]	~2x in the low-row check alone [P]; then kept tokens per check	low-row kernels at bandwidth rather than latency; fewer dependent kernels per layer ([KERNELS]) [G]
kept tokens per check	chat 2.5, math 6.5, HumanEval 10.7 per check [K, section 4.3]	218; 41 of [DRAFTER]'s 83 items killed; best: the MTP head on chat, +24.5 % [K]	a drafter that never missed: ~105-115 tok/s of chat at today's check cost [P, [DRAFTER]]; then the fixed cost	a chat head trained on ~1.3M tokens of the model's own answers (8 % of that gave x1.014, under its x1.03 line [K, [DRAFTER] 3.1]); Strata's MTP layer accepts 87 % of its drafts [K, F:2026-11-08_1630]
SSD bytes per kept token	45 GB code: 18.5 ms cold-minus-warm per 16-row check [K, F:2026-11-07_ 0025]; chat misses ~4 ms at 1-2 rows [K, F:2026-11-07_ 2355]; 12 GB chat 449-491 MB per kept token [K, F12L:chw19_run.log]	303 SSD + 168 bytes; per-layer pools -17.2 % bytes, +8.3 % tok/s, exact; rank tiers about -20 % expert bytes [K, section 7]	at 45 GB code, at most the 18.5 ms per check; then the fixed cost	eviction nearer the offline optimum (OPT: 55.3 % fewer misses and 19.3 % fewer stalled layers than LRU at K 266, 45 GB; section 10.10) [K]; next-layer read-ahead (31-40 ms loader-idle per 12 GB call [P, F12P])
the memory line	rows at 41.7-43.7 GB, at least 1.3 GB under; C 224 -> C 256 +0.1 % [K, F:2026-11-07_0500]	101	little decode at 45 GB; PF-B, 0.8-1.2 GB over the line [K]; at 32 / 24 / 12 GB memory sets the cache, so bytes per kept token bind	trimming PF-B's peak; lossy routes that pass their gates [G]

wall	today's limit	ledger lines; best result	unlocks; then binds	what could break it
first token and prefill	chat as shipped 2.26 s, reading 9.6 GB per prompt with PF-D [K, BM:data/pfd/pfd_report_0009.txt]; without it a 37-101-token prompt reads 20.4 GB, though 65.5 % of the routed 3-bit parts are in memory [K, BM:data/pfb/pfb_report.txt]; long prompts compute-bound, SSD at 3.8-5.2 of 14 GB/s [K, F:2026-11-08_0800]	41; PF-D 2.91 -> 2.26 s exact; layer-major 1.9-3.4x [K]	~1.6 s [P]; Strata's 0.54-1.01 s on the same prompts shows more is reachable here [K, section 9]	PF-D everywhere, a better predictor, reads overlapped with compute [P]
quality gates	lossy parts need not compose: relaxed acceptance (IFEval 87) and 8-bit mixers (89) together scored 84, against 90 [K, section 10.6]; CD16 failed MBPP; the 12 GB FAST mode failed same-scores [K, sections 10.3-10.4]	74 quality + 74 exactness	the ungated rows of 11.1; at 12 GB the FAST mode's 44.34 on HumanEval only [K, F12R:c1u16]	lossy rules that pass the gates, gated together [G]

11.3 The path

Today, then 11.1, then each wall. Measured steps are sure; priced steps are bets.

- **45 GB, code (HumanEval / MBPP).** Today 164.0 / 112.7 [K]. Measured, not gated: L by itself, 181.3 / 119.8 [K]. A bet: never waiting for cold experts, +6.2 % on an older configuration (BETWEEN) [K], never run with L. Past that, the SSD wall's room is 18.5 ms per check [K]; nothing further is priced.
- **45 GB, chat.** Today 50.7 [K]. Measured, not gated: the chat stack, 56.0 [K]. Bets: the check at its floor, about 2x in the check alone [P]; a drafter that never missed, ~105-115 tok/s at today's check cost [P]. 150 tok/s needs both walls at once: a ~25 ms cycle and ~3.75 kept per check [P, F:2026-11-07_2355]. First token: 2.26 s today [K], 1.83 s with the lossy form [K], ~1.6 s priced [P].
- **32 GB, chat.** Today 33.8 exact and 39.0 lossy as shipped [K]. Measured, not gated: zs32, 51.3 [K].
- **24 and 12 GB.** The 24 GB line's rows and its lossy chat route are in the forthcoming [SMALL-RAM]. At 12 GB, code today is 18.34 tok/s verified [K, F12R:tab12e]; the FAST mode's 44.34 (HumanEval only) failed its gate [K, F12R:c1u16], so it is a bet until a lossy rule passes. Chat today is 12.76 verified, 13.97 with P2' (not gated) [K, F12R:chat12, chw14c2]; two-row calls measured x1.053 over P2' (priced x1.10-1.15). Bets, not multiplied: less Python, at most x1.09 [P]; the prompt pass, 2.9 -> ~1.8 s [P]; eviction, bounded by OPT [P].

What is sure and what is a bet. Sure: the shipped rows, and the ungated rows of 11.1 as speed. Every step past a wall is a bet: no wall in 11.2 has been broken in a run, and on chat two must fall together for 150 tok/s. Effort so far: 315 ledger lines on the fixed cost, 471 on SSD bytes, 41 on prefill.

Not measured at all. Concurrent requests (Splash reports 171 tok/s aggregate with 4 concurrent requests on this machine class, self-reported), answers longer than 256 tokens, and prompts longer than 7.9K tokens.

12. What is new in this work

To our knowledge, none of these findings has been published before. For each we give what we found, how we found it, the evidence, why it happens (what is measured [K] and what is still a hypothesis [G]), what it gives, how it could be improved, and where else it could apply. One of the four is an idea that did not pay off (12.3).

12.1 The SSD's wake penalty at sub-millisecond idle gaps

What we found. On this MacBook Pro's internal SSD, the first read after an idle gap of 0.25 ms or more is slower. A streamed MoE runs into this on every layer, because its expert reads come in one burst per layer with GPU work in between. Reads spaced closely enough keep the drive awake and remove the penalty.

How we found it. We timed single reads after controlled idle gaps in micro-benchmarks that load no model, and split the engine's loader cost per call into its parts (section 8.1).

Evidence. After 0.10 ms of idle there is no penalty, and from 0.25 ms on the full one [K, MS:data/ssdmicro.txt]. It costs +50-65 us per burst after 0.25-1 ms gaps and +100-120 us after 20 ms, and a CPU core kept busy changes nothing [K, T93]. A trickle of 4 KB reads at a real spacing of at most 270 us removes the whole +44 us; 400 us or more leaves half [K, T44]. In the engine, a 50 us keepalive saves about 3 ms per check [K, BM:docs/NIGHT_1104.md:37]; it is part of the exact default's +12.1 % bundle, and no A/B isolates it on the final engine.

Why it happens. Measured: the penalty depends on the idle gap and not on CPU load, and reads that end 0.25 ms or more before the next demand read let the drive doze again [K, MS:data/ssdmicro3.txt]. Hypothesis [G]: a low-power state, entered after about 0.10-0.25 ms of idle, that costs about 45-65 us to leave. Still unknown: which part idles (the NVMe controller, the PCIe link, the SoC's storage controller or the kernel's I/O path), and whether a deeper state follows after 20 ms. The mechanism is documented for NVMe drives in general (idle power states with published exit latencies; Microsoft's NVMe power-management guidance for StorNVMe), and keeping drives awake with periodic I/O is old practice (for example Torpid).

What it gives. About 3 ms per check in the engine (above), within the exact default's +12.1 % bundle; per loader call the wake was 44 us of a 283 us fixed cost [K, T97]. It needs only a trickle of small reads, whose energy cost is not measured.

How it could be improved. A single 4 KB read about 200 us before a predicted burst removes 97.5 % of the penalty in micro-benchmarks and 87-94 % at the engine's real gaps, with far fewer reads; it is built but not live [K, T47, T98]. An A/B of the keepalive alone on the final engine is still missing.

Where else it could apply. Untested: any program that reads small bursts with short idle gaps from Apple's internal SSD, such as database index lookups, game asset streaming or data loaders for training. Tested only on this MacBook Pro.

12.2 Small inputs of MLX custom kernels become constants

What we found. MLX's custom-kernel function builds a kernel's signature from the arrays it is called with: a small input is declared `const constant`, a larger one `device`. A kernel written for device pointers then fails to build when it is called with a small placeholder, and a kernel that polls a small input never sees later writes to it.

How we found it. A one-element placeholder input broke the build of a prefill kernel in a run; a probe then varied the input's size ([KERNELS] 5.10).

Evidence. A one-element placeholder was declared `const constant uint32_t*` and the build failed; a 1,024-element placeholder was declared `device`, and the run then matched the path without the kernel (tokens 12 / 12, logits hashes 721 / 721) [K, [KERNELS] 5.10, probe R32; BM:data/pfd/pfd_report_0009.txt line 16]. The polling case, a spin on an array under 4 KB that never released, is in the engine's notes; that run was not saved [BM:FINDINGS.md section 4].

Why it happens. Hypothesis [G]: Metal lets data of 4 KB or less be passed by value and copied when the command is encoded, and small arrays take that path. Still unknown: where MLX sets the size cut ([KERNELS] 5.10).

What it gives. No speed by itself: it turned a build failure into a working kernel. With a 1,024-element placeholder, the PF-D kernel builds and its tokens and logits equal the path without it (12 / 12, 721 / 721) [K, [KERNELS] 5.10], and PF-D brings the shipped chat configuration's first token from 2.91 to 2.26 s (section 4.5). It also argues against polling a small input as a CPU-GPU handoff.

How it could be improved. The engine now tests its kernels with its own placeholders [K, BM:scripts/pfd_unit.py line 147]; where MLX sets the size cut is still to be found in its source ([KERNELS] 5.10).

Where else it could apply. Anyone who writes MLX custom kernels that poll or share state with the CPU, in any field. Tested only in our engine.

12.3 Reusing the previous position's expert output: killed on bytes

What we found. On rows the drafter is sure of, the middle layers' routed-expert output barely changes from one position to the next. Reusing the previous position's output, as a video codec reuses a frame, flips few greedy tokens in layers 12-35. It did not pay off: the experts it saves are not the ones that cost SSD reads. Router-side reuse (for example OEA) changes which experts run, and reuse across the steps of one position is known for diffusion and video models; this reuses an expert's output across adjacent positions.

How we found it. In the decision map, a replay that measures how often a change flips the model's greedy token, reuse was tried per block of 12 layers, then chained as an engine would run it.

Evidence. Flips on drafter-sure code rows: 0.14 % (layers 12-23) and 0.10 % (24-35), against 22.7 % (0-11) and 18.9 % (36-47); layers 12-35 together 1.03 %, between the pass and kill lines; the chained form an engine would run, 8.11 % on code, a kill. The union of experts a check reads shrank only x0.91, so the idea was killed on bytes. Reusing attention or GDN outputs instead flips 29.0 % / 88.5 % [K, the ledger FableGod/TRIED/TRIED.md; the decision map's summaries NIGHT1_SUMMARY.md and NIGHT2_STAGE1.md, not in the public snapshot].

Why it happens. Measured: the middle layers' expert outputs change little between adjacent sure positions, while the first and last 12 layers' do not. Hypothesis [G]: middle layers carry slowly changing context, while early layers track the current token and late layers its prediction. The chained form fails because each reused output feeds the next position's state (as with the FAST mode, section 10.4) [G].

What it gives. Nothing in this engine. The union of experts a check reads shrank only x0.91, about 9 % fewer [P], while the chained form flipped 8.11 % of code rows [K]; SSD bytes bind here (section 11.2), so it was killed. As a negative result it saves others the experiment.

How it could be improved. The per-block form flips only 0.14 % (layers 12-23) or 0.10 % (24-35), one block at a time (1.03 % for both together) [K]; where compute rather than SSD bytes binds, for example with every expert resident, it is the form to test [G]. Not tested.

Where else it could apply. Untested: temporal reuse of conditional-compute outputs in any sequence model with routed layers, such as speech, video or recommendation.

12.4 The M5 matrix unit's fp32 sums match no fixed order

What we found. The M5 GPU's matrix unit accumulates fp32 products in an order that no sequential, pairwise or blocked order reproduces, so an ordinary (ALU) kernel cannot stand in for a matrix-unit kernel bit for bit.

How we found it. An adversarial probe, after the method Fasi et al. (2021) used for NVIDIA's tensor cores, compared the matrix unit's outputs with every fixed summation order we wrote ([KERNELS] 5.6).

Evidence. 1,024 adversarial tests x 512 outputs: the best fixed order matches 45-45.6 %; against the exact sum, 45 % are bitwise, 39 % are 1 ulp off, 14 % 2-8 ulp and 2.4 % more; integer operands match 100 %; the relaxed-precision flag changes nothing [K, [KERNELS] 5.6, probe R12; LP:scripts/nax_order_probe.py].

Why it happens. Measured: integer sums match, so the difference lies in floating-point rounding during accumulation. Hypothesis [G]: the unit adds partial products in a tree with intermediate rounding, or with wider internal accumulators, that no software order reproduces.

What it gives. It is why the exact reference runs on the matrix unit itself (NAX6-N, section 3.1) [G]; that move also brought +6.2 % whole decode (44.74 vs 42.12 tok/s, 13 prompts) [K, section 4.4; LP:data/natfri/natfri_pooled.txt]. It rules out an ALU fallback that is bitwise to the matrix unit.

How it could be improved. A model of the unit's accumulation, as Fasi et al. (2021) built for NVIDIA's tensor cores, would tell an ALU kernel how to match it; not attempted [G].

Where else it could apply. Anyone who needs bitwise reproducibility on Apple's matrix units: scientific computing, deterministic training and inference, verification of numerical code. Tested here only at this engine's shapes.

Appendix C. How the results were produced

- Several sessions worked in parallel on one machine, each owning a part (the engine, kernels, the drafter, the evaluation suite, SSD measurements, the 12 GB line, idea screening). GPU time was serialised by a lock; timings that are compared were taken on a quiet machine.
- Every test has PASS and KILL lines written before it runs; verdicts come from results only. The evaluation suite's pre-registrations and results are dated files (`EvalSuite/docs/PREREG_*`, `RESULT_*`).

- **Software.** MLX pinned to 0.32.2 with the pad-to-64 fix for MLX issue #4632 (sorted gather_qmm past 32,767 rows when M is not a multiple of 64); 0.32.4 changes tokens against our exact references. The bug was reported upstream on 5 Oct 2026, before we hit it on 18 Oct ([KERNELS]). Our engine is released under GPL-3 ([RELEASE]). It builds on the model code of MTPLX by Youssof Altoukhi (Apache-2.0; github.com/youssofal/mtplx), installed from a pinned commit; the release keeps MTPLX's license and NOTICE and shows "Powered by MTPLX" in its startup banner and docs ([RELEASE]). Strata (MIT; github.com/Niko1221/Strata) was run as released in Appendix G; anything taken from it keeps its notice.
- **Memory.** Lifetime peak = the process's maximum phys_footprint (ri_lifetime_max_phys_footprint) in GB of `10^9` bytes; the 45 GB line is 41.9 GiB. It is the engine's own footprint on a 64 GB machine, not an OS limit. Besides it, the runs hold a 7.45 GB volatile expert tier as purgeable memory (section 2).
- **Source keys.** Every source in the text is a key and a path below one of these folders (the data are released with the paper). Paths are relative to the root of the public snapshot, github.com/kamenlevi/350-tps; commit hashes are those of each folder's own repository, and the snapshot holds the files as last committed:

key	file or folder
BM:*file*	the engine's folder (`BigMoE/`)
EV:*name*	an evaluation pre-registration or result, `EvalSuite/docs/<name>.md`; ES:*file* = `EvalSuite/<file>`
F:*slug*	a dated findings note, `FableGod/findings/<slug>.md`
LT:'*heading*'	a section of `Leap/TABLE_350.md`; LL = `FableGod/TRIED/leap.md`
LP:*file*	the kernels' folder, `Leap/<file>`
MS:*file*	`MoonShots/<file>` (tables re-derived for this paper, the related-work check)
SE:*file*	the Strata head-to-head harness and records, `StrataEval/<file>`
T*n*	an entry of the ledger `FableGod/TRIED/TRIED.md` (Appendix D); TR:*file* = `FableGod/TRIED/<file>`
INV / NOV / PA / Q	`NewIdeas/PUBLISH_INVENTORY.md` / `FableGod/TRIED/NOVELTY.md` / `NewIdeas/PRIOR_ART.md` / `NewIdeas/QUEUE.md`
F12	`Fit12/PLAN_12GB_90_fit12.md`, by section heading ('Lever 3 RESULT' when none is given)
F12C	`Fit12/scripts/rows_price.py` (the width-policy replay; inputs chw13 / chw14)
F12P	`Fit12/data/scr/chw16/chw16_prof.txt` and `Fit12/data/scr/chw15/chw15_prof.txt` (reader `Fit12/scripts/chw15_read.py`)
F12H	`Fit12/data/scr/chw18` (chw18_eq.txt, chw18_hist.txt, chw18s.txt)
F12L:*file*	`Fit12/logs/<file>` (the 12 GB track's run logs)
F12R:*tag*	row *tag* of `Fit12/data/paper/small_ram_12gb_rows.txt` (the 12 GB rows, written from the run logs by `Fit12/scripts/paper_rows.py`)
RF	`MoonShots/research/results_first/RESULTS_FIRST.md` (4 Nov; its 12 GB row is the withdrawn 40.8)
KP / KW / L350	`TrainLab/K_PLAN_1106.md` / `FableGod/goal/K_WALL_1106.md` / `FableGod/full/LOSSY350_FULL.md`
FAR:*file*	`FableGod/far/<file>` (far_ceiling.txt, far_ridge.txt: CPU replays of the expert cache, 4 Nov 2026)

- Every idea, with its verdict and deciding number, is one line in a ledger: 1,345 merged ideas on 6 Nov, of which 142 adopted, 267 passed, 608 killed [P: sums of the section tallies in `FableGod/TRIED/TRIED.md`]. The ledger and the novelty record are the release's appendix (INV section 3).

Appendix D. Ledger keys

A key T*n* is a stable label for one entry of the ledger `FableGod/TRIED/TRIED.md`; *n* was the entry's line number in the merge of Fri 6 Nov ~21:00. To find an entry, search the ledger for its ID. The last column is the entry's line when this table was generated on 6 Nov 2026; `check_sources.py -keys` regenerates it.

key	ledger entry ID	line
T7	Concurrent miss loads	7
T10	Item 5	10
T12	M-17 LAYSZ45	12
T14	OOO + TIER_COMBINE	14
T18	R-69b OOO-LIVE	18
T29	B12LL	29
T44	M-28 SSDMICRO keepalive spacing (4 KB pread + usleep(n), the engine's loop	44
T47	M-29 SSDMICRO2 wake-ahead ping	47
T50	NW-BUILD (the build of NEVERWAIT_TEST.md v2; adapted, RE-OP	50
T54	PLAN-12GB-90 (a plan, not an idea)	54
T76	W1-PICKUP	76
T93	M-20 SSDWAKE	93
T97	M-28 SSDMICRO per-call cost decomposition + JOINT in micro	97
T98	M-29 follow-up	98
T103	Q1U-T3	103
T106	QOM-T3	106
T199	M-28 SSDMICRO pieces	199
T201	M-28 SSDMICRO page cache instead of F_NOCACHE for demand records	201
T202	M-31 HEADPING	202
T209	NW-ENGINE	209
T209c	NW-ENGINE	210
T288	FAR-WAIT	289
T307	REF2-T (R-58)	308
T309	t 0.3 truncation	310
T312	5b U3C len	313
T334	P11	335
T462	U-32	463
T476	GATE16 sync-free 16-row gate	477
T478	L1-23 batch-invariant 16-row dense kernels	479
T479	L1-25 experts_nax	480
T489	R-55 GATE16	490
T490	R-56d	491
T495	Spin gate	496
T527	N7-13 NAX6-N	528
T552	SPLIT-ARMS-CHAT	553
T553	SPLIT-ARMS-CODE	554
T565	Item 2	566
T610	SPLIT-ENGINE-OFF	611
T675	K-62j NAX6	676
T744	SPLIT-NAX	745
T782	D-23d CHATSWITCH	783
T855	CD16-DROP	856
T993	LIVE-CUT AUDIT	994
T1011	CF03-EQ-CF0	1012
T1049	WALL-TIEFLOOR	1050
T1057	Fold-fast with the memory fix (b12llff3, decision 5)	1058
T1088	PURGE-1	1089
T1195	Batch-invariant reframing	1196
T1208	R-40 STEP 4b / 4c -> REF2b key buckets	1209
T1209	R-40 INV1	1210
T1210	R-40 REF2	1211
T1214	R-58 REF2-T	1215
T1218	REF2 sdpa_classes	1219
T1219	REF2b	1220
T1225	MMA-tile exactness check (the kernel owner's claim)	1226
T1244	Chunked (WY / UT) GDN verify	1245
T1259	R-56 REF2 live loop	1260
T1265	12 GB lever 1	1266
T1350	COMMENT-LOOPHOLE	1351

Appendix E. Every row, pooled and geo, per pass

`MoonShots/papers/main_table.txt`, generated by `MoonShots/papers/main_table.py` from `BigMoE/data/r57/<tag>_<C>_runs.json` (read-only), as of 9 Nov 2026. An independent re-derivation of every cell agrees.

```
MAIN paper result table, re-derived from the engine's raw run records (read-only): per config and prompt set,
```

45 EXACT E (NAX6-N + levers + KEEP + wired), E5 | humaneval | prompts 12 x 2 pass(es) [e5h1,e5h2] | whole decode pooled 97.8 geo 99.3 (passes 100.0 / 98.6) | whole request pooled 31.7 | first token median 4.59 s (3.72-6.26) | tokens/task 220
45 EXACT E (NAX6-N + levers + KEEP + wired), E5 | mbpp | prompts 12 x 2 pass(es) [e5m1,e5m2] | whole decode pooled 68.4 geo 71.2 (passes 69.9 / 72.4) | whole request pooled 21.8 | first token median 3.73 s (3.26-5.60) | tokens/task 123
45 EXACT EB (E + bitwise Sunday bundle), E5 | humaneval | prompts 12 x 2 pass(es) [e5hb1,e5hb2] | whole decode pooled 101.4 geo 102.7 (passes 102.3 / 103.2) | whole request pooled 32.6 | first token median 4.41 s (3.85-6.23) | tokens/task 220
45 EXACT EB (E + bitwise Sunday bundle), E5 | mbpp | prompts 12 x 2 pass(es) [e5mb1,e5mb2] | whole decode pooled 70.4 geo 73.2 (passes 71.7 / 74.7) | whole request pooled 21.7 | first token median 3.80 s (3.32-4.85) | tokens/task 123
45 LOSSY S5 v1'+wired (= CERT config), S5 | humaneval | prompts 12 x 2 pass(es) [s5v1,s5v2] | whole decode pooled 175.3 geo 175.8 (passes 174.6 / 176.9) | whole request / first token: no pfs in 24 of 24 records | tokens/task 216
45 LOSSY S5 v1'+wired (= CERT config), S5 | mbpp | prompts 12 x 2 pass(es) [s5mv1,s5mv2] | whole decode pooled 111.1 geo 118.3 (passes 118.1 / 118.4) | whole request / first token: no pfs in 24 of 24 records | tokens/task 144
45 LOSSY S5 qall (FAST baseline), S5 | humaneval | prompts 12 x 2 pass(es) [s5q1,s5q2] | whole decode pooled 91.9 geo 92.5 (passes 92.0 / 92.9) | whole request / first token: no pfs in 24 of 24 records | tokens/task 220
45 LOSSY S5 qall (FAST baseline), S5 | mbpp | prompts 12 x 2 pass(es) [s5mq1,s5mq2] | whole decode pooled 65.0 geo 65.8 (passes 66.1 / 65.5) | whole request / first token: no pfs in 24 of 24 records | tokens/task 133
45 LOSSY S5' B (CERT config), S5' | humaneval | prompts 12 x 2 pass(es) [spb1,spb2] | whole decode pooled 171.0 geo 174.0 (passes 168.6 / 179.6) | whole request pooled 36.0 | first token median 4.56 s (4.01-6.40) | tokens/task 217
45 LOSSY S5' B (CERT config), S5' | mbpp | prompts 12 x 2 pass(es) [spmb1,spmb2] | whole decode pooled 114.5 geo 120.1 (passes 118.3 / 121.9) | whole request pooled 25.8 | first token median 3.79 s (3.43-4.95) | tokens/task 130
45 LOSSY S5' L = CERT + the bundle (formerly X), S5' | humaneval | prompts 12 x 2 pass(es) [spx1,spx2] | whole decode pooled 181.3 geo 181.3 (passes 181.7 / 180.9) | whole request pooled 36.6 | first token median 4.51 s (3.96-6.25) | tokens/task 216
45 LOSSY S5' L = CERT + the bundle (formerly X), S5' | mbpp | prompts 12 x 2 pass(es) [spmx1,spmx2] | whole decode pooled 119.8 geo 123.6 (passes 122.5 / 124.7) | whole request pooled 25.3 | first token median 3.76 s (3.37-5.40) | tokens/task 126
45 EXACT EB chat/math, N45 | ifeval | prompts 12 x 2 pass(es) [nie1,nie2] | whole decode pooled 37.1 geo 39.9 (passes 39.9 / 39.8) | whole request pooled 20.8 | first token median 2.89 s (2.23-3.88) | tokens/task 141
45 EXACT EB chat/math, N45 | math500 | prompts 12 x 2 pass(es) [nme1,nme2] | whole decode pooled 77.9 geo 78.8 (passes 79.4 / 78.3) | whole request pooled 38.9 | first token median 2.73 s (2.52-6.75) | tokens/task 256
45 LOSSY L = CERT + the bundle, chat/math, N45 | ifeval | prompts 12 x 2 pass(es) [nil1,nil2] | whole decode pooled 43.4 geo 45.0 (passes 45.2 / 44.8) | whole request pooled 21.1 | first token median 3.00 s (2.43-3.93) | tokens/task 128
45 LOSSY L = CERT + the bundle, chat/math, N45 | math500 | prompts 12 x 2 pass(es) [nml1,nml2] | whole decode pooled 86.9 geo 87.8 (passes 85.3 / 90.4) | whole request pooled 39.6 | first token median 2.96 s (2.71-6.98) | tokens/task 256
45 EXACT E + COH (chat), CHS4 ycoh / ycoh2 | ifeval | prompts 12 x 2 pass(es) [ycoh,ycoh2] | whole decode pooled 37.9 geo 40.1 (passes 39.9 / 40.3) | whole request pooled 21.2 | first token median 2.83 s (2.17-3.79) | tokens/task 141
45 EXACT E + COH + SB1 (chat), CHS4 yes1 | ifeval | prompts 12 x 1 pass(es) [yes1] | whole decode pooled 38.3 geo 40.6 (passes 40.6) | whole request pooled 21.3 | first token median 2.82 s (2.34-3.71) | tokens/task 141
45 LOSSY M1-B + COH (chat), CHS5 z0 / z0b | ifeval | prompts 12 x 2 pass(es) [z0,z0b] | whole decode pooled 49.8 geo 53.7 (passes 53.4 / 54.1) | whole request pooled 24.4 | first token median 2.92 s (2.34-3.81) | tokens/task 145
45 LOSSY M1-B + COH + SB1, CHS5 zs | ifeval | prompts 12 x 1 pass(es) [zs] | whole decode pooled 51.9 geo 55.8 (passes 55.8) | whole request pooled 24.8 | first token median 2.98 s (2.51-3.65) | tokens/task 145
45 LOSSY stack M1-B+COH+SB1+bundle/HC8_G+rank cut 8+R1 (ungated), CHS5 zsgcr | ifeval | prompts 12 x 1 pass(es) [zsgcr] | whole decode pooled 56.0 geo 59.9 (passes 59.9) | whole request pooled 24.6 | first token median 2.94 s (2.55-3.74) | tokens/task 134
45 LOSSY chat as shipped: zs + PF-D v4 + read-ahead, pdz4a | ifeval | prompts 12 x 1 pass(es) [pdz4a] | whole decode pooled 50.7 geo 54.3 (passes 54.3) | whole request pooled 28.8 | first token median 2.26 s (1.55-2.59) | tokens/task 145
45 LOSSY zs, control without PF-D, pdz0 | ifeval | prompts 12 x 1 pass(es) [pdz0] | whole decode pooled 50.3 geo 53.7 (passes 53.7) | whole request pooled 24.6 | first token median 2.91 s (2.34-3.74) | tokens/task 145
45 LOSSY L + COH as shipped (snapshot rel1), REL | humaneval | prompts 12 x 1 pass(es) [rlh] | whole decode pooled 164.0 geo 164.3 (passes 164.3) | whole request pooled 36.2 | first token median 4.60 s (3.78-6.07) | tokens/task 218
45 LOSSY L + COH as shipped (snapshot rel1), REL | ifeval | prompts 12 x 2 pass(es) [rli,rlr] | whole decode pooled 45.2 geo 45.5 (passes 45.8 / 45.3) | whole request pooled 22.4 | first token median 3.01 s (2.41-3.81) | tokens/task 138
45 LOSSY L + COH as shipped (snapshot rel1), REL | math500 | prompts 12 x 1 pass(es) [rlt] | whole decode pooled 78.9 geo 79.9 (passes 79.9) | whole request pooled 37.9 | first token median 3.20 s (2.71-6.80) | tokens/task 256
45 LOSSY L + COH as shipped (snapshot rel1), REL | mbpp | prompts 12 x 1 pass(es) [rlm] | whole decode pooled 112.7 geo 116.0 (passes 116.0) | whole request pooled 25.3 | first token median 3.72 s (3.35-4.88) | tokens/task 127
32 EXACT e32 (C 160), C32 | ifeval | prompts 12 x 1 pass(es) [e32] | whole decode pooled 33.8 geo 34.5 (passes 34.5) | whole request pooled 19.8 | first token median 2.83 s (2.24-3.62) | tokens/task 141

```
32 LOSSY zs32 (C 160), C32 | ifeval | prompts 12 x 1 pass(es) [zs32] | whole decode pooled 51.3 geo 54.0
(passes 54.0) | whole request pooled 22.5 | first token median 2.99 s (2.47-3.84) | tokens/task 124
32 LOSSY L + COH as shipped at C 160 (snapshot rel1), REL | ifeval | prompts 12 x 1 pass(es) [rli_160] | whole
decode pooled 39.0 geo 37.5 (passes 37.5) | whole request pooled 21.1 | first token median 2.90 s (2.37-3.81) |
tokens/task 138
```

Appendix F. Measurements on the Apple M5 GPU (summary; the full form is [KERNELS])

All timings are from one M5 Pro; "quiet" means no other GPU job.

- **F.1 One dependent dispatch costs about 1.6 us** (1.24-1.56 us per kernel in a chain of 200), and a check launches about 1,000-4,000 kernels, so dispatch costs about 1-6 ms per check, not the 20 ms an earlier fit assumed [K per launch, LT:'STEP 0 PART B RESULT'; P for the check; [KERNELS] 6.3].
- **F.2 The time is lost in narrow matrix-vector kernels;** a per-shape K split (DN-KS) recovers much of it, at rounding level ([KERNELS] 6.4).
- **F.3 Single-call attention** (bitwise below 512 keys) and a 17-32-row drafter block brought the 2-row fixed cost from 55.2 to about 42-44 ms in one day [K, LL, 'SDPA-1'; [KERNELS] 4.4].
- **F.4 Where a check's time goes.** With every lossy lever of 6 Nov morning, the fixed cost of a check (the steady 2-row cycle minus twice the warm per-row slope) was 53.7 ms: a KILL against the pre-registered lines (PASS at 30 ms, KILL above 36 ms) [K, F:flat2]. The split by piece is in [KERNELS] 6.5.
- **What it caps [P].** At 12 tokens kept per check, a 54 ms fixed cost alone caps code near 220 tok/s even with free experts. Speed on code is `kept / (fixed + per-row x rows + SSD wait)`; all three terms and the kept count must move together.
- **Kept tokens depend on the workload [K, KP].** 10.36 per check on the engine's 12 HumanEval prompts; on exact walks 9.57 (HumanEval+), 5.94 (MBPP), 7.00 (GSM8K) and 3.01 (agent turns). On the 12 HumanEval prompts at a 256 token cap, 67.8 % of generated tokens are one verbatim copy of the prompt's signature and docstring [K, F:lossy350-thinking-hour; L350 section 1]; MBPP, math and IFEval answers copy 1-3 %. A code tok/s number without its workload is not comparable.

F.5 CPU-GPU visibility on unified memory

- A GPU kernel that spins on shared memory sees CPU writes only after 0.5-1,000 ms with relaxed or device-scope atomics; visibility is guaranteed only at command-buffer boundaries [K, BM:FINDINGS.md section 4]. Small inputs (under 4 KB) are copied at encode time, so a kernel spins on a snapshot forever (from the engine's notes; the run behind it was not saved).
- Two forms work: the CPU releases a layer by writing a fresh region that the GPU polls line by line (never-read lines cannot be stale; the engine's notes give about 69 us per gated layer against about 230 us for a full synchronisation, from runs that were not saved [BM:FINDINGS.md section 4; T495]), and system-coherent loads, which bring the loader-to-GPU pickup to a median of 2.9 us [K, T76].
- In the other direction, device-scope GPU stores reach the CPU only on flush, about 20-23 us after the next dispatch [K, T76].
- Read-ahead hints on the SSD slowed the GPU by 73 % in another group's report, and look-ahead prefetch did not pay at 45 GB here (section 10.9): on this hardware SSD traffic and the GPU share the memory system.

F.6 The model's own structure

- **Experts are independent functions.** On the real weights (three layers): no shared base, no common subspace, no near-duplicate neurons; routed expert outputs on one input are near-orthogonal (mean cosine +0.02). Substituting a resident expert for a missing one equals dropping it [K, F:lossy350-thinking-hour].
- **A union law.** If every row of an R-row check is kept, distinct (expert, layer) groups per kept token fall only about 25 % per doubling of R: 276 / 218 / 163 on code at R 8 / 16 / 32 [P on K routing, F:union-law]. Wider checks cannot make experts cheap by sharing.
- **Chat is an acceptance problem.** At equal check length chat routes fewer experts per token than code. Its cost comes from short checks (about 2-4 rows) and rejected rows [P on K, same].

Appendix G. Same machine, same prompts: Strata (detail of section 9)

Setup [K, F:2026-11-08_1630_strata-head-to-head and its two reports]. Strata 0.1.58-metal (commit 146df3c; MIT; github.com/Niko1221/Strata), its IQ3_XXS file (3-bit, 75.8 GB on disk; huggingface.co/ISTA-DASLab/Qwen3.8-Flash-Next-GSQ-RCO-GGUF; we tested only this file, and its 4-bit files are untested), experts streamed from the SSD, on this machine, the same 48 prompt ids as Table 6.1, greedy, 256 new tokens, EOS stop, one warm process per arm and pass. Arm D reads its hyper-connection weights exactly (BF16), arm B reads them as Q8_0 (lossy, its default); both use its MTP layer. Full cache: arena 33 GB (the size Strata picks on a free Mac), one pass per arm, lifetime peak 41.15 GB (D) and 40.76 GB (B), against our 41.7-43.2 GB.

Table G.1. Whole decode / whole request / first token (median), pooled [K].

set	ours, exact (EB)	ours, lossy as shipped (L + COH)	Strata D (exact HC read)	Strata B (Q8_0 HC read)
12 HumanEval	101.4 / 32.6 / 4.41 s	164.0 / 36.2 / 4.60 s	46.4 / 40.6 / 0.66 s	47.0 / 40.9 / 0.67 s
12 MBPP	70.4 / 21.7 / 3.80 s	112.7 / 25.3 / 3.72 s	43.6 / 35.8 / 0.54 s	45.6 / 36.9 / 0.54 s
12 IFEval (chat)	37.1 / 20.8 / 2.89 s	45.2 / 22.4 / 3.01 s	40.6 / 33.0 / 0.83 s	41.8 / 33.7 / 0.83 s
12 math	77.9 / 38.9 / 2.73 s	78.9 / 37.9 / 3.20 s	46.3 / 39.9 / 1.01 s	47.1 / 40.5 / 1.01 s

- Strata's columns are re-derived from its 96 request records: whole decode = generated tokens / decode time, whole request = generated tokens / request wall time (sent to done), both pooled; first token = the median of sent to first token (all match its report) [K, SE:h2h/results_arena33/{B1,D1}.requests.jsonl]. Its peaks are /usr/bin/time's peak memory footprint, the same counter and unit (`10^9` bytes) as ours.
- Our lossy column is the configuration we ship (Table 6.1b; run 9 Nov on the same prompt ids). CERT decodes faster on code (171-175 and 111-114 tok/s) but has no chat or math rows of its own; L's are in Table 6.1c.
- **Chat as shipped (zs + PF-D v4; lossy, same scores on IFEval and the card's benchmarks; Table 6.1c, a later run, one pass) [K]:** 50.7 / 28.8 / 2.26 s at 43.7 GB, against Strata's 40.6-41.8 / 33.0-33.7 / 0.83 s: decode x1.2 its Q8_0 arm's, whole request behind (x0.85), first token 2.7x slower.
- **Where we win:** decode, each against Strata's arm of the same kind (our exact against D, our lossy against B): x1.6-2.2 exact and x2.5-3.5 lossy on code, x1.7 exact and x1.7 lossy on math, x1.08 lossy on chat.
- **Where we lose:** the first token, 2.7-7.0x slower (medians per set; 2.7x on chat as shipped, with PF-D); every whole-request number up to 256 tokens (Strata wins or ties math); and exact chat decode (37.1-38.3 vs 40.6). Lossy chat as shipped is slightly ahead (45.2 vs 41.8; L's earlier runs 43.4-44.2), and our gate-open chat lever M1-B reaches 49.8 (Table 6.1c).
- **Break-even [P: first-token medians and pooled decode].** Ours passes Strata's whole-request time at about 260 tokens on HumanEval, 245 on MBPP and 255 on math (lossy as shipped), at about 1,200 on chat with it (L's earlier runs: 1,750-2,500), at about 550 on chat with M1-B and at about 340 on chat as shipped (zs with PF-D), each against Strata's Q8_0 arm.
- **Strata's own levers [K, same note]:** its MTP layer gives +53 % decode at its default cache (25.3 -> 38.7 tok/s pooled, all sets; 87 % of drafted tokens accepted); its Q8_0 read adds 2-3 % pooled over its exact read (1-5 % per set); a larger cache (25-28 -> 33 GB) adds 12-23 % per set.
- **Equal memory, our 32 GB line (details in the forthcoming [SMALL-RAM]) against Strata's default cache (peak 33.5-34.9 GB), chat [K]:** ours exact 33.8 / 19.8 / 2.83 s at 30.6 GB (12 / 12 token-identical to one-token decoding), ours lossy ungated 51.3 / 22.5 / 2.99 s at 31.2 GB; the shipped lossy-32 (L + COH at C 160; its card gate ran at C 256 only, and its IFEval tokens equal C 256's on 12 / 12) 39.0 / 21.1 / 2.90 s at 31.3 GB [K, BM:data/rel/rel_rows_report.txt]; Strata 35.8 (exact read) and 37.3 (Q8_0) decode, 28.2 and 29.5 whole request, first token 1.06 s (exact read) and 0.98 s (Q8_0) (BM:data/chs/c32_report.txt; e32, zs32 in Appendix E; Strata's from SE:h2h/results/{B,D}{1,2}.requests.jsonl). PF-D was not run at 32 GB.
- **Caveats [K, same note].** Different files: Strata's 3-bit IQ3_XXS against our tiered store (ranks 0-1 at 4 bits, 2-9 at 3, in both our modes). Quality was not compared between the engines; 2 of 24 rows are token-identical to our exact reference on HumanEval and on chat, none on MBPP or math. Strata's full-cache arms are one pass each (its default-cache passes repeat within 5 %). Long prompts (1.6K+ tokens) were not compared. Our code and math rows predate the layer-major prefill.

Appendix H. Published runs on this model (all self-reported unless noted)

Every row was re-read from its source on 8-9 Nov 2026 (`MS:research/relwork/RELWORK_CHECK.md`). None of these states whole request and first token for the same runs as its decode figure, and none used our prompts, except our own Strata run (Appendix G).

who (source)	expert weights	exactness as stated	memory as stated	hardware	decode tok/s as stated	other
Splash (README "Performance"; release 1.4.0 notes)	Q2_0, 2.4 bits per weight	not stated for these rows	not stated	**M5 Pro, 20-core GPU, 64 GB**	95 (short coding prompts, 1,024-token outputs)	prefill 1,312 tok/s at 32K; cached first token 171 ms on a 32K replay; 4 concurrent: 171 aggregate
Strata, as measured by Splash (same README)	Q2_0	-	-	same	about 73 [P: 95 / 1.3]	-
Strata, our run (Appendix G)	IQ3_XXS, 3 bits	exact or Q8_0 hyper-connection read	lifetime peak 40.8-41.2 GB	our M5 Pro, 64 GB	40.6-47.1 [K]	first token 0.54-1.01 s [K]
Kapitsa (README)	Q2_0, streamed from the SSD	"the big model checks every token"	48 GB machine	M5 Pro, 48 GB	61	also 113.9 greedy on an M5 Max (Nov)
Sushi 1.4.0 (github.com/beamivalice/sushi, README "Benchmarks"; MIT)	STQ trellis quant, 2.75 bits per weight (2 and 4 bpw packs too)	not stated	weights 44.9 GiB, resident ("requires 64 GB+"); its 2 bpw pack streams at 32 GB	M5 Max, 128 GB	138.2-168.6 (4K-128K context); 4 bpw: 122.0-153.5	prefill 1,122-2,049 tok/s; MTP drafting
Mference (github.com/NeelM0906/Mference, README)	not stated	not stated	resident	M3 Ultra	18.2-21.0	prompt 24-210 tok/s
MTPLX 2.11.3-2.12.0 (github.com/youssofal/MTPLX, README; Apache-2.0; our engine builds on its model code)	4-bit (flat, or dynamic with 8-bit attention)	"exact rejection sampling", "correct at any temperature"; token identity with one-token decoding not stated	resident: peaks 78-87 GiB ("96 GB and up", "128 GB and up"); the n-gram table streams from the SSD	M5 Max, 128 GB	79.3 (9K-token code prompt, 1,500 tokens); 125.8 (one agent request, 18.4K of 18.5K prompt tokens cached); 50.3-63.5 at 65K-200K tokens	prefill 1,094 tok/s at 65K; MTP drafting, depth 3
SlotStream (README)	original MLX 4-bit	exact	33 GB budget	M5 Pro, 48 GB	20.91-23.69	-
Evolving Agent Teams (arXiv 2610.36848)	MLX 4-bit, streamed	not stated	48 GB machine	M4 Pro, 48 GB	30.3	-
Palm-Infra (README; docs/ssd-offload.md; Apache-2.0)	NVFP4 experts, CPU SSD offload	not stated	20 GiB expert cache	CPU, eight threads (its docs: a 48 GB Apple Silicon Mac)	21.8, best observed; a controlled four-thread run with a 16 GiB cache: 15.74 (median of five processes)	prefill 16.2 best observed, 18.70 in the controlled run; a CPU-side engine

who (source)	expert weights	exactness as stated	memory as stated	hardware	decode tok/s as stated	other
this work, 45 GB line (Table 6.1)	about 3.2 bits	exact to NAX6-N; lossy as shipped (L + COH): same scores on the card's IFBench, GPQA Diamond and LiveCodeBench v6; CERT: same scores on HumanEval+, MBPP, math and IFEval; chat as shipped: lossy, same scores on IFEval and the card's benchmarks	lifetime peak 41.7-43.7 GB	M5 Pro, 64 GB	exact 37.1-101.4; lossy as shipped 45.2-164.0; CERT 111.1-175.3 on code [K]	first token 2.7-4.6 s; 2.26 s on chat as shipped (PF-D) [K]

- **What this table can and cannot say.** The prompts, answer lengths, files and memory settings differ from row to row. Splash's row is on our machine class with a 2.4-bit file and coding prompts; on those terms it reports a faster decode than our exact mode on code and than every chat number of ours, and a far faster prefill and first token. Our lossy decode on HumanEval and MBPP (113-164 tok/s as shipped) is above every other decode figure from an M5 Pro in the table, but HumanEval flatters (section 6) and no run of Splash on our prompts exists: its figures are self-reported, on its own prompts and file. Sushi's 138-169 tok/s decode and 1.1-2.0K tok/s prefill come from a larger machine (M5 Max, 128 GB) with every weight resident; its 2.75-bit weights alone (48.2 GB) are above our 45 GB line, and its 2-bit pack is the one it streams. MTPLX's 79.3-125.8 tok/s are from the same larger machine with 4-bit weights resident (peaks 78-87 GiB, about twice our line).
- **Other >= 80B MoE rows, re-opened.** Memory-Sovereign Inference (arXiv 2608.23805) streams Qwen3-Next-80B in Q4_K_M with bit-identical logits; its 11.6 tokens/s is "a mechanism diagnostic", not a throughput claim. PagedMoE (arXiv 2611.03173) reaches 8.89 tokens/s on Qwen3.5-397B with a 24 GiB expert cache on a PC with an RTX 5090.
- **Coverage.** Engines and papers are as found by the check of Sun 8 Nov and Mon 9 Nov (`research/relwork`); later releases are not covered.

Appendix I. Prior art

Items are grouped by what we took from them; the full entries are in the References. Every arXiv identifier was checked against its title and authors through the arXiv API on 8-9 Nov 2026, and every web source was re-read then.

Batch invariance and determinism. H. He and Thinking Machines Lab, "Defeating Nondeterminism in LLM Inference", Connectionism, Sep 2025 (doi 10.64434/tml.20250910; batch-invariant RMSNorm / matmul / attention, fixed split sizes); its batch_invariant_ops library (MIT; github.com/thinking-machines-lab/batch_invariant_ops); vLLM's batch invariance mode (VLLM_BATCH_INVARIANT, vLLM docs) and SGLang's deterministic inference (LMSYS blog, "Towards Deterministic Inference in SGLang and Reproducible RL Training", 22 Sep 2025), both CUDA only; LLM-42 (arXiv 2601.17768). Our port and the three MLX-specific breaks are section 3 and [KERNELS]. Our early notes did not cite this work; they should have (PA, row 1).

SSD-streamed MoE inference. Memory-Sovereign Inference (arXiv 2608.23805): output-exact NVMe streaming of an 80B model with byte-identical logits. Public engines: Strata (MIT; github.com/Niko1221/Strata), Splash (github.com/incoai/splash), Kapitsa (github.com/nace-ai/kapitsa), SlotStream (github.com/carloslfu/slotstream), anemll-flash-mlx (slot banks, expert ids as data), Flash-MoE, Palm-Infra (github.com/TencentYoutuResearch/Palm-Infra), Sushi (MIT; github.com/beamivalice/sushi), Mference (github.com/NeelM0906/Mference). Evolving Agent Teams (arXiv 2610.36848); PagedMoE (arXiv 2611.03173). Alizadeh et al., "LLM in a flash" (arXiv 2312.11514). KTransformers (Chen et al., SOSP '25, doi 10.1145/3731569.3764843): Expert Deferral.

Expert caching and prefetch. Pre-gated MoE (Hwang et al., 2308.12066); ProMoE (2410.22134); MoE-Infinity (2401.14361); AdapMoE (2408.10284); HOBBIT (2411.01433); FlashMoE (2601.17063); SwapMoE (2308.15030); Specter (2511.14102); SP-MoE (2510.10302); Fiddler (2402.07033).

Mixed precision for experts. HOBBIT; MC-MoE (2410.06270); bandwidth-efficient adaptive MoE via low-rank compensation (2512.17073); DynaExq (2511.15015); DyMoE (2603.19172); SliceMoE (2512.12990);

Matryoshka Quantization (2502.06786); QTIP trellis codes (2406.11235).

Fewer or shared experts under speculation. OEA, Opportunistic Expert Activation (2511.02237); PSR, Prefetch-Steered Routing (2610.34721); AcceptMoE (2608.02989); MoE-Spec (2602.16052); EVICT (2605.00342); DraftExpert (2607.24434); EcoSpec (2607.12696); Utility-Driven SD for MoE (2506.20675); Lynx (2411.08982); Mixture of Cache-Conditional Experts (2412.00099).

Drafters and verification. Leviathan et al., "Fast Inference from Transformers via Speculative Decoding" (2211.17192; the cost ratio); Chen et al., speculative sampling (2302.01318); DFlash (Chen, Liang and Liu, "DFlash: Block Diffusion for Flash Speculative Decoding", ICML 2026; 2602.06036; github.com/z-lab/dflash, MIT); DFlash2, the base of our fine-tuned block drafter ([DRAFTER]): Inco AI, "DFlash 2: Keep Drafting Parallel" (blog, Aug 2026, inco.ai/blog/dflash2), checkpoint incoai/Qwen3.8-Flash-Next-DFlash2 (Hugging Face; Apache-2.0; a finetune of Qwen/Qwen3.8-Flash-Next; revision a22a174 of 19 Oct 2026; the engine loads it and our fine-tuned weights over it); MTPLX by Youssof Altoukhi (software, v2.12.2 at commit 9882703f, the version we ran; Apache-2.0; github.com/youssofal/mtplx: the model's own MTP head as the drafter on Apple Silicon, with exact rejection sampling; its first backend for this model family is dated 29 Aug 2026; our engine builds on its model code); DFlare (2606.02091); EAGLE-3 (2503.01840); PARD-2 (2605.08632); WhiFlash (2606.07710; switches between an autoregressive and a diffusion drafter token by token); TriSpec (2601.23180); SAM-Decoding (2411.10666); prompt-lookup decoding (github.com/apoorvumang/prompt-lookup-decoding).

Relaxed and lossy verification. MARS (2601.15498); DIVERSED (2604.07622); "Revisiting Lossy Verification" (2607.26627); AdaptiveSpec (2609.02897); the answer-lengthening effect of relaxed rules (2610.23818).

Kernels. MLX (Apple; MIT): issue #4632 (sorted gather_qmm past 32,767 rows, reported 5 Oct 2026) and PR #3922 ("Fix sorted gather_qmm NAX row overflow above 32K", merged 26 Aug 2026); details in [KERNELS]. MonoMoE (2609.04244, padding waste in decode MoE matmuls); ONNX Runtime's MoE small-row path; QuaRot (2404.00456) / SpinQuant (2405.16406) (folding a norm weight into the next linear). Apple describes the M5 GPU as having "a Neural Accelerator in each core" (Apple Newsroom, Oct 2025); how MLX reaches them is in [KERNELS].

Storage. The exit latency of an NVMe link's low-power state (SK Hynix, US 11,789,658 B2); waking sleeping hardware ahead of use (Zhu et al., "Software Engagement with Sleeping CPUs", HotOS '15); direct I/O into device-visible buffers (NVIDIA GPUDirect Storage; BaM, Qureshi et al., 2203.04910).

Benchmarks. HumanEval (Chen et al., 2107.03374); HumanEval+ and MBPP via EvalPlus (Liu et al., 2305.01210); MBPP (Austin et al., 2108.07732); IFEval (Zhou et al., 2311.07911); MATH (Hendrycks et al., 2103.03874) and its 500-problem subset MATH-500 (Lightman et al., 2305.20050); GSM8K (Cobbe et al., 2110.14168).

The model. Qwen3.8-Flash-Next (Hugging Face: Qwen/Qwen3.8-Flash-Next); Qiu et al., "On the Design of Qwen3.8-Next Architecture: Evaluation, Efficiency, and Training Stability" (2608.30320).

Use of AI tools

The code, the experiments and the text of this paper were produced with substantial help from AI coding agents. The author directed the work and is responsible for its content. The numbers were re-derived from the raw run records named beside them, and the references were opened by hand or by script.

References

- Keivan Alizadeh et al. "LLM in a flash: Efficient Large Language Model Inference with Limited Memory." arXiv:2312.11514, 2023.
- Youssof Altoukhi. MTPLX (software, v2.12.2, commit 9882703f; Apache-2.0). https://github.com/youssofal/mtplx
- Zihao An et al. "PARD-2: Target-Aligned Parallel Draft Model for Dual-Mode Speculative Decoding." arXiv:2605.08632, 2026.
- Anemll. anemll-flash-mlx (software). https://github.com/Anemll/anemll-flash-mlx
- Apple. "Apple unleashes M5, the next big leap in AI performance for Apple silicon." Apple Newsroom, 15 Oct 2025. https://www.apple.com/newsroom/2025/10/apple-unleashes-m5-the-next-big-leap-in-ai-performance-for-apple-silicon/
- Saleh Ashkboos et al. "QuaRot: Outlier-Free 4-Bit Inference in Rotated LLMs." arXiv:2404.00456, 2024.

- Jacob Austin et al. "Program Synthesis with Large Language Models." arXiv:2108.07732, 2021.
- Shiyi Cao et al. "Evolving Agent Teams." arXiv:2610.36848, 2026.
- Charlie Chen et al. "Accelerating Large Language Model Decoding with Speculative Sampling." arXiv:2302.01318, 2023.
- Hongtao Chen et al. "KTransformers: Unleashing the Full Potential of CPU/GPU Hybrid Inference for MoE Models." In Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles (SOSP '25), pp. 1014-1029, 2025. doi:10.1145/3731569.3764843.
- Jian Chen, Yesheng Liang and Zhijian Liu. "DFlash: Block Diffusion for Flash Speculative Decoding." arXiv:2602.06036, 2026. ICML 2026. Code: https://github.com/z-lab/dflash (MIT).
- Liangkun Chen et al. "SP-MoE: Speculative Decoding and Prefetching for Accelerating MoE-based Model Inference." arXiv:2510.10302, 2025.
- Mark Chen et al. "Evaluating Large Language Models Trained on Code." arXiv:2107.03374, 2021.
- Yuseon Choi et al. "SliceMoE: Bit-Sliced Expert Caching under Miss-Rate Constraints for Efficient MoE Inference." arXiv:2512.12990, 2025.
- Kexin Chu et al. "Dynamic Expert Quantization for Scalable Mixture-of-Experts Inference." arXiv:2511.15015, 2025.
- Karl Cobbe et al. "Training Verifiers to Solve Math Word Problems." arXiv:2110.14168, 2021.
- Flash-MoE (software). https://github.com/danveloper/flash-moe
- Raja Gond et al. "LLM-42: Enabling Determinism in LLM Inference with Verified Speculation." arXiv:2601.17768, 2026.
- Yu Gong et al. "MonoMoE: An Efficient Fused Mega-kernel for Quantized MoE Decoding." arXiv:2609.04244, 2026.
- Vima Gupta et al. "Lynx: Enabling Efficient MoE Inference through Dynamic Batch-Aware Expert Selection." arXiv:2411.08982, 2024.
- Dengke Han. "DraftExpert: Expansion-Aware Self-Speculative Decoding for End-Device MoE Inference." arXiv:2607.24434, 2026.
- Awni Hannun, Jagrit Digani, Angelos Katharopoulos and Ronan Collobert. "MLX: Efficient and flexible machine learning on Apple silicon" (software, 2023; MIT). https://github.com/ml-explore/mlx
- Horace He and Thinking Machines Lab. "Defeating Nondeterminism in LLM Inference." Thinking Machines Lab: Connectionism, 10 Sep 2025. doi:10.64434/tml.20250910. https://thinkingmachines.ai/blog/defeating-nondeterminism-in-llm-inference/
- Dan Hendrycks et al. "Measuring Mathematical Problem Solving With the MATH Dataset." arXiv:2103.03874, 2021.
- Yuxuan Hu et al. "SAM Decoding: Speculative Decoding via Suffix Automaton." arXiv:2411.10666, 2024.
- Siming Huang. "Prefetch-Steered Routing: Deterministic Cache-Aware Expert Selection for Offloaded Mixture-of-Experts Decoding." arXiv:2610.34721, 2026.
- Wei Huang et al. "Mixture Compressor for Mixture-of-Experts LLMs Gains More." arXiv:2410.06270, 2024.
- Yuegui Huang et al. "DyMoE: Dynamic Expert Orchestration with Mixed-Precision Quantization for Efficient MoE Inference on Edge." arXiv:2603.19172, 2026.
- Ranggi Hwang et al. "Pre-gated MoE: An Algorithm-System Co-Design for Fast and Scalable Mixture-of-Expert Inference." arXiv:2308.12066, 2023.
- Inco AI. "DFlash 2: Keep Drafting Parallel." Blog, Aug 2026. https://inco.ai/blog/dflash2
- Inco AI. Qwen3.8-Flash-Next-DFlash2 (drafter checkpoint, revision a22a174, 19 Oct 2026; Apache-2.0). https://huggingface.co/incoai/Qwen3.8-Flash-Next-DFlash2
- Inco AI. Splash (software, release 1.4.0). https://github.com/incoai/splash
- ISTA-DASLab. Qwen3.8-Flash-Next-GSQ-RCO-GGUF (model files in GGUF format; Apache-2.0). Hugging Face. https://huggingface.co/ISTA-DASLab/Qwen3.8-Flash-Next-GSQ-RCO-GGUF
- Haoyun Jiang et al. "TriSpec: Ternary Speculative Decoding via Lightweight Proxy Verification." arXiv:2601.23180, 2026.
- Keisuke Kamahori et al. "Fiddler: CPU-GPU Orchestration for Fast Inference of Mixture-of-Experts Models." arXiv:2402.07033, 2024.
- Byeongju Kim et al. "FlashMoE: Reducing SSD I/O Bottlenecks via ML-Based Cache Replacement for Mixture-of-Experts Inference on Edge Devices." arXiv:2601.17063, 2026.
- Rui Kong et al. "SwapMoE: Serving Off-the-shelf MoE-based Large Language Models with Tunable Memory Budget." arXiv:2308.15030, 2023.
- Young D. Kwon et al. "WhiFlash: Accelerating Speculative Decoding with Token-Level Cross-Paradigm Routing." arXiv:2606.07710, 2026.
- Yaniv Leviathan, Matan Kalman and Yossi Matias. "Fast Inference from Transformers via Speculative

Decoding." arXiv:2211.17192, 2022. ICML 2023.
- Yuhui Li et al. "EAGLE-3: Scaling up Inference Acceleration of Large Language Models via Training-Time Test." arXiv:2503.01840, 2025.
- Shuang Liang et al. "AcceptMoE: Commitment-Weighted Self-Sizing Verifier Expert Sets for Efficient MoE Speculative Decoding." arXiv:2608.02989, 2026.
- Hunter Lightman et al. "Let's Verify Step by Step." arXiv:2305.20050, 2023.
- Yutong Lin et al. "PagedMoE: Cross-Precision Expert Caching for Token-Level Dynamic Precision in SSD-Resident MoE Inference." arXiv:2611.03173, 2026.
- Jiawei Liu et al. "Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation." arXiv:2305.01210, 2023.
- Zechun Liu et al. "SpinQuant: LLM quantization with learned rotations." arXiv:2405.16406, 2024.
- Zhenyu Liu et al. "Bandwidth-Efficient Adaptive Mixture-of-Experts via Low-Rank Compensation." arXiv:2512.17073, 2025.
- LMSYS Org. "Towards Deterministic Inference in SGLang and Reproducible RL Training." LMSYS blog, 22 Sep 2025. https://lmsys.org/blog/2025-09-22-sglang-deterministic/
- Massimiliano Fasi, Nicholas J. Higham, Mantas Mikaitis and Srikara Pranesh. "Numerical behavior of NVIDIA tensor cores." PeerJ Computer Science 7:e330, 2021. doi:10.7717/peerj-cs.330
- Bradley McDanel et al. "MoE-Spec: Expert Budgeting for Efficient Speculative Decoding." arXiv:2602.16052, 2026.
- Quinn McNemar. "Note on the sampling error of the difference between correlated proportions or percentages." Psychometrika 12(2): 153-157, 1947. doi:10.1007/BF02295996.
- Mference (software). https://github.com/NeelM0906/Mference
- Microsoft. "NVMe" (power management for storage hardware devices: the StorNVMe driver's power states, entry and exit latencies, idle timeouts). Windows hardware design guidance, Microsoft Learn. https://learn.microsoft.com/en-us/windows-hardware/design/component-guidelines/power-management-for-storage-hardware-devices-nvme
- MLX issue #4632, "[BUG] gather_qmm(sorted_indices=True) returns wrong results on M5 Max (macOS 27) when rows > 32768 and not a multiple of 64", 5 Oct 2026. https://github.com/ml-explore/mlx/issues/4632
- MLX pull request #1773, "Faster synchronization `Fence` primitive", merged 18 Jan 2025. https://github.com/ml-explore/mlx/pull/1773
- MLX pull request #3922, "Fix sorted gather_qmm NAX row overflow above 32K", merged 26 Aug 2026. https://github.com/ml-explore/mlx/pull/3922
- mlx-lm (software; MIT). https://github.com/ml-explore/mlx-lm
- nace-ai. Kapitsa (software). https://github.com/nace-ai/kapitsa
- Pranav Nair et al. "Matryoshka Quantization." arXiv:2502.06786, 2025.
- Niko1221 and the Strata contributors. Strata (software, version 0.1.58-metal, commit 146df3c; MIT). https://github.com/Niko1221/Strata
- NVIDIA. Magnum IO GPUDirect Storage. https://developer.nvidia.com/gpudirect-storage
- Costin-Andrei Oncescu et al. "Opportunistic Expert Activation: Batch-Aware Expert Routing for Faster Decode Without Retraining." arXiv:2511.02237, 2025.
- ONNX Runtime (software; MIT). https://github.com/microsoft/onnxruntime
- Lehan Pan et al. "Making Every Verified Token Count: Adaptive Verification for MoE Speculative Decoding." arXiv:2605.00342, 2026.
- Zihan Qiu et al. "On the Design of Qwen3.8-Next Architecture: Evaluation, Efficiency, and Training Stability." arXiv:2608.30320, 2026.
- Zaid Qureshi et al. "GPU-Initiated On-Demand High-Throughput Storage Access in the BaM System Architecture." arXiv:2203.04910, 2022.
- Qwen Team. Qwen3.8-Flash-Next (model weights; Qwen Community License 1.0). https://huggingface.co/Qwen/Qwen3.8-Flash-Next
- Anish Saxena et al. "Utility-Driven Speculative Decoding for Mixture-of-Experts." arXiv:2506.20675, 2025.
- Apoorv Saxena. "Prompt Lookup Decoding" (software), Nov 2023. https://github.com/apoorvumang/prompt-lookup-decoding
- SK hynix. "Peripheral component interconnect express (PCIe) interface system and method of operating the same." US Patent 11,789,658 B2. https://patents.google.com/patent/US11789658B2/en
- Andrii Skliar et al. "Mixture of Cache-Conditional Experts for Efficient Mobile Device Inference." arXiv:2412.00099, 2024.
- SlotStream (software). https://github.com/carloslfu/slotstream. Issue #5, "Measured: Mac14,3, 16GB", 2 Sep 2026: https://github.com/carloslfu/slotstream/issues/5

- Jingwei Song et al. "MARS: Unleashing the Power of Speculative Decoding via Margin-Aware Verification." arXiv:2601.15498, 2026.
- Xiaoniu Song et al. "ProMoE: Fast MoE-based LLM Serving using Proactive Caching." arXiv:2410.22134, 2024.
- Lukas Stepanek. "Memory-Sovereign Inference: Output-Exact Execution Beyond Full Residency." arXiv:2608.23805, 2026.
- Sushi (software, release 1.4.0; MIT). https://github.com/beamivalice/sushi
- Peng Tang et al. "HOBBIT: A Mixed Precision Expert Offloading System for Fast MoE Inference." arXiv:2411.01433, 2024.
- Tencent Youtu Research. Palm-Infra (software; Apache-2.0), including docs/ssd-offload.md. https://github.com/TencentYoutuResearch/Palm-Infra
- Thinking Machines Lab. batch_invariant_ops (software; MIT). https://github.com/thinking-machines-lab/batch_invariant_ops
- Albert Tseng et al. "QTIP: Quantization with Trellises and Incoherence Processing." arXiv:2406.11235, 2024.
- Oszkár Urbán et al. "Margins, Not Windows: Training-Free Per-Step Lossy Speculative Decoding." arXiv:2609.02897, 2026.
- vLLM project. "Batch Invariance." vLLM documentation. https://docs.vllm.ai/en/latest/features/batch_invariance/
- Chiatzen Wang et al. "Acceptance Is Not Speed: Length Inflation in Training-Free Lossy Speculative Decoding." arXiv:2610.23818, 2026.
- Tianyu Wang et al. "Revisiting Lossy Verification in Speculative Decoding: Mechanisms, Trade-offs, and Failure Modes." arXiv:2607.26627, 2026.
- Wenfeng Wang et al. "Specter: Fast MoE Inference via Quantization-Enabled Multi-Step Speculative Offloading." arXiv:2511.14102, 2025.
- Ziyi Wang et al. "DIVERSED: Relaxed Speculative Decoding via Dynamic Ensemble Verification." arXiv:2604.07622, 2026.
- Jincheng Xie et al. "Less Experts, Faster Decoding: Cost-Aware Speculative Decoding for Mixture-of-Experts." arXiv:2607.12696, 2026.
- Leyang Xue et al. "MoE-Infinity: Efficient MoE Inference on Personal Machines with Sparsity-Aware Expert Cache." arXiv:2401.14361, 2024.
- YorkshireDev. Project Torpid: a disk drive sleep prevention utility (software; GPL-3.0). https://github.com/YorkshireDev/Torpid
- Jiebin Zhang et al. "DFlare: Scaling Up Draft Capacity for Block Diffusion Speculative Decoding." arXiv:2606.02091, 2026.
- Shuzhang Zhong et al. "AdapMoE: Adaptive Sensitivity-based Expert Gating and Management for Efficient MoE Inference." arXiv:2408.10284, 2024.
- Jeffrey Zhou et al. "Instruction-Following Evaluation for Large Language Models." arXiv:2311.07911, 2023.
- Qi Zhu, Meng Zhu, Bo Wu, Xipeng Shen, Kai Shen and Zhiying Wang. "Software Engagement with Sleeping CPUs." In Proceedings of the 15th Workshop on Hot Topics in Operating Systems (HotOS XV), 2015.

Companion papers:

- [DRAFTER] Kamen Levi. Drafting for a 125B mixture-of-experts model that does not fit in RAM: training its own MTP head, one drafter per text type, and the levers that failed. 2026. A companion paper of this set.
- [EVAL] Kamen Levi. Certifying lossy decoding on the model's known benchmarks: same-scores gates for a 125B mixture-of-experts model on one 64 GB Mac. 2026. A companion paper of this set.
- [KERNELS] Kamen Levi. Batch-invariant kernels on Apple's M5 GPU and its matrix units. 2026. A companion paper of this set.
- [RELEASE] Kamen Levi. Running a 125B mixture-of-experts model on a 64 GB Mac: the engine package. 2026. A companion paper of this set.
- [SMALL-RAM] Kamen Levi. A 125B mixture-of-experts model in 32, 24 and 12 GB of memory on one Mac. Forthcoming (by 11 November 2026).

Batch-invariant kernels on Apple's M5 GPU and its matrix units

An exact speculative-decoding reference for a streamed 125B mixture-of-experts model, and what MLX 0.32.2 computes at each row count.

Kamen Levi
Independent researcher

9 November 2026

Abstract

Speculative decoding is exact only if verifying k drafted tokens in one batched call produces the same bits as decoding them one at a time. On Apple's M5 GPU with MLX 0.32.2 it does not. The framework picks a different kernel for 1, 2-8, 16 and thousands of rows (vector kernels, split-K kernels, matrix-unit GEMMs), and each rounds differently. With the stock kernels, all 16 rows of a verify pass already differ from one-token decoding at the first matmul of a 125B mixture-of-experts model [K].

We build a batch-invariant kernel set for that model (48 layers, 512 experts per layer, 10 per token; the expert store runs at about 3.2 bits per expert weight):

- dense projections padded to one 16-row shape, on a matrix-unit kernel that multiplies the raw codes;
- routed experts on one 16-row matrix-unit tile per expert, then a fixed split-K over a record layout native to the matrix unit's fragments;
- a hyper-connection read that replicates MLX's own split-K loop bit for bit;
- attention pinned to fixed key buckets and length classes.

Under it, 16 one-token steps and one 16-row verify agree on all 37 records of a forward through a 12-layer slice [K], and full-length audits agree on every kept row ([MAIN] section 3.2). The dense part of a check falls from 56.8 to 27.4 ms [K]. On the expert path the matrix-unit kernel beats our ALU kernel by x1.062 in whole decode in an engine A/B [K]. Batching a prompt's MoE layers over all chunks is exact once two kernels are pinned, and prefills 1.6K-7.9K-token prompts x1.14-1.21 faster [K]. We map which MLX kernels are row-invariant at decode and prefill widths, and report ten MLX findings with reproductions, among them a sorted-gather bug that leaves rows unwritten past 32,767 rows (independently of its upstream report) and stale reads of CPU-written memory inside one command buffer.

Where we lose. One-token decoding through the padded kernels runs at x0.75 of the stock kernels [K]. Our exact expert kernel runs at 70-74 % of the rate its own loads stream at [K]. The exact batched prefill peaks at 46.2 GB, over the 45 GB line [K]. And on the same machine and the same 48 prompts, against Strata (MIT; github.com/Niko1221/Strata), our decode is ahead on code and math and about level on chat, exact chat decode is behind, and Strata's first token (0.5-1.0 s) beats ours (about 2.7-4.6 s), so up to 256 generated tokens Strata wins or ties every whole-request number ([MAIN] Appendix G has the table).

---

1. Introduction

1.1 Why exactness needs batch invariance

- One-token greedy decoding defines the reference output. A speculative check computes 16 positions in one call. If a kernel's result for one row depends on how many rows share the call, the check's argmax can flip at a near-tie, and the output drifts from the reference.
- The same holds for prefill. A prompt processed in 1,024-token chunks is the reference; a faster prefill that puts more rows into each call must reproduce the chunked bits, or the first decoded tokens move.
- MLX chooses kernels by row count (section 2.2), so stock MLX gives neither property on this model (section 3).

1.2 Contributions

1. **An invariance map of MLX 0.32.2 on the M5** (section 3): for each op class of this model, whether a row's bits depend on the call's row count, at decode and verify widths (1-16 rows) and at prefill widths (1K-8K rows). Every entry is measured.

2. **A batch-invariant kernel set** (section 4) on the GPU's SIMD units and its matrix units, with what each kernel costs or saves.
3. **MLX findings** (section 5), each with a minimal reproduction script and its workaround.
4. **GPU and matrix-unit measurements** (section 6): peak, arithmetic order, streaming efficiency, dispatch cost, and where a check's time goes.
5. **Negative results** (section 7) and **what we left on the table** (section 9).

1.3 What we do not claim

- Not a new principle. Batch-invariant kernels with a fixed reduction order per element are published (Thinking Machines Lab, section 8). Ours is a port of the idea to Metal, MLX and the M5's matrix units, at this model's shapes, with the speculative check as the consumer.
- Not exact to the original weights, and not exact across references. "Exact (to R)" means output tokens, and the logits row behind every kept token, are bit-identical to one-token greedy decoding under reference R ([MAIN] section 3.1). At 45 GB, R is NAX6-N.
- Not a whole-system speed claim. Speeds here are per kernel or per check unless a row says otherwise; the system's whole decode, whole request and first token live in [MAIN] section 6.
- Not first. Two MLX findings were found before us: the sorted-gather bug was reported upstream on 5 Oct 2026 (section 5.1), and the narrow-output split-K dependence was first found by our own prefill-kernel study (section 5.2).

2. Background

2.1 The M5 GPU and its matrix units

- Apple's term is "a Neural Accelerator in each core" of the M5 GPU; Apple claims "over 4x the peak GPU compute performance compared to M4" for AI workloads (self-reported; Apple Newsroom, 15 Oct 2025). We call these units the matrix units. Metal 4 programs them through the Metal Performance Primitives (matmul2d on cooperative tensors).
- Measured on our MacBook Pro (M5 Pro, 64 GB): a peak of about 14.0 T MAC/s in 16x32x32 bf16 x bf16 -> fp32 operations, with about 10 % run-to-run spread (the same kernel read 10.5 and 11.7 T MAC/s on two days) [K, D:nax_peak.txt; LT:'M27-P'].
- Memory: a 4 GB streaming probe reads 285-286 GB/s [K, D:bw_probe_k90.txt].

2.2 How MLX 0.32.2 picks a quantized kernel

As read from MLX's `quantized.cpp` at v0.32.2 by our prefill-kernel study [PK, N1]:

- decode widths (S <= 8): qmv / qmv_wide / gather_qmv, on the SIMD units; no matrix units;
- dense calls above the vector limit: `qmm_splitk` with split_k = 512 / ceil(M / 32); when split_k <= 1 (N >= 512 at prefill widths) the call goes to `qmm_t_nax` (bm 32 if M <= 32, else 64; bn 64; bk 64);
- narrow outputs (N = 1 here) stay on split-K, with a split that moves with M;
- the sorted gather of SwitchGLU's prefill path (>= 64 ids) runs `affine_gather_qmm_rhs_nax`, bm 32 if rows / E < 64 else 64. At 16 rows the wide vector kernel declines, and the merged hyper-connection read takes MLX's matrix-unit split-K GEMM [K, LL line 117 'R-09b S4'].

2.3 The model's shapes

- Hidden size 2560. 48 layers: 36 Gated DeltaNet (GDN) layers and 12 attention layers. Four residual streams joined by low-rank hyper-connections (hc 4, rank 320). Per layer: one shared expert (2560 -> 640 -> 2560) with a one-output sigmoid gate, and 512 routed experts of the same shape, 10 per token [the model's config.json on Hugging Face, Qwen/Qwen3.8-Flash-Next].
- Checkpoint quantization: router 8-bit g64; the shared expert, its gate and the routed experts 4-bit g64 [same file]. Our expert store runs at about 3.2 bits per expert weight (ranks 0-1 at 4 bits, ranks 2-9 at 3 bits; [MAIN] section 4.1). Kernel names below say 4-bit or 3-bit for the record format a kernel reads, never for a run.

2.4 Memory

GB = `10^9` bytes of the lifetime peak (ri_lifetime_max_phys_footprint), as the run reports print it; the 45 GB line is 41.9 GiB. A memory line is the engine's own footprint on a 64 GB machine, not an operating-system

limit; a 7.45 GB expert tier held as purgeable memory is not counted in it ([MAIN] section 2).

3. The invariance map (measured)

One row per op class of this model. "Inv." = a row's bits do not depend on how many rows share the call. Sources are in the result table (R-numbers).

op class (MLX 0.32.2, M5)	decode / verify: 1 vs 16 rows	prefill: 1,024 vs 4K-8K rows
quantized matmul, 4-bit / 8-bit, N >= 512	**no**: different kernels below and above the vector limit (R1)	inv. once on `qmm_t_nax`: from M 16-256 by shape; the 8-bit router from M 541 (R30)
quantized matmul, narrow N (1-32)	**no** (R1)	**no**: split-K with an M-dependent split (R17)
MLX's own 16-row call, padded rows	inv. across 1-16 real rows (R3)	-
bf16 matmul, K 2560	**no**	**no**: 210-538 of 4K-8K rows differ (R18)
sorted gather_qmm (routed experts)	inv. as shipped	inv. across bm 32 / 64 and groupings, but **rows left unwritten** past 32,767 rows unless padded (R19)
GDN recurrence (fp32 state, sequential)	inv. (R2)	not tested: prefill keeps GDN in 1,024-token chunks
conv1d (GDN and PLE shapes)	inv., S 2-16 (R21)	-
PLE reductions (gate sum, rms_norm, HC mean)	inv., S 2-16 (R21)	-
SDPA (attention)	**no**: other code at 1 row than inside 16; key-count classes past 512 keys and below 128 (R25)	not tested: prefill keeps attention in 1,024-token chunks
softmax (precise) / argpartition / take / elementwise	inv.	inv. (R16)
sigmoid, compiled graph vs eager	**no**: the compiled form uses a faster exp (section 5.5)	-
matrix-unit fp32 accumulation	deterministic, but matches no fixed fp32 order (R12)	-

How each "no" was handled:

- **Quantized and bf16 matmuls at decode widths:** every call with fewer than 16 rows runs at 16 rows (section 4.1). MLX's own 16-row call is padding-independent, so padding alone restores invariance; our own 16-row kernels then made the padded calls fast.
- **Narrow N at prefill widths:** the only narrow projection in the prefill MoE block, the shared expert's one-output gate, stays on the 1,024-row chunks (section 4.6).
- **bf16 at prefill widths:** no bf16 matmul is batched past the chunk size; the quantized ones are.
- **Sorted gather:** rows padded to a multiple of 64 (section 5.1).
- **Attention:** calls padded to 16 rows, key buckets past 512 keys, length classes below 128 keys (section 4.4).
- **Phase-dependent lowering:** the commit replay runs inside the verify scopes (section 5.5).

4. The kernel set

The set is called REF2 in our sources; its extensions REF2b (attention buckets and classes, commit phase, slot call), REF2-T (rank-tiered experts) and NAX6-N (matrix-unit experts on native records) each became a reference after a same-scores test ([MAIN] section 3.4). A check runs every layer at 16 rows.

4.1 Dense projections: one 16-row shape

- **Pad to 16.** Every quantized or bf16 matmul with fewer than 16 rows runs at 16. MLX's own 16-row call is padding-independent at every dense shape tested: rows 1 / 2 / 5 / 8 / 12, padded, equal the all-real call bitwise [K, R3].
- **qmm16c.** A 16-row 4-bit kernel on the matrix units that multiplies the raw 4-bit codes and applies scales and biases after the MMA. It is row-invariant by construction, and has its own numerics: max error against fp32 0.010 on o_proj vs MLX's 0.023 [K, R6]. The 4-bit dense set takes 14.0 ms per 16-row check vs 35.2 ms for MLX's calls (byte floor 7.1); with it the whole dense part falls from 56.8 to 27.4 ms (floor 11.9) [K, R4, R5].

- **DENSE-N.** The same kernel on weights stored in the matrix unit's fragment order: 8.85 -> 7.39 ms per check (x1.20, 74 % of the byte floor), bitwise equal to qmm16c; the in-place form keeps one copy of the weights [K, LL lines 251, 255].
- **qmm16c8 and QMM32** for the drafter's projections: 8-bit codes (error equal to MLX's 8-bit call, better on down; 1.4-4.8x faster than bf16 per drafter shape), and a 32-row form that reads the weights once and equals two 16-row halves bitwise (-0.44 ms per propose vs the halves) [K, LL lines 282, 349].

4.2 Hyper-connection reads: replicating MLX's own split-K loop

- The merged down + inject GEMM (N 324) is a bit-for-bit replica of MLX's matrix-unit split-K loop, with weight loads issued three steps ahead. It saves 14.68 ms per check against the engine's 24.51 ms per 96 reads, and every projection equals MLX bitwise (96 / 96) [K, R9].
- One simdgroup walking all three K partitions is slower (15.93 / 25.69 ms vs 9.82 per 96 reads): 11 simdgroups are too few [K, LL line 118].
- When a reference already exists, a fast exact kernel can be a replica of the framework's own reduction order rather than a new order that then needs its own certification.

4.3 Routed experts

- **experts_nax**, one 16-row matrix-unit tile per expert: batch-invariant (rows 1 / 2 / 4 / 7 / 8 / 12 alone equal the same rows inside 16), and 2.3x closer to fp32 than MLX's sorted path (max error 0.00061 vs 0.00139). 0.776 vs 1.242 ms per layer at 16 rows, 75 % of the byte floor: 37.2 vs 59.6 ms per check [K, R7, R8].
- **Slot call.** The kernel runs on the expert pool's slots, with no gather into a fresh scratch per layer: -384 ms per 16-row check in the live loop of that day, exact, for checks of up to 16 rows [K, LL line 153].
- **NAX6 / NAX6-N.** A fixed split-K over the matrix units (NAX6), then records stored in the fragments' native order (NAX6-N). In an engine A/B against our 3-bit ALU tier kernel (3c-N), each exact to its own reference, on the 13 prompts kept in all four passes at C 192: whole decode 44.74 vs 42.12 tok/s (x1.062), lifetime peak 38.11 vs 38.10 GB [K, R23]. That screen recorded no prefill wall, so whole request and first token are not available for this A/B (a limit of the 6 Nov harness, not re-run).
- **One-call out-of-order.** Resident experts first, late slots resolved inside the same kernel, with coherent(system) reads of the pool (section 5.7). On 12 IFEval prompts at C 256, two runs each: with coherent reads, 12 / 12 prompts token-identical to one-token decoding in both runs, whole decode 37.8 / 37.9 tok/s, whole request 21.1 / 21.3 tok/s, first token 2.83 s (2.23-3.79) / 2.80 s (2.17-3.73); with plain reads 3 / 12 in both runs, with different divergences, at 37.7 / 38.0 tok/s. Lifetime peak 41.70 GB in all four [K, R14].

4.4 Attention

- Calls with fewer than 16 rows run through the verify path at 16, and the cache is trimmed back [K, R2].
- **Key buckets.** Past 512 keys, every 16-row attention call runs at the next multiple of 512 keys (zero-padded, masked); rows whose one-token step lands in another bucket are recomputed there. With buckets, every logits row past 512 keys equals the bucketed one-token reference: 160 / 160 (a geometry prompt, positions 520-679) and 220 / 220 (HumanEval/68, positions 367-586, across 512); with buckets off, the control breaks (2 / 160) [K, R25].
- **Length classes below 128 keys:** 1,477 / 1,477 tokens and 441 / 441 logits rows equal on 6 prompts [K, R25].
- **One SDPA call per layer** instead of 12 head chunks: -4.06 ms of fixed cost per check (48.40 -> 44.34 ms), bitwise to the reference below 512 keys [K, R26]. Past 512 keys its bits follow a different class split, so it is a FAST option, not part of the exact set [K, LT:'SDPA-1 RESULT'].

4.5 The self-test

`ref2.self_test` checks on random data, for each kernel, that a row computed alone (padded to 16) equals the same row inside a 16-row call: qmm16c (6144 x 2560), MLX's own 16-row call (512 x 2560), the expert kernel (rows alone, and the split form equal to the fused experts_nax), and a bf16 Linear [S:ref2.py, self_test]. The evaluation suite asserts it before every run against an exact reference [ES:evalsuite/live16.py], and we run it after every change to the kernel module. It checks invariance only; exactness against a reference is the audits' job ([MAIN] section 3.2).

4.6 Prefill

- The reference is 1,024-token chunks. Batching a layer's MoE block over all of a prompt's chunks (with the next layers' experts read ahead) is exact only with (a) the sorted rows padded to a multiple of 64 (section 5.1) and (b) the shared expert's one-output gate kept per chunk (section 5.2). Every other op of the block is row-invariant: router, softmax, top-10 selection and normalisation, the padded expert GLU even with 6,216 rows on one expert, the weighted sum, the shared expert and the residual write [K, R15, R16].
- In the engine, with both pinned and up to 8,192 rows per call: decode tokens equal and logits hash equal on 8 / 8, 28 / 28 and 22 / 22 checks for 1.6K / 4.7K / 7.9K-token prompts; prefill 215 / 432 / 500 tok/s vs 189 / 358 / 433 for the chunked path (x1.14 / 1.21 / 1.16); lifetime peak 46.23 vs 45.09 GB [K, R20]. At 2,048 rows per call the peak is 45.81 GB at x1.12-1.17 [K, R20]: both stay over the 45 GB line.

5. MLX findings

5.1 The sorted gather leaves rows unwritten past 32,767 rows

- **Symptom.** SwitchGLU's up projection alone on a 7,932-token prompt (79,320 sorted rows, top-10): 32,776 rows differ between one call and 1,024-token calls; the one-call rows are unwritten [K, R19].
- **Cause.** `affine_gather_qmm_rhs_nax` computes each simdgroup's remaining rows as `min(SM, short(max(0, (M - (y_row + tm)))))` when M is not a multiple of the tile [MLX:quantized_nax.h line 1547]. Past 32,767 remaining rows the `short` wraps negative, and those simdgroups skip compute and store.
- **Workaround.** Pad the sorted rows to a multiple of 64 (BM), which takes the aligned branch: padded one call equals the 1,024-token calls bitwise at 15,650 / 47,300 / 79,320 rows, and runs the routed experts x1.42 / 1.89 / 1.96 faster [K, R19].
- **Upstream.** MLX issue #4632, "[BUG] gather_qmm(sorted_indices=True) returns wrong results on M5 Max (macOS 27) when rows > 32768 and not a multiple of 64", opened 5 Oct 2026 and closed 6 Oct; the reporter could not reproduce it on MLX 0.32.3, and the thread attributes the fix to PR #3922. It was reported there before we hit it independently [PK, N3; BM: LEDGER.md BUG1]. We stay on 0.32.2 with the padding, because 0.32.4 changes tokens against our exact references.
- Reproduction: S:pfb_diag2.py (the symptom), S:pfb_fix.py (the padded call).

5.2 Narrow-output split-K depends on the row count

The shared expert's gate (4-bit, one output) runs through `qmm_splitk` with an M-dependent split. One call vs 1,024-row calls: equal up to M 1,025, then 41 rows differ at M 1,100, 310 at 1,565, all rows 0-2,047 at 2,048, 4,068 at 8,192. The N 640 projections of the same expert are equal at the same M [K, R17]. On the real layer-0 weights our prefill-kernel study found 4,748 of 8,192 rows differ [PK, N3]. Reproduction: S:pfb_seg.py.

5.3 bf16 matmul is not row-invariant at prefill widths; quantized N >= 512 is

At K 2560, a bf16 matmul over 4K-8K rows differs from 1,024-row calls on 210-538 rows; the 4-bit and 8-bit quantized calls at N >= 512 are equal [K, R18]. Our first version of this test skipped quantized outputs under 32 columns and so missed section 5.2; the correction is in section 7.2. Reproduction: S:pfb_dense_inv.py.

5.4 Attention's key-count classes

- MLX's fused SDPA has exactly one class boundary at 128 keys between 120 and 33,000 keys on the vector kernel [K, LL line 152 'R-40 STEP 3m / 3l'].
- Past about 512 keys, MLX's full attention kernel for 16 queries gives key-length-dependent bits: 2 / 16 rows bitwise, max logit difference up to 4.6 [K, LL line 157 'R-40 STEP 4a'].
- A synthetic test had said SDPA was invariant to the key length (0 of 40 extra-key lengths changed a row); real data broke it [K, LL line 151 'R-40 STEP 3o']. Invariance tests need the real weights and activations.
- Reproduction: S:r40s3o_screen.py, S:r40s3m_screen.py, S:r40s4a_screen.py, S:r40s4c_screen.py.

5.5 Phase-dependent lowering

- The model's attention gate is a sigmoid. Inside a compiled verify graph it is lowered with a faster exp than in eager mode, so the commit's replay outside the verify scopes differed in the last bit. Running

the commit replay inside the verify scopes fixed it: a math prompt that had diverged went to 256 / 256 tokens equal [K, LL line 148 'REF2 commit_phase'].
- Kernels that replay MLX's eager bf16 sigmoid must use `metal::precise::exp`: the JIT default and `fast::exp` each miss 1 of 65,536 bf16 inputs; precise misses 0 [K, LL line 190].

5.6 The matrix unit's fp32 sums match no fixed order

1,024 adversarial tests x 512 outputs: no sequential, pairwise or blocked fp32 order matches the matrix unit's result (best 45-45.6 %). Against the exact sum, 45 % are bitwise, 39 % off by 1 ulp, 14 % by 2-8 ulp and 2.4 % by more. Integer operands match 100 %, and the relaxed-precision flag changes nothing [K, R12]. So an ALU kernel cannot stand in bitwise for a matrix-unit one, and choosing between them by group size or row count breaks row invariance. Reproduction: S:nax_order_probe.py.

5.7 Stale reads of CPU-written memory inside one command buffer

- **Probe.** A 2 MB slot that the GPU has read is rewritten by the CPU, then read again (20 trials each, two runs): the first read after the rewrite is clean (0 / 20 trials stale in both); the same kernel's plain loads return stale words in 20 / 20 and 19 / 20 trials (all 524,288 words in every trial of the first run); a later dispatch in the same command buffer returns 505,808-524,288 and 96-524,288 stale words (20 / 20 in both); adding a buffer memory barrier still leaves 16 / 20 and 20 / 20 trials stale; coherent(system) loads read 0 stale words (0 / 20 in both); a slot last read by an earlier, completed command buffer is clean (0 / 20, second run) [K, R13].
- **Cost of the fix,** rough speed under a share lock. Coherent loads on the pool pointers only (codes, low-bit words, scales, biases): +0.7 % and +1.5 % on NAX6-N's kernels in two runs, +26.3 % on the 3-bit ALU kernel in the one run where its coherent form built, because its single-byte code loads are slow when coherent [K, R13].
- **In the engine.** The one-call path of section 4.3 made exact chat nondeterministic (3 / 12 prompts equal, with different divergences per run) until coherent reads were turned on (12 / 12 twice) [K, R14].

5.8 Streams are thread-local; eval releases the GIL

MLX 0.32.2 releases the GIL in `eval` (a side thread keeps 99-104 % of its solo rate), and streams are thread-local: a second thread must create its own stream, and the main thread must build the graph on it. In that form a two-thread build / encode is bitwise equal to the serial loop over 200 checks [K, LT:'ENC2 + PF-B dense invariance'; D:enc2*.txt]. Its speed is a negative result (section 7).

5.9 fp32 matmuls default to TF32

MLX 0.32.2 sets `MLX_ENABLE_TF32=1` by default. An fp32 matmul then has an error of 4.15e-4 against 6.5e-7 with it off, which broke an fp32 training reference in our drafter study until the flag was set to 0 [K, TR:mtphead line 124].

5.10 A custom kernel's inputs change address space with their size

`mx.fast.metal_kernel` generates the kernel's signature from the arrays it is called with, and an input's address space depends on its size. A prefill kernel of the engine (one that rebuilds pool-resident experts on the GPU) passes inputs it does not need in a call as one-element placeholders. In the run, MLX declared exactly those inputs `const constant uint32_t*`, and the kernel failed to build: "assigning 'const constant uint32_t *' *... to 'const device uint* ' changes address space of pointer". The kernel's unit test had passed because it called the kernel with full-size arrays only. A 1,024-element placeholder is declared `device`; with it the same run completed, with tokens equal to the path without the kernel on 12 / 12 prompts and logits hashes equal at 721 / 721 checks [K, R32]. The workaround is a placeholder that is not small, and a unit test that calls the kernel with the smallest inputs the engine passes (the engine's test now does) [K, R32]. MLX's custom-kernel guide shows generated signatures with `device` inputs only and does not mention the size rule (opened 9 Nov); we did not locate the rule's size cut in MLX's source.

6. GPU and matrix-unit measurements

All on one MacBook Pro; "quiet" = no other GPU job. Kernel times are from dependent chains of the engine's own calls; the per-call synchronised timings swing 2-3x between processes and are not used [K, LT:'STEP 0

PART B RESULT'].

6.1 Peak and arithmetic

About 14.0 T MAC/s at the matrix units (section 2.1, R11); their fp32 accumulation order (section 5.6, R12).

6.2 Streaming efficiency of the expert kernels

At S 16, quiet, a loads-only replica of NAX6-N's memory access reads 295-296 GB/s, the machine's streaming rate. NAX6-N itself runs at 74 % of it (218-219 GB/s), a mixed FAST kernel at 79-81 %, and NAX6 at 58-59 %; across S 8-32 the replica reads 287-305 GB/s and NAX6-N runs at 70-74 % [K, R24]. The gap is exposed compute, not bandwidth: making an earlier expert kernel bandwidth-bound did not pay (section 7).

6.3 One dependent dispatch costs about 1.6 us

A chain of 200 dependent kernels that move almost no bytes: 1.24 us per kernel at 32 threads, 1.40-1.56 us at 1,024 to 20,480 threads, 4.13 us when 1 MB moves [K, R27]. A check launches about 1,000-4,000 kernels, so dispatch is worth about 1-6 ms per check (1,000-4,000 x 1.24-1.56 us), not the 20 ms an earlier fit assumed [P, R27].

6.4 The dense time is lost in narrow matrix-vector kernels

Per call in a dependent chain, at 2 / 16 rows [K, R27]. MB = `10^6` bytes the call reads and writes (weights, scales and biases, its input and output; the raw report counts them in KiB); GB/s = MB / us, at 2 and 16 rows:

kernel	tiles	us per call	MB per call	GB/s	calls per check
GDN input projection	512	113 / 115	24.2	210-213	36
attention query projection	384	75 / 74	18.2	243-247	12
GDN / attention output projection (K 6,144)	80	112 / 112	9.1	81-82	48
shared expert gate / up	41	48 / 47	2.0	42-43	48
attention key and value projections	16 each	48 / 49	0.84	17-18	24
GDN a / b projections	3	45 / 46	0.22	5	36

- Wide kernels already stream near the machine's rate. The narrow ones walk the whole K dimension with one simdgroup per 32-column tile, so with 3-80 tiles the GPU is mostly empty and the call costs about 1.1-1.2 us per K group whatever its bytes [K, R27].
- **Split K per shape (DN-KS).** Enough simdgroups per call: output projection 114.8 -> 41.4 us (214 GB/s on its 8.85 MB of weights), a / b 47.3 -> 10.4, key / value 47.4 -> 9.4, shared gate / up 48.4 -> 14.7, shared down 16.7 -> 9.1; -7.74 ms per 2-row check over these five in the microbenchmark and -4.19 ms of fixed cost in the engine (55.17 / 55.30 -> 51.05 ms) [K, R28]. The split changes the summation order, so it is not bitwise to the reference; it stays row-invariant, so a reference built with it would again be exact to itself, after its own certification, which we did not run (section 9).

6.5 Where a check's time goes

- **Fixed cost** = the steady 2-row cycle minus twice the warm per-row slope: 53.74 ms with the 6 Nov fast stack, against a pre-registered PASS line of 30 ms (KILL above 36) [K, R29]. The per-row slope is 3.01 ms per row on the expert kernel of that day [K, R29].
- **Per piece, a warm 2-row check, two measures of the same run** [K, R29; F:2026-11-06_0140_ flat2-fixed-cost-per-check]:

piece	wall time, one sync per piece (ms)	GPU time (ms)
GDN	15.7	13.2
routed experts	17.4	11.1 (about 4.6 of it fixed)
hyper-connection reads	11.0	8.6
attention	8.6	6.4
router + shared expert	5.7	5.7
MoE tail	3.5	1.1

piece	wall time, one sync per piece (ms)	GPU time (ms)
sum	111.1 (all pieces, incl. embed / norm / head 2.3)	47.8 (all pieces, incl. HC write 0.8, PLE 0.65)

The wall column pays a fresh command-buffer start per piece, so read it as proportions, not time. The GPU column is each piece's synced time minus the 0.133 ms round trip per sync; its sum is about the whole unsynchronised check, so the check is GPU-bound and the 13.4 ms CPU graph build is hidden behind GPU work. Only the experts grow with rows: 11.1 ms of GPU time at 2 rows, 57.0 ms at 16 with all experts resident [K, same source].

- At about 12 tokens kept per check, a 54 ms fixed cost alone caps code near 220 tok/s even with free experts [P, from R29].

6.6 Prefill throughput

MLX's 4-bit qmm runs at about 27-29 TF from 1K rows; the routed experts' gather_qmm rises from 7.1 TF at 1K rows to 13.8 at 8K [K, R30]. In the batched prefill MoE call (section 4.6) the routed experts run at 11.2 TF on a 7.9K-token prompt vs 5.7 TF in 1,024-token calls [K, R19].

7. Negative results

7.1 Kernels that did not pay (each exact unless marked)

idea	result	verdict	source
int4 operands straight into the matrix units (Metal 4 matmul2d, bf16 x uint4)	bitwise and row-invariant, but slower: 16-row experts 0.992 vs 0.757 ms per layer (58 % of floor); dense x0.68-0.87	KILL	R10; LL line 72
software-pipelined expert kernel (NAX7)	11 % slower (96 / 96 bitwise)	KILL	LL line 219
loader / compute simdgroup specialisation (NAX7-WS)	35 % slower	KILL	LL line 220
the expert kernel as three dispatches (NAX7-F)	+1.2 %, one unexplained hang at S 32 under load	KILL	LL line 223
persistent expert kernels (NAX8-P1)	best configuration -5.2 % at S 16; slower in every residency	KILL	LL line 306
3-bit code decode by table lookup	device table -1.1 %, threadgroup table -15.1 % at S 16	KILL	LL line 312
int8 activations x int4 codes, int32 sums (NAX-C; FAST)	same speed as NAX6 (-0.3 % at S 16), error 1.07e-2 x RMS; on native records -1.7 %	KILL	LL lines 226, 305
making the expert kernels bandwidth-bound (K-60)	the headroom was not bandwidth	KILL	LL line 168
two-thread graph build / encode (ENC2)	bitwise, but x0.96 on its toy	KILL	LL line 389
GEMV on native records for 1-2-row checks (NGEMV; FAST)	-1.41 / -0.80 ms per check in isolation; no engine gain, the chat check is CPU-bound	KILL as speed	LL lines 384, 388
split-K drafter attention (DSDPA)	x0.56, then x0.51 at 2,054 keys against a x0.50 line	missed	LL lines 304, 326
trellis-coded experts on the matrix units (TRL-NAX; FAST)	0.87x NAX6-N's time at S 16, but the trellis store failed the quality screen	not built	LL lines 322, 325
the Neural Engine beside the GPU's expert kernel	per-layer GPU -> ANE -> GPU handoff 78-147 us	KILL	LL line 243
one simdgroup per HC read (R-09b F4 / F8)	15.93 / 25.69 vs 9.82 ms per 96 reads	KILL	LL line 118
GPU timeline from a Metal System Trace	the trace streams counters to the temp dir at about 120 MB/s; replaced by knock-outs and dependent chains	method KILL	LL line 97

7.2 Results we corrected

- **"Batched prefill is row-invariant at the MoE block's dense shapes."** Our first test passed, but it skipped quantized outputs under 32 columns, the shared expert's gate among them. The engine's batched prefill then broke the logits hash; the gate's split-K was the cause (section 5.2), and keeping it per chunk made the prefill exact [K, R15, R16; LT:'PF-B EXACTNESS', CORRECTION].
- **"SDPA is invariant to the key length"** held on synthetic data and failed on real data (section 5.4).
- Across the paper set, three headlines were withdrawn or corrected: "12 GB, 40.8 tok/s 'exact'" (a FAST arm; withdrawn); "45 GB FAST stack, 157.9 tok/s" (included CD16, which failed MBPP; withdrawn); "180.3 tok/s" was the registered steady metric, which is 142.8 on whole decode, pooled (metric corrected 7 Nov; the correction note's 142.1 is a geo mean). None is a kernel result; [MAIN] section 10.0 has them.

8. Related work

- **Batch invariance.** Thinking Machines Lab (H. He et al., "Defeating Nondeterminism in LLM Inference", 10 Sep 2025) trace inference nondeterminism to kernels that are not batch-invariant and make RMSNorm, matmul and attention invariant by keeping each element's reduction order fixed whatever the batch; attention uses fixed-size KV splits rather than a fixed number of splits. Their code (batch-invariant-ops) runs in vLLM's deterministic mode; on their Qwen-3-8B benchmark default vLLM took 26 s, the deterministic mode 55 s, and 42 s with an improved attention kernel (self-reported). Our key buckets (section 4.4) are the same idea as fixed-size KV splits. What differs here is the consumer (a 16-row speculative check against one-token decoding, not a serving batch), the hardware (Metal, the M5's matrix units, whose fp32 order no ALU kernel can match, section 5.6), and that the reference is fixed first and the fast kernels are made to reproduce it, sometimes by replicating the framework's own loop (section 4.2).
- **Deterministic serving modes.** vLLM has an opt-in batch-invariance mode (`VLLM_BATCH_INVARIANT=1`; its documentation lists NVIDIA GPUs only) and SGLang a deterministic mode (`-enable-deterministic-inference`) that integrates the Thinking Machines kernels, with attention at a fixed split-KV size; both say the mode costs speed (vLLM documentation, "batch invariance"; LMSYS blog, "Towards Deterministic Inference in SGLang and Reproducible RL Training", 22 Sep 2025).
- **Determinism through verification.** LLM-42 (Gond, Kamath, Ramjee and Panwar, arXiv 2601.17768, Jan 2026) decodes on a nondeterministic fast path and replays the candidate tokens under a fixed-shape reduction schedule, committing the consistent ones and rolling back the rest; it argues that batch-invariant kernels impose a fixed cost even when little traffic needs determinism. Our check is the same pattern with one difference: the reference itself is defined at the verify's fixed 16-row shape (one-token decoding runs through the same padded kernels, section 4.1), so the fast path that drafts can be any drafter and the cost of exactness is the x0.75 of one-token decoding (R31), paid back by the tokens each check keeps.
- **Speculative decoding.** Leviathan et al. (ICML 2023; arXiv 2211.17192) and Chen et al. (2023; arXiv 2302.01318) define draft-then-verify decoding that keeps the target model's distribution; with greedy decoding, exactness further needs the verify pass to reproduce one-token decoding's bits, which is this paper's subject. Our drafters are our fine-tune of DFlash2 (Inco AI; Apache-2.0), a block drafter of the DFlash method (Chen, Liang and Liu, arXiv 2602.06036; code z-lab, MIT), and the model's MTP head ([DRAFTER]).
- **Frameworks.** MLX (Apple; MIT), and MTPLX by Youssof Altoukhi (https://github.com/youssofal/mtplx; Apache-2.0), whose model code for this architecture and MTP decoding loop our engine builds on. The sorted-gather bug was reported upstream as MLX issue #4632 (section 5.1).
- **Same model, same machine.** Strata (MIT; github.com/Niko1221/Strata), from our own runs ([MAIN] section 9 and Appendix G). Published runs of this model on other machines are in [MAIN] Appendix H (self-reported).

9. What we left on the table

- **GPU idle time.** During chat decode the GPU is busy 81 % of the time on average (p10 74 %, p90 86 %); in-kernel spin counts as busy, so the idle share is at least 19 % [K, R22].
- **Exposed compute in the expert kernel.** NAX6-N runs at 70-74 % of the rate its own loads stream at; the rest, 26-30 %, is exposed compute [K, R24].
- **A non-bitwise split-K for the narrow projections** saves 4.19 ms of fixed cost per check in the engine; it needs a new reference and its certification, which we did not run [K, R28].
- **The chat check's CPU side.** NGEMV's -1.41 / -0.80 ms per check (FAST) showed no engine gain because the chat check is CPU-bound: about 31.5 ms of graph build in a 33 ms check [K, LL line 388]. Every GPU-side saving on chat waits for a shorter build.
- **The prefill peak.** The exact batched prefill gains x1.14-1.21 but peaks at 45.8-46.2 GB, over the 45 GB line; a variant under the line was not found [K, R20].
- **Prefill overlap.** At 7.9K tokens the expert reads overlap compute for about 1.2 of about 6 s, and the batching delivered about half of its kernel-level estimate [P, from D:pfb_bigmoe_report.txt].
- **Matrix units at verify widths.** MLX runs no matrix units below its vector limit (S <= 8) [PK, N1]; our 16-row kernels fill that gap only for the shapes we wrote.

Result table (sources in Appendix A)

#	result	exact? (to which reference)	number	mark	source
R1	stock kernels: 16 one-token steps vs one 16-row verify	-	all 16 rows differ at record 0 (layer 0 GDN matmul); logits 0 / 16	[K]	D:inv0_screen.txt; LT:'R-40 REF2'
R2	the REF2 kernel set, same test	exact to REF2	16 rows x 37 records bit-identical, 12-layer slice, 1K context	[K]	D:inv1_screen.txt
R3	MLX's 16-row call, rows 1 / 2 / 5 / 8 / 12 padded	inv.	bitwise equal to the all-real call, every dense shape	[K]	D:kern23.txt
R4	qmm16c, the 4-bit dense set per 16-row check	exact (REF2)	14.0 vs 35.2 ms (floor 7.1)	[K]	D:kern23.txt
R5	dense part per 16-row check	exact (REF2)	56.8 -> 27.4 ms (floor 11.9)	[K]	D:kern23.txt
R6	qmm16c accuracy vs fp32, o_proj	-	max error 0.010 vs MLX 0.023	[K]	D:kern23.txt
R7	experts_nax per layer, 16 rows	exact (REF2)	0.776 vs 1.242 ms (75 % of floor); 37.2 vs 59.6 ms per check	[K]	D:qmmg25.txt
R8	experts_nax accuracy vs fp32	-	0.00061 vs 0.00139 (2.3x)	[K]	D:qmmg25.txt
R9	HC read as a replica of MLX's split-K loop	bitwise = MLX (96 / 96)	-14.68 ms per check vs 24.51	[K]	D:r09c_screen.txt
R10	int4 operands into the matrix units	bitwise, inv.	16-row experts 0.992 vs 0.757 ms per layer; in_proj_qkv 3.05 vs 2.60; o_proj 3.41 vs 2.31; in_proj_z 2.26 vs 2.07 ms (KILL)	[K]	D:qmm27.txt; LT:'L1-27'
R11	matrix-unit peak, 16x32x32 bf16 -> fp32	-	~14.0 T MAC/s; run-to-run ~10 %	[K]	D:nax_peak.txt
R12	matrix-unit fp32 sum vs any fp32 order	-	no order matches (best 45-45.6 %); vs the exact sum 45 / 39 / 14 / 2.4 %	[K]	D:nax_order_probe.txt, nax_order_fit.txt
R13	stale CPU-written slot inside one command buffer (2 MB, 20 trials)	correctness	two runs: B0 control 0 / 20 (both); same kernel stale 20 / 20 and 19 / 20; next dispatch 505,808-524,288 and 96-524,288 words, 20 / 20 (both); + memory barrier 16 / 20 and 20 / 20; coherent(system) 0 / 20 (both); earlier command buffer 0 / 20 (run 2). Cost, rough speed (share lock): NAX6-N +0.7 % and +1.5 % (two runs), 3c-N +26.3 % (one run)	[K]	D:s7b_probe.txt, s7b_probe2.txt, coh_check.txt; LT:'S7B-PROBE', 'COH'
R14	coherent one-call in the engine (12 IFEval, C 256, two runs each)	exact (NAX6-N one-call)	coherent: == one-token decoding 12 / 12 both runs, whole decode 37.8 / 37.9, whole request 21.1 / 21.3 tok/s, first token 2.83 (2.23-3.79) / 2.80 (2.17-3.73) s; plain: 3 / 12 both runs, 37.7 / 38.0 and 20.0 / 20.1 tok/s; lifetime peak 41.70 GB	[K]	D:chs4/chs4_report.txt lines 21-24; whole request and first token pooled from BM:data/r57/ycoh{,2}_256_runs.json, ycoc{,2}_256_runs.json
R15	prefill MoE block, chunked vs batched call shapes	exact to chunked prefill	only the shared-expert gate differs (304 / 1,565; 2,326 / 4,730; 4,451 / 7,932 rows)	[K]	D:pfb_moe_inv.txt
R16	the same with the gate per chunk	exact	all ops equal at 6 shapes	[K]	D:pfb_seg.txt
R17	narrow-N split-K, one call vs 1,024-row calls	**not** inv.	equal to M 1,025; 41 rows differ at 1,100, 310 at 1,565, rows 0-2,047 at 2,048, 4,068 at 8,192; N 640 control equal	[K]	D:pfb_seg.txt; PK N3
R18	bf16 vs q4 / q8 matmul at 4K-8K rows	q inv., bf16 not	bf16 210-538 rows differ; q4 / q8 equal	[K]	D:pfb_dense_inv.txt

#	result	exact? (to which reference)	number	mark	source
R19	sorted-gather bug	correctness	79,320 sorted rows: 32,776 differ (unwritten); padded to 64: one call == 1,024-token calls at 15,650 / 47,300 / 79,320 rows, experts 31.2 -> 22.0, 80.4 -> 42.6, 136.3 -> 69.4 ms (5.7 -> 11.2 TF at 7.9K)	[K]	D:pfb_diag2.txt, pfb_fix.txt
R20	batched prefill MoE + read-ahead in the engine, gate per chunk	exact to chunked prefill	tokens == and logits hash 8 / 8, 28 / 28, 22 / 22; 215 / 432 / 500 vs 189 / 358 / 433 tok/s (1.6K / 4.7K / 7.9K); peak 46.23 vs 45.09 GB; at 2,048 rows per call 212 / 418 / 486 tok/s, 45.81 GB	[K]	BM:data/pfd/pfd_report.txt at commit a6d5347, part B (it rounds peaks to 0.1 GB); peaks from BM:data/r57/{pdb38s,pdpa1,pdb32s}_256_runs.json (mem)
R21	conv1d and PLE reductions at S 2-16	inv.	conv1d: 0 of 64 rows (0 of 655,360 elements) differ at every S, both shapes; PLE reductions: 0 differing rows for every op at every S	[K]	D:r40s3s_screen.txt; LT:'R-40 STEP 3s', '3r'
R22	GPU busy during chat decode (C 256)	-	mean 81 %, p10 74, p90 86, min 72, max 88 (40 x 0.5 s; in-kernel spin counts as busy)	[K]	D:gpu_util_m1b.txt
R23	NAX6-N vs the 3-bit ALU kernel (3c-N), engine A/B, C 192, quiet, 13 prompts kept in all 4 passes	exact (each to its own reference)	whole decode 44.74 vs 42.12 tok/s (x1.062; the steady-state rate, information only, read x1.0665); lifetime peak 38.11 vs 38.10 GB; whole request and first token not recorded by that screen	[K]	D:natfri/r57/nf*_192_runs.json via S:natfri_pooled.py -> D:natfri/natfri_pooled.txt; LT:'NATIVE-FRI' RUN 2
R24	expert kernels vs the machine's streaming rate (S 8 / 16 / 32, quiet)	-	4 GB probe 285-286 GB/s; loads-only replica of NAX6-N 287-305 (S 16: 295-296); NAX6-N 70-74 % (S 16: 74 %, 218-219 GB/s); FAST hybrid 76-81 % (S 16: 79-81 %); NAX6 55-59 % (S 16: 58-59 %)	[K]	D:k90_arm012.txt, bw_probe_k90.txt; LL line 311
R25	key buckets past 512 keys; SDPA classes below 128 keys	exact (REF2b)	buckets 160 / 160 and 220 / 220 rows; buckets off 2 / 160; classes 1,477 / 1,477 tokens and 441 / 441 logits rows on 6 prompts	[K]	D:r40s4c_screen.txt; LT:'R-40 STEP 4c'; TABLE_350 line 2980
R26	one SDPA call per layer	bitwise below 512 keys (FAST past it)	fixed cost 48.40 -> 44.34 ms (-4.06)	[K]	D:stepb/out/stepb_{b,batt,b2}*224_sdpa**; LT:'SDPA-1 RESULT'; LL line 345
R27	dependent dispatch and narrow-kernel costs	-	1.24-1.56 us per dependent kernel (4.13 with 1 MB); per-kernel table of section 6.4	[K]	LT:'STEP 0 PART B RESULT'; D:stepb/
R28	split K per shape (DN-KS)	not bitwise (new order, row-invariant)	-7.74 ms per 2-row check (micro); engine fixed cost 55.17 / 55.30 -> 51.05 ms (-4.19)	[K]	D:dnks_bench.txt; D:stepb/out/stepb_{fri,friks,fri2}*224_ab**; LT:'DN-KS step 1 / 2 RESULT'
R29	fixed cost of a check (FLAT-2)	-	53.74 ms (PASS line 30, KILL > 36); slope 3.01 ms per row	[K]	LT:'FLAT-2 RESULT'
R30	MLX prefill kernels and row invariance by M	-	4-bit qmm ~27-29 TF from 1K rows; gather_qmm 7.1 TF (1K) -> 13.8 (8K); qmm row-invariant from M 16-256 by shape, the 8-bit router from M 541	[K]	LL line 380; LT:'PREFILL 500'
R31	one-token decoding through the padded kernels	exact (REF2)	11.65 vs 15.50 tok/s over 210 rows (x0.752; row time, prefill included; information only in that report); peak 35.6 GB	[K]	EV:RESULT_ref2_2026-10-28, Speed table; LT:'R-40 REF2' step 2

#	result	exact? (to which reference)	number	mark	source
R32	custom-kernel inputs by size (section 5.10)	exact	one-element placeholder inputs declared `const constant`: build error in the run; a (1, 1024) uint32 placeholder builds; the re-run's tokens equal the path without the kernel 12 / 12, logits hash 721 / 721; unit test with the engine's placeholders PASS	[K]	BM:data/pfd/pdz4a_256.txt at commit 3175ef3 lines 71-80 (the error); BM commit 9aa398b (the fix); BM:data/pfd/pfd_report_0009.txt line 16 (pdz4a); BM:scripts/pfd_unit.py line 147

Appendix A. Sources

Folder paths appear only here. Paths are relative to the root of the public snapshot, github.com/kamenlevi/350-tps; commit hashes are those of each folder's own repository, and the snapshot holds the files as last committed.

key	file
LT:'*heading*'	a section of `Leap/TABLE_350.md` (the kernels ledger)
LL line *n*	line *n* of `FableGod/TRIED/leap.md` (one line per idea, with its verdict)
TR:mtphead line *n*	line *n* of `FableGod/TRIED/mtphead.md` (the drafter study's ledger)
D:*file*	`Leap/data/<file>` (raw output)
S:*file*	`Leap/scripts/<file>` (the test that produced it)
PK	`PrefillKernels/FINDINGS.md` (the prefill-kernel study)
BM:*path*	`BigMoE/<path>` (the engine; "at commit c" = the file at the engine repository's commit c)
EV:*name*	`EvalSuite/docs/<name>.md` (the evaluation suite's results)
ES:*file*	`EvalSuite/<file>` (the evaluation suite's code)
MLX:*file*	`mlx/include/mlx/backend/metal/kernels/<file>` in the installed MLX 0.32.2 package
F:*name*	`FableGod/findings/<name>.md` (dated run notes)
[MAIN], [EVAL], [DRAFTER], [RELEASE]	the companion papers

R14's whole request and first token were pooled on 8 Nov from the four runs' records (n / (sum of prefill walls + sum of n / tok/s)); the same arithmetic as S:natfri_pooled.py.

Appendix B. Reproduction

- Every [K] row names its raw output (D:) and, in sections 5 and 7, its script (S:). The kernels and tests are released under GPL-3 with the engine ([RELEASE]).
- Software: MLX pinned to 0.32.2 with the pad-to-64 fix for MLX issue #4632 (sorted gather_qmm past 32,767 rows when M is not a multiple of 64); 0.32.4 changes tokens against our exact references.

Appendix C. Limits stated instead of numbers

- GDN and attention at prefill widths: not tested, because the prefill keeps both in 1,024-token chunks (section 3).
- NAX6-N vs 3c-N: whole request and first token were not recorded by the 6 Nov screen (R23).

Use of AI tools

The code, the experiments and the text of this paper were produced with substantial help from AI coding agents. The author directed the work and is responsible for its content. The numbers were re-derived from the raw run records named beside them, and the references were opened by hand or by script.

References

- Youssof Altoukhi. MTPLX (software, v2.12.2, commit 9882703f; Apache-2.0). https://github.com/youssofal/mtplx
- Apple. "Apple unleashes M5, the next big leap in AI performance for Apple silicon." Apple Newsroom, 15 Oct 2025. https://www.apple.com/newsroom/2025/10/apple-unleashes-m5-the-next-big-leap-in-ai-performance-for-apple-silicon/
- Charlie Chen et al. "Accelerating Large Language Model Decoding with Speculative Sampling." arXiv:2302.01318, 2023.
- Jian Chen, Yesheng Liang and Zhijian Liu. "DFlash: Block Diffusion for Flash Speculative Decoding." arXiv:2602.06036, 2026. ICML 2026. Code: https://github.com/z-lab/dflash (MIT).
- Raja Gond et al. "LLM-42: Enabling Determinism in LLM Inference with Verified Speculation." arXiv:2601.17768, 2026.
- Awni Hannun, Jagrit Digani, Angelos Katharopoulos and Ronan Collobert. "MLX: Efficient and flexible machine learning on Apple silicon" (software, 2023; MIT). https://github.com/ml-explore/mlx
- Horace He and Thinking Machines Lab. "Defeating Nondeterminism in LLM Inference." Thinking Machines Lab: Connectionism, 10 Sep 2025. doi:10.64434/tml.20250910. https://thinkingmachines.ai/blog/defeating-nondeterminism-in-llm-inference/
- Inco AI. "DFlash 2: Keep Drafting Parallel." Blog, Aug 2026. https://inco.ai/blog/dflash2
- Inco AI. Qwen3.8-Flash-Next-DFlash2 (drafter checkpoint, revision a22a174, 19 Oct 2026; Apache-2.0). https://huggingface.co/incoai/Qwen3.8-Flash-Next-DFlash2
- Yaniv Leviathan, Matan Kalman and Yossi Matias. "Fast Inference from Transformers via Speculative Decoding." arXiv:2211.17192, 2022. ICML 2023.
- LMSYS Org. "Towards Deterministic Inference in SGLang and Reproducible RL Training." LMSYS blog, 22 Sep 2025. https://lmsys.org/blog/2025-09-22-sglang-deterministic/
- MLX issue #4632, "[BUG] gather_qmm(sorted_indices=True) returns wrong results on M5 Max (macOS 27) when rows > 32768 and not a multiple of 64", 5 Oct 2026. https://github.com/ml-explore/mlx/issues/4632
- MLX pull request #3922, "Fix sorted gather_qmm NAX row overflow above 32K", merged 26 Aug 2026. https://github.com/ml-explore/mlx/pull/3922
- Niko1221 and the Strata contributors. Strata (software, version 0.1.58-metal, commit 146df3c; MIT). https://github.com/Niko1221/Strata
- Qwen Team. Qwen3.8-Flash-Next (model weights; Qwen Community License 1.0). https://huggingface.co/Qwen/Qwen3.8-Flash-Next
- Thinking Machines Lab. batch_invariant_ops (software; MIT). https://github.com/thinking-machines-lab/batch_invariant_ops
- vLLM project. "Batch Invariance." vLLM documentation. https://docs.vllm.ai/en/latest/features/batch_invariance/

Companion papers:

- [DRAFTER] Kamen Levi. Drafting for a 125B mixture-of-experts model that does not fit in RAM: training its own MTP head, one drafter per text type, and the levers that failed. 2026. A companion paper of this set.
- [EVAL] Kamen Levi. Certifying lossy decoding on the model's known benchmarks: same-scores gates for a 125B mixture-of-experts model on one 64 GB Mac. 2026. A companion paper of this set.
- [MAIN] Kamen Levi. What It Takes to Run a 125B LLM Fast on a 64 GB Mac, and How to Go Faster. 2026. A companion paper of this set.
- [RELEASE] Kamen Levi. Running a 125B mixture-of-experts model on a 64 GB Mac: the engine package. 2026. A companion paper of this set.

Certifying lossy decoding on the model's known benchmarks: same-scores gates for a 125B mixture-of-experts model on one 64 GB Mac

Kamen Levi
Independent researcher

9 November 2026

Abstract

Lossy decoding is what makes a 125B mixture-of-experts model fast on one 64 GB Mac ([MAIN]), and it changes the output. We ask a narrow question: does a lossy configuration score the same as its exact reference on the model's known benchmarks?

- **The gate:** pre-registered and paired, one benchmark at a time.
  - The paired accuracy delta must be above −5 points, and the exact two-sided McNemar test must not reject (p >= 0.05).
  - It runs in two prompt orders, against the exact one-token greedy reference of the same memory mode.
- **Exactness** is audited separately: every token, and the logits row behind every kept token, must be bitwise equal to the reference's own one-token decode.
- **What passed, 20 October to 9 November 2026 [K]:**
  - L + COH, the 45 GB configuration we publish, and the chat configuration (M1-B + COH) passed the model card's IFBench, GPQA Diamond and LiveCodeBench v6 in both orders.
  - Its certified base ("v1' + wired"; L adds to it a bundle of kernel changes that is bitwise in the exact mode) passed all 10 of its pre-registered lines: HumanEval+, MBPP, math and IFEval in both orders, a pooled line and a long-code line. A certificate covers only the configuration it tested, so it does not carry to L + COH.
  - Exact decode paths that differ only in numerics (REF2, REF2b, NAX6) passed the same gate against each other.
  - Four more lossy configurations passed on the benchmarks they were gated on: 8-bit mixers; a short stack gate; a 32 GB point on HumanEval+ only; the chat configuration on IFEval (before its card gate).
- **What failed [K]:**
  - six quality-defined modes;
  - a code profile on the −5-point boundary;
  - every 12 GB candidate we gated on task scores.
- **Where we lose:**
  - The gate is coarse. Exact paths that differ only in numerics moved one benchmark by 8 points on 50 items [K]. An arm with no true loss fails one order of one benchmark with a probability we price at 5-15 % [P].
  - Most of our gates ran on subsets of public benchmarks and on home-made lines. The model card's own benchmarks (IFBench, GPQA Diamond, LiveCodeBench v6) were adopted as the gate on 8 November, on 381 keys: IFBench 150 of 300, GPQA Diamond 100 of 198 and one 131-problem LiveCodeBench window. L + COH has passed them, and so has the chat configuration (M1-B + COH; 6 of 6 lines); the 32 GB point (zs32) is not gated (no card gate ran).
  - We list our mistakes, among them a reference switched after we had seen a result, and a chat path we called exact without having measured it.

Marks: [K] measured, [P] priced, [G] judgement. Each number carries a source key (Appendix A). Every number below was re-read from its source on 8 or 9 Nov 2026. None is quoted from a summary.

1. Introduction

The setting. The engine of [MAIN] runs a 125B mixture-of-experts model on one 64 GB Mac.

- It has an exact mode and lossy modes. In the exact mode, every token is the reference's one-token greedy choice.
- The lossy modes are faster because they skip, approximate or reuse work: drafts accepted on a weaker check, expert computation at lower precision, and quantization tiers.
- A lossy mode is only useful if it keeps the model's abilities. "Keeps its abilities" is easy to claim and hard to check.

The narrow question. We do not ask whether a lossy mode produces the same tokens; it does not. We ask whether it scores the same as its exact reference on benchmarks the model is known by:

- paired, prompt by prompt;
- with a fixed, pre-registered threshold;
- in more than one prompt order, because a lossy mode's output can depend on what ran before it (section 4.2).

Contributions:

1. A gate protocol: the rule, two histories, the own-mode exact reference, and pre-registration with disclosed amendments (section 4).
2. An exactness audit that makes "exact" a measured property, not a label (section 3).
3. About 30 pre-registered verdicts on one model and one machine, passes and failures alike (section 5).
4. A catalogue of our own mistakes and the fix each led to (section 6).
5. A move, mid-project, from home-made lines to the model card's benchmarks, with the reasons and the cost (sections 4.5 and 8).

What we do not claim.

- A pass is "same scores on the named benchmarks, at the stated N, in both orders". It does not claim unchanged quality, and it does not mean there is no loss.
- A gate at 100-164 items cannot see a loss of 2-3 points (section 4.6).
- Exact means exact to a named reference (45 GB: NAX6-N), not to the original 4-bit weights ([MAIN] section 2).

2. Setting

- **Model and machine:**
  - Qwen3.8-Flash-Next: 125B parameters, mixture of experts, 48 layers, 512 routed experts per layer, 10 per token.
  - One MacBook Pro, M5 Pro, 64 GB unified memory, internal SSD.
  - The store holds about 3.2 bits per expert weight: ranks 0-1 at 4 bits, ranks 2-9 at 3 bits ([MAIN] section 2).
- **Memory lines:** a run is on the N GB line when its lifetime peak footprint is at most N GB.
  - GB = `10^9` bytes of the lifetime peak (ri_lifetime_max_phys_footprint), so the 45 GB line is 41.9 GiB [K, ES:evalsuite/decode.py footprint_peak_gb]; a 7.45 GB expert tier held as purgeable memory is not counted in it ([MAIN] section 2).
  - The line is the engine's own footprint, not an OS limit.
- **The exact reference chain [K]:**
  - The stock reference: the original 4-bit weights, decoded 20 October.
  - REF2, a batch-invariant exact path ([KERNELS]).
  - REF2b, which adds key buckets.
  - REF2-T, which adds the store's 3-bit tiers. The tiers are lossy against REF2b; REF2-T passed the gate and became the reference.
  - NAX6, the matrix-unit expert kernel, and NAX6-N, the same on the converted store. NAX6-N is the 45 GB exact reference since 5 November.
  - Each step passed the same gate as a lossy arm would (Table 1a). Each exact path was audited bitwise (Table 1b).
- **Lossy levers.** These are described where they are built: [MAIN] section 5 for the 45 / 32 GB levers, a forthcoming companion paper ([SMALL-RAM], by 11 November 2026) for the 24 / 12 GB loops. Here they are arms. A configuration is a frozen code snapshot plus an environment file, named in each result's source.
- **Decoding:** greedy, non-thinking chat template, one prompt at a time, with fixed caps [K, ES:evalsuite/data.py; tools/mbpp_build.py; tools/card_items.py]:

– MATH-500 1,536 and GSM8K 768;
– IFEval 1,024;
– HumanEval+ 768;
– MBPP 1,024;
– IFBench 1,536;
– GPQA Diamond 2,048;
– LiveCodeBench v6 2,048.

3. Exactness is audited, not assumed (Table 1b)

The audit. "Exact (to R)" means that the output tokens, and the logits row behind every kept token, are bit-identical to one-token greedy decoding under R ([MAIN] section 2). We test exactly that.

- A fixed set of 36 full-length prompts: 12 code, 12 IFEval, 12 math.
- Each is decoded by the arm's own one-token AR and by its live 16-row drafting loop.
- Every kept logits row is compared bitwise. A prompt whose AR leaves the stored reference rows is VOID.

The first audit failed, and that was useful.

- R-56 tested REF2's live loop: 28 of 36 prompts were exact, and math only 4 of 12 [K, EV:RESULT_r56].
- Below the AR's 512-key point, 2,153 of 2,199 compared rows were bitwise equal. Past it, the attention bits followed the key length, and tokens flipped only at ties [K, EV:RESULT_r56].
- An earlier claim that REF2 was "bit-exact inside the live loop" had compared its equal prompts over their first 48-97 positions only.
- The fix (key buckets: REF2b) then passed 36 of 36, with 16,305 of 16,305 kept rows bitwise [K, EV:RESULT_r56b].
- Every later exact path was audited the same way before it was used as a reference: REF2-T; REF2-T with out-of-order experts and a fused combine; NAX6-N (Table 1b).
- A speed change that must not change outputs gets the same kind of audit against its own control. A PASS lets it inherit its configuration's gate verdicts without a gate of its own. Examples: two changes in [RELEASE]'s 45 GB chat configuration, the prefill staging PF-D v4 and the schedule change SB1 (Table 1b).

Line checks. A reference decoded at a smaller expert cache (C 160 instead of 192) is used only after a line check. 12 IFEval rows at that C must be token-identical to the rows at C 192; at C 160 they were (12 / 12) [K, ES:results/card/linecheck_C160.txt].

Three words kept apart.

- *Exact* (to R): bitwise, as above.
- *Verified*: every committed token is the full model's greedy choice at its computed width, but not bit-identical to one-token decoding. This is the 12 GB loop (described in the forthcoming [SMALL-RAM]).
- *Deterministic*: the same output on a rerun.
- We conflated them once (section 6, item 9).

4. The same-scores gate

4.1 The rule

"Decision 4" has been used since 28 October [K, EV:PREREG_ref2_2026-10-28].

- In each named benchmark, the paired delta (arm − reference) must be above −5 points, AND the exact two-sided McNemar p must be at least 0.05.
- A benchmark fails if either part fails. The configuration fails if any benchmark fails.
- A gain with $p < 0.05$ is reported as "DIFFERS (HIGHER)" and counts as a pass.
- The floor (the same weights with only the summation order changed) is reported beside each verdict as the noise yardstick.

What it replaced. The suite's first rule set (19-20 October) had 5-17 lines [K, ES:docs/DESIGN.md]:

- per-set deltas, answer agreement, length and hesitation ratios, and "no rise in truncations or loops".

- The truncation / loop line failed the stock reference against a re-decode of itself, although 200 of its 210 rows were token-identical [K, EV:RESULT_ref2]. It became information only.

4.2 Two prompt orders

Why. A lossy mode's output can depend on history.

- **Cache state:** which experts earlier prompts left resident. Two runs of one frugal 12 GB configuration differed "by up to" 4 points per set [K, EV:RESULT_smallram].
- **The drafts:** on long agent turns, two orders gave token-identical rows on 3 of 17 turns [K, EV:RESULT_agentfast].
- Since 3-6 November, every lossy gate runs the keys in two orders (sorted forward, then reversed) and must pass in both [K, EV:PREREG_TEMPLATE_cachedep; EV:PREREG_stack_2026-11-06].
- **One order is allowed only after a history-independence test.** The 12 GB IFBench arm's harness decoded 13 prompts in another order and matched bit for bit, with SSD bytes moving −1.1 % to +12.5 % between orders [K, F12H].
- The template also asked for a repeat check: the same history must reproduce the same tokens. It appears in no later pre-registration [K]. This is a gap (section 8).

4.3 The reference of the arm's own mode

The rule. A lossy arm is compared with the exact reference of its own memory mode: NAX6-N at 45 GB, since 5 November. It is not compared with an older or a different exact path. Exact paths themselves differ on task scores by more than the gate's resolution (Table 1a).

The one time this rule moved after a result.

- On 7 November the FAST code profile ("STACK") failed against REF2-T on HumanEval+ by the smallest margin: −5.00 points in both orders, p 0.25 [K, EV:RESULT_stack].
- The next gate's reference was then changed to NAX6-N, the stack's own exact base. The amendment says so: "Made after seeing a result" [K, EV:PREREG_s4].
- The rule itself had been in force since 5 November. The disclosure stays. S4, C30 and CERT's HumanEval+ forward order all fail against REF2-T as information (Table 2).
- The 12 GB gates still compare with REF2-T (section 8).

4.4 Pre-registration

- Every gate has a PREREG document committed before its first row. It names the arm, the reference, the sets and N, the orders, the rule and the stop conditions.
- Amendments are dated and committed before the rows they affect. Scheduling-only amendments say so. Rule changes say when they were made relative to the data.
- **Exceptions disclosed in our records [K]:**
  - one amendment made after 80 rows, with only row counts seen [EV:PREREG_pmin12];
  - the reference change above, made after a result [EV:PREREG_s4];
  - one arm reported as "one of three tries" on the same 60 prompts [EV:PREREG_night_gates].

4.5 Benchmarks: from subsets and home-made lines to the model card's

Until 8 November.

- The gates ran on fixed subsets of public benchmarks:
  - math, 50 problems from MATH-500 levels 3-5 and GSM8K;
  - IFEval, 100 prompts, strict, prompt level;
  - HumanEval+, 60 problems, pass@1;
  - later MBPP 100, HumanEval+ 164 and math 100 [K, EV:RESULT_cert].
- Beside them ran home-made lines that sometimes decided verdicts:
  - blunders per 10,000 tokens on long agent and code turns;
  - KLD drift;
  - a multilingual blunder line.

The 8 November rule. Lossy configurations are judged against exact on the model's known benchmarks. Home-made lines (blunders, KLD, multilingual drift) are information only [K, EV:PREREG_chat_gate].

The card gates [K, EV:PREREG_card_gates]:

- **Sets:** the model card's three public benchmarks.
  - IFBench (300 prompts; the official grader vendored; prompt-level loose, the paper's headline metric).
  - GPQA Diamond (198; generative, with the simple-evals template and answer pattern).
  - LiveCodeBench v6: the 131 problems of the 2025-02 to 2025-05 window that the card's family names. LiveCodeBench's own run_test grader is vendored and runs on public and private tests.
- **Subsets:** the gate keys are seeded subsets, IFBench 150 and GPQA Diamond 100 (seed 1108), plus the whole LCB window: 381 keys. They were sized so that one arm in two orders fits one evening window: about 450K tokens and 3.6 h [P, EV:PREREG_card_gates]. The first arm, L + COH, decoded 867,659 tokens in 5.8 h of decoding (forward 164 min, reverse 176 min, re-runs 6 min; process starts included) [K, EV:RESULT_card_lcoh_2026-11-09]. Its rate, about 42 tok/s with the starts, was within the priced 38-45 tok/s. Its rows were about twice as long as priced (1,139 tokens per row against about 590), so an arm needs a night, not an evening.
- **The extension rule:** an arm that fails one order only on the −5-point delta, with p >= 0.05, is PENDING. That benchmark is extended to its full set in both orders before it counts as a fail. A McNemar p < 0.05 stays a fail at once.
- **What the card gates do not test:** the card's numbers (81.3, 91.9 and 91.7) are thinking mode with sampling. Ours are non-thinking greedy, on quantized weights, so they cannot be compared with the card at fine resolution (section 8).
- **The caps bind on two sets** [K, ES:results/ref2tnaxn.r6/decode.jsonl]. The exact reference stops at its cap on 54 of 100 GPQA Diamond rows and 88 of 131 LiveCodeBench rows (IFBench: 10 of 150). Arm and reference run under the same caps. So the gate asks whether a lossy arm keeps the reference's results within that budget, not what the model scores at full length. A lever that makes answers longer would lose rows to the cap, and that would count as a loss.

4.6 Power: what the gate can and cannot see

The noise floor, measured. Between exact paths that differ only in numerics [K, Table 1a]:

- math moved by up to 8 points on 50 items (REF2 vs the stock re-decode);
- IFEval moved by up to 5 points on 100 items (REF2c-T vs REF2-T, a FAIL on the boundary). The same pair passed on 220 fresh IFEval prompts (+1.36 points, p 0.51) [K, EV:RESULT_ifx].

Priced for the card gates [P, EV:PREREG_card_gates "Known limits"]. Take an arm with no true loss whose rows differ from the reference on 10-15 % of keys.

- The −5-point line fails it in one order with these chances:
  - ~5 % at 10 % discordance (N 100);
  - ~15 % at 15 % discordance (N 100);
  - ~7 % at 15 % discordance (N 150).
- Over 3 benchmarks × 2 orders, it fails somewhere with a real chance, ~25-50 %.
- The full sets would cut that to ~2-5 % per order, at about +236K tokens per arm.
- The extension rule (4.5) is our answer to that cost.

The boundary. Deltas of exactly −5.0 fail. Four of our fails sit on it (Table 3, item 6).

5. Results

Table 1a: exact paths, same-scores gate against each other

Core 210 = math 50, IFEval 100, HumanEval+ 60. One order. Cells give arm vs reference accuracy %, the paired delta in points, and McNemar p. All [K].

arm (kind)	vs	math	IFEval	HumanEval+	verdict	source
REF2 (batch-invariant exact path)	stock reference, re-decoded	90.0 vs 82.0, +8.0, p 0.12	87.0 vs 90.0, −3.0, p 0.38	91.7 vs 91.7, 0.0, p 1.00	PASS; token-identical 68/210 (floor 59/210)	EV:RESULT_ ref2_2026-10-28

arm (kind)	vs	math	IFEval	HumanEval+	verdict	source
REF2b (+ key buckets)	stock re-decode	88.0 vs 82.0, +6.0, p 0.25	87.0 vs 90.0, −3.0, p 0.38	91.7 vs 91.7, 0.0, p 1.00	PASS	EV:RESULT_ ref2b_2026-10-31
REF2-T (+ the store's 3-bit tiers; lossy vs REF2b, then the reference)	REF2b	88.0 vs 88.0, 0.0, p 1.00	90.0 vs 87.0, +3.0, p 0.38	95.0 vs 91.7, +3.3, p 0.50	PASS; token-identical 48/210 (floor 63/210)	EV:RESULT_ r58_2026-10-31
REF2c-T (another exact path)	REF2-T	86.0 vs 88.0, −2.0, p 1.00	85.0 vs 90.0, −5.0, p 0.0625: FAIL	93.3 vs 95.0, −1.7, p 1.00	FAIL on the boundary	EV:RESULT_ ref2ct_2026-11-02
REF2c-T, IFEval re-test on 220 fresh prompts	REF2-T		84.5 vs 83.2, +1.36, p 0.51 (corrected 11-04; verdict unchanged)		PASS	EV:RESULT_ifx_ 2026-11-03
NAX6 (matrix-unit expert kernel)	REF2-T	88.0 vs 88.0, +0.0, p 1.00	87.0 vs 90.0, −3.0, p 0.375	91.7 vs 95.0, −3.3, p 0.50	PASS; the 45 GB reference	EV:RESULT_ nax6_2026-11-05

Table 1b: exactness audits

Bitwise against the arm's own one-token AR, on the 36 full-length prompts. The last three rows compare against a named control instead.

audit	result [K]	verdict	source
R-56: REF2's 16-row live loop	28/36 prompts (math 4/12); bitwise below the 512-key point: 2,153 of 2,199 rows	FAIL	EV:RESULT_r56_2026-10-30
R-56b: REF2b (also with the sync-free gate)	36/36, 16,341 tokens, 16,305 / 16,305 kept rows	PASS	EV:RESULT_r56b_2026-10-31
R-56c: REF2-T's live loop	36/36, 15,525 tokens, 15,489 / 15,489 kept rows	PASS	EV:RESULT_r56c_2026-11-01
R-56d: REF2-T + out-of-order experts + fused combine	36/36; loop = own AR = R-56c's AR; 15,489 / 15,489 rows	PASS	EV:RESULT_r56d_2026-11-01
NAX6-N	36/36, 15,726 tokens, 15,690 / 15,690 kept rows; NAX6-N == NAX6	PASS	EV:RESULT_nax6_2026-11-05
card reference line check, C 160 vs C 192	12 / 12 IFEval rows token-identical	PASS	ES:results/card/linecheck_ C160.txt
PF-D v4 + read-ahead: chat prefill takes experts already in the pool; on by default in the 45 GB chat configuration (zs: M1-B + COH + SB1, C 256)	vs its own control with PF-D off, 12 IFEval prompts: tokens 12 / 12, logits hashes 721 / 721. With its check on (exact path E): 87,912 staged rows, 0 differing	PASS (bitwise to its control)	BM:data/pfd/pfd_report_ 0009.txt; BM:data/r57/pdz4a_ 256_runs.json, pdz0_256_ runs.json, pde4k_256_runs.json
SB1, a schedule change in the chat configuration (zs = M1-B + COH + SB1)	tokens: zs, and zs + PF-D v4 as shipped, equal M1-B + COH on 12 / 12 IFEval prompts (C 256, one history). Logits on the exact path E: E + SB1 vs E, 676 / 676 hashes equal (12 prompts). On the chat configuration the link is token-level only: those runs kept no logits hashes	PASS (tokens; logits on E)	BM:data/r57/z0_256_tokens.json, zs_256_tokens.json, pdz4a_256_ tokens.json; BM:data/r57/yeh_ 256_runs.json, yesh_256_ runs.json; BM:docs/WED_ TESTS.md (CHS5)

Table 2: lossy configurations

The 45 GB reference is REF2-T until 7 November and NAX6-N from then on (section 4.3). Deltas are paired points, and p is McNemar's. All [K].

45 / 32 GB lines

configuration	vs	sets (n) × orders	result	verdict	source
HC mixers at 8 bits (HC8)	REF2-T	core 210 × 1	math −4.0, p 0.50; IFEval −1.0, p 1.00; HumanEval+ −3.3, p 0.50	PASS	EV:RESULT_ hc8_2026-11-02
RELAX ship mode (code mode B + chat mode E)	REF2-T	core 210 × 1	math +2.0; IFEval −3.0, p 0.25; HumanEval+ −1.7. Decision 4 PASS; agent-blunder and drift bars failed	BETWEEN	EV:RESULT_ relax_2026-11-05
RELAX A_E / C_E / B_F / B_G / T3ALL	REF2-T	core 210 × 1	math −6.0, p 0.25 / HumanEval+ −5.0, p 0.25 / IFEval −5.0, p 0.12 / IFEval −6.0, p 0.07 / HumanEval+ −6.7, p 0.12	KILL (all five)	EV:RESULT_ relax_2026-11-05
ship8 (B + E8; HC8 process-global)	REF2-T	core 210 × 1	IFEval 84.0 vs 90.0, −6.0, p 0.03	KILL	EV:RESULT_ relax_drift_ 2026-11-06
STACK (FAST code profile, C 224)	REF2-T	math 50 + HumanEval+ 60 × 2	HumanEval+ 90.0 vs 95.0, −5.00, p 0.25 in both orders	FAIL (kept on record)	EV:RESULT_ stack_2026-11-07
STACK without CD16, MBPP	NAX6-N	MBPP 100 × 2	76.0 vs 79.0, −3.00, p 0.4531 in both orders	PASS	EV:RESULT_ stack_2026-11-07
S4 v1	NAX6-N	math 50 + HumanEval+ 60 × 2	math 86.0 vs 88.0, −2.00, p 1.0; HumanEval+ 88.3 vs 91.7, −3.33, p 0.50 (both orders)	PASS; vs REF2-T HumanEval+ FAIL (information)	EV:RESULT_s4_ 2026-11-07
C30: L at C 160, before COH (the ~32 GB lossy context point)	NAX6-N	HumanEval+ 60 × 2	forward −3.33, p 0.625; reverse +0.00, p 1.0	PASS (HumanEval+ only); vs REF2-T forward −6.67: FAIL (information)	EV:RESULT_ c30_2026-11-07
CERT: "v1' + wired", the certified 45 GB configuration (C 256). L adds the exact-safe bundle (BM:data/ stack/s5p_env.txt; bitwise in the exact mode, [MAIN] 4.4); the published L + COH has its own gate (below)	NAX6-N	HumanEval+ 164; math, MBPP, IFEval 100 each; × 2	HumanEval+ −2.44, p 0.125 / +0.00, p 1.0. Math +0.00, p 1.0 / −2.00, p 0.625. MBPP −3.00, p 0.375 / +2.00, p 0.727. IFEval −1.00, p 1.0 / −2.00, p 0.754. Pooled over HumanEval+, math and MBPP in both orders (728 pairs; IFEval not pooled) −0.96, p 0.265. Long code (41 rows, 55,966 tokens): 0 blunders	PASS, 10 of 10 lines. Chat NOT exact: 1 of 12 rows identical	EV:RESULT_ cert_2026-11-08
chat configuration M1-B + COH	NAX6-N	IFEval 100 × 2	forward −1.00, p 1.0; reverse −4.00, p 0.388	PASS (IFEval)	EV:RESULT_ chat_m1b_ 2026-11-08
plain FAST on long agent turns	NAX6-N's verify path (a home-made line)	17 turns × 2	85 blunders over 34 rows, 19,749 tokens; the line was 0	FAIL; agent turns run exact	EV:RESULT_ agentfast_ 2026-11-06

24 / 12 GB lines and the store

configuration	vs	sets × orders	result	verdict	source
the store's tier plan (ranks 2-9 at 3 bits)	the original 4-bit weights	math 50, IFEval 100, HumanEval+ 60, RULER 24 × 1	+0.0, p 1.00 / −4.0, p 0.29 / +0.0, p 1.00 / +4.2, p 1.00; loops 4 vs 2 (limit 3)	16 / 17 lines; adopted 20 Oct	EV:RESULT_ plan4433333333; TR:evalsuite.md
T0: 12 GB two-pass top-2	stock reference	core 210 × 1	math −2.0, p 1.00; IFEval −6.0, p 0.07; HumanEval+ −11.7, p 0.02	KILL	EV:RESULT_t0_ 2026-11-05

configuration	vs	sets × orders	result	verdict	source
12 GB FAST family, θ 0.8 / 0.9 / 0.95	REF2-T	HumanEval+ 60 × 2	−11.67, p 0.0156 / −8.33, p 0.0625 / −5.00, p 0.25	FAIL (all three)	EV:RESULT_ t80ck1u_2026-11-07
PMIN12: a cut of the MTP chain	its control	239 rows × 1	identical scores; peak 11.99 GB vs the 11.95 GB line	FAIL (memory only)	EV:RESULT_ pmin12_2026-11-01
fold-fast	its exact twin	239 rows × 1	answer agreement 210/210; peaks up to 15.60 GB	not PASS (memory)	EV:RESULT_r41c_ 2026-10-28
24 GB fast mode b16	its natural-routing base / the stock reference	core 210 + MGSM 80 × 1	IFEval −4.0 vs base; −5.0 vs stock	5/5 vs base; 4/5 vs stock (FAIL)	EV:RESULT_ fastmode_b16
small-RAM family (older rule set; two runs pooled)	the stock reference	core 210 × 1-2	e.g. B12LL-b16 IFEval −7.0, p 0.02; 24 GB D1 pooled IFEval −6.0	mixed; "READ WITH NOISE"	EV:RESULT_ smallram_2026-10-22

The card gates (the model card's benchmarks)

item	state	source
NAX6-N reference on the 381 gate keys	complete, 381 / 381 rows [K] (9 Nov). Rows at the cap: IFBench 10 / 150, GPQA Diamond 54 / 100, LCB v6 88 / 131 [K]	ES:results/ref2tnaxn.r6; EV:PREREG_ card_gates
L + COH (the published 45 GB configuration; QOM on LCB, exact expert reads on IFBench and GPQA Diamond)	PASS, 6 of 6 lines [K] (9 Nov): lossy, same scores on IFBench, GPQA Diamond and LCB v6. IFBench 40.7 vs 44.0 %, −3.33, p 0.302 in both orders. GPQA Diamond 47.0 vs 44.0 %, +3.00, p 0.508 in both. LCB v6 30.5 vs 32.1 %, −1.53, p 0.688 / 29.0 %, −3.05, p 0.289. On IFBench and GPQA the two orders gave the same tokens on every row	EV:RESULT_card_lcoh_2026-11-09
M1-B + COH (the chat configuration; the shipped 45 GB chat configuration adds SB1 and PF-D v4, both audited in Table 1b)	PASS, 6 of 6 lines [K] (9 Nov): lossy, same scores on IFBench, GPQA Diamond and LCB v6. Forward / reverse against 44.0, 44.0 and 32.1 %: IFBench 44.0 / 41.3 % (p 1.0 / 0.50), GPQA Diamond 42.0 / 42.0 % (p 0.75 / 0.79), LCB v6 29.0 / 32.1 % (p 0.39 / 1.0)	EV:RESULT_card_m1b_2026-11-09; EV:PREREG_card_gates
zs32: M1-B + COH + SB1 at C 160 (32 GB)	not gated: no card gate ran (amendment 11)	EV:PREREG_card_gates amendments 8 and 11

6. Pitfalls and our mistakes (Table 3)

All items come from our own records [K]. Each led to a rule or a tool that is still in use.

#	what happened	consequence / fix	source
1	The stock reference did not reproduce on later code: rounding changed on 10 of 210 rows between two engine versions	the re-decode became the baseline; the cause was not looked into	EV:RESULT_ref2_2026-10-28
2	The truncation / loop line failed the stock reference against a re-decode of itself	demoted to information	EV:RESULT_ref2_2026-10-28
3	The vendored IFEval checker called its language detector unseeded, so a re-score could move IFEval by one item	seeded; all tables re-scored, no verdict moved	EV:RESULT_smallram_2026-10-22
4	IFEval's letter-frequency check with a non-letter ("!") scored a randomly drawn letter	one re-test corrected (84.1 → 84.5, p 0.75 → 0.51); verdict unchanged	EV:RESULT_ifx_2026-11-03
5	"Bit-exact inside the live loop" had been checked on the first 48-97 positions only; it fails past 512 keys	key buckets, and bitwise audits of every kept row (section 3)	EV:RESULT_r56_2026-10-30

#	what happened	consequence / fix	source
6	Deltas of exactly −5.0 fail the line (REF2c-T IFEval, STACK HumanEval+, the θ 0.95 arm, RELAX C_E and B_F). REF2c-T then passed on 220 fresh prompts	a re-test stage; the card gates' PENDING / extension rule	EV:RESULT_ref2ct, RESULT_ifx, PREREG_card_gates
7	The reference was changed after a result (STACK → S4's NAX6-N)	disclosed; both comparisons reported (section 4.3)	EV:PREREG_s4_2026-11-07
8	"Chat stays exact" in the stack was assumed, not measured	CERT measured it: not exact (1 of 12 rows); the wording became "lossy, same scores"	EV:PREREG_cert, RESULT_cert
9	The 12 GB "exact" mode is verified, not exact, and "×1.12 over exact" was measured against a FAST arm (40.8 tok/s)	headline withdrawn; a 32-token setup check	EV:PREREG_t80ck1u_2026-11-07
10	The HC8 switch was process-global as built, so chat ran HC8 although only code was gated	the ship8 KILL	EV:RESULT_relax_drift_2026-11-06
11	Live code changed mid-run: a commit landed between the pin check and the import	the chunk was voided; frozen snapshots with per-process module manifests	EV:PREREG_agentfast_2026-11-06
12	A dry run took the real path: an apostrophe inside a shell ${...:?} message swallowed lines	2 rows voided; rule: no apostrophe inside ${} text	EV:PREREG_chat_gate_2026-11-08
13	Home-made lines (blunders, KLD, multilingual drift) decided verdicts until 8 November	information only; gates use known benchmarks	EV:PREREG_chat_gate, PREREG_card_gates
14	A cheap screen (the decision map) predicted PASS; the decode failed (12 GB θ family)	screens rank, gates decide	EV:RESULT_t80ck1u_2026-11-07
15	A cutoff bounded chunk starts, not ends, so a chunk ran past its window	cutoffs now bound chunk ends	EV:RESULT_relax_2026-11-05
16	On the shared machine, a reference runner read another job's "ended, incomplete" signal as "done", and a waiting decode client started 30 s into a heat pause	only a finished signal releases it; a guard stops clients during a pause (no row was affected)	EV:PREREG_card_gates amendments 4, 6
17	The arm screen stops a row mid-decode when its process's 600 s budget runs out, and flags it. The row writer did not read the flag and kept 10 such rows in the first card arm's forward order, ending them as "length" below the cap. The engagement check caught a later one (a 1-token LCB row with no QOM use) and voided the arm	cut rows are left out and their keys re-run; the arm resumed before any verdict. An audit of 40 earlier runs found no verdict that used a cut row	EV:PREREG_card_gates amendment 9

7. Related work

Nothing here claims priority. Each arXiv id below resolves to the named title and first author (arXiv API, 9 Nov); both repositories are MIT-licensed; the three journal papers match their Crossref records; the blog post and llama.cpp's KL-divergence mode were opened.

- **Evaluation harnesses:**
  - EleutherAI's lm-evaluation-harness (github.com/EleutherAI/lm-evaluation-harness; Biderman et al., arXiv 2405.14782);
  - HELM (Liang et al., arXiv 2211.09110).
- **The benchmarks:**
  - HumanEval (Chen et al., arXiv 2107.03374); EvalPlus / HumanEval+ (Liu et al., arXiv 2305.01210); MBPP (Austin et al., arXiv 2108.07732);
  - MATH (Hendrycks et al., arXiv 2103.03874), with the MATH-500 split of Lightman et al. (arXiv 2305.20050); GSM8K (Cobbe et al., arXiv 2110.14168);
  - IFEval (Zhou et al., arXiv 2311.07911); IFBench (Pyatkin et al., arXiv 2507.02833); GPQA (Rein et al., arXiv 2311.12022); LiveCodeBench (Jain et al., arXiv 2403.07974);
  - simple-evals (OpenAI; github.com/openai/simple-evals) for the GPQA template.
- **Quantization evaluation beyond average accuracy:**
  - "Accuracy is Not All You Need" (Dutta et al., arXiv 2407.09141), on flips;
  - accuracy-recovery studies of quantized LLMs, for example Kurtic et al. (arXiv 2411.02355);

  - KLD-based checks in local-inference tools (llama.cpp's perplexity tool has a KL-divergence mode).
- **Paired tests:**
  - McNemar (1947), Psychometrika 12(2);
  - Dietterich (1998), Neural Computation 10(7), on comparing classifiers;
  - non-inferiority and TOST (Schuirmann 1987, Journal of Pharmacokinetics and Biopharmaceutics 15(6)).
- **Exactness of fast decoding:**
  - speculative decoding (Leviathan et al., arXiv 2211.17192; Chen et al., arXiv 2302.01318), whose exactness is distributional;
  - batch invariance and nondeterminism in LLM inference (Thinking Machines Lab, "Defeating Nondeterminism in LLM Inference", Sep 2025; [KERNELS] has the kernel side).

8. What we left on the table

- **Power** [P, EV:PREREG_card_gates]. Full card sets (IFBench 300, GPQA Diamond 198, LiveCodeBench v6 175) cut an arm with no true loss's chance to fail somewhere from about 25-50 % to about 2-5 % per order. They cost about +236K tokens per arm (about +1.7 h) and +118K on the reference (about +2.5 h). These prices predate the first arm, which decoded about twice the tokens priced for it (section 4.5), so they are likely low.
- **Longer caps** [P, ES:results/ref2tnaxn.r6/decode.jsonl]. The exact reference stops at the 2,048-token cap on 142 GPQA Diamond and LCB rows (section 4.5). Doubling those caps to 4,096 would add at most 290,816 tokens to the reference: 5.7-6.5 h at its measured 14.2 tok/s without heat pauses and 12.5 tok/s with them, plus each arm's share.
- **The card's own setting** [P, EV:PREREG_card_gates "The card anchor"]. The card's numbers are thinking mode with sampling.
  - Pricing exact in that mode with assumed lengths gives about 4.6M tokens, about 90 h of the exact reference at about 14 tok/s.
  - We did not run it. Our gates are non-thinking greedy.
- **The repeat check** of the two-history rule: the same history must reproduce the same tokens. It is in the template but in no later pre-registration [K, EV:PREREG_TEMPLATE_cachedep].
- **The 12 GB line's reference:** its gates compare with REF2-T, not with an exact reference of its own mode (its loop is verified, not exact). This is open.
- **The 12 GB IFBench arm:** its harness passed the history test (one order allowed) [K, F12H]. It runs after the release.
- **The multilingual question:** MGSM's place under the 8 November rule (a known benchmark, but the card does not report it) is open, and that gate is on hold.

Appendix A: sources

Keys follow [MAIN] Appendix C. Paths are relative to the root of the public snapshot, github.com/kamenlevi/350-tps; commit hashes are those of each folder's own repository, and the snapshot holds the files as last committed. The keys:

- EV: = EvalSuite/docs/ (RESULT_, *PREREG_*);
- ES: = EvalSuite/ (code, results/);
- F12H = the 12 GB track's history test, Fit12/data/scr/chw18;
- BM: = BigMoE/, the engine;
- TR: = FableGod/TRIED/, the attempts logs.

Per-row data: each RESULT names its results//decode.jsonl and report files. In the public snapshot, the answer text of every GPQA Diamond row in the decode.jsonl files under ES:results/ reads "(withheld: GPQA Diamond)": the answers often quote the questions, and the benchmark is kept out of training data. Token ids, timings and verdicts are unchanged.

- EV:RESULT_ref2_2026-10-28, RESULT_ref2b_2026-10-31, RESULT_r58_2026-10-31, RESULT_ref2ct_2026-11-02, RESULT_ifx_2026-11-03 (with its 11-04 correction), RESULT_nax6_2026-11-05.
- EV:RESULT_r56_2026-10-30, RESULT_r56b_2026-10-31, RESULT_r56c_2026-11-01, RESULT_r56d_2026-11-01.
- EV:RESULT_hc8_2026-11-02, RESULT_relax_2026-11-05, RESULT_relax_drift_2026-11-06, RESULT_agentfast_2026-11-06, RESULT_stack_2026-11-07, RESULT_s4_2026-11-07, RESULT_

c30_2026-11-07, RESULT_cert_2026-11-08, RESULT_chat_m1b_2026-11-08, RESULT_card_lcoh_2026-11-09, RESULT_card_m1b_2026-11-09.

- EV:RESULT_plan4433333333, RESULT_t0_2026-11-05, RESULT_t80ck1u_2026-11-07, RESULT_pmin12_2026-11-01, RESULT_r41c_2026-10-28, RESULT_fastmode_b16, RESULT_smallram_2026-10-22.
- EV:PREREG_ref2_2026-10-28 (decision 4), PREREG_TEMPLATE_cachedep, PREREG_stack_2026-11-06, PREREG_s4_2026-11-07, PREREG_cert_2026-11-08, PREREG_chat_gate_2026-11-08, PREREG_card_gates_2026-11-08 (amendments 1-11), PREREG_pmin12_2026-11-01, PREREG_night_gates_2026-11-07, PREREG_t80ck1u_2026-11-07, PREREG_agentfast_2026-11-06.
- ES:docs/DESIGN.md (the first rule set); ES:evalsuite/decode.py (footprint_peak_gb); ES:evalsuite/ifbench and ES:evalsuite/lcb (the vendored graders, with their licenses); ES:tools/card_items.py, tools/card_keys.json (the card sets and keys); ES:results/card/ (line check).
- F12H (chw18_eq.txt, chw18_hist.txt): the 12 GB history test.
- BM:data/pfd/pfd_report_0009.txt (lines pdz4a and pde4k) and BM:data/r57/pdz4a_256_tokens.json, pdz4a_256_runs.json, pdz0_256_tokens.json, pdz0_256_runs.json, pde4k_256_runs.json: the PF-D v4 audit. Re-counted from the runs files on 11-09: tokens 12 / 12, logits hashes 721 / 721, and summed check counters 87,912 rows with 0 differing.
- BM:data/r57/z0_256_tokens.json (M1-B + COH), zs_256_tokens.json (+ SB1), yeh_256_runs.json (E) and yesh_256_runs.json (E + SB1), with BM:docs/WED_TESTS.md, section CHS5: the SB1 audit. Re-counted on 11-09: zs, zs + PF-D v4, z0's repeat and PF-D's control all equal z0 on 12 / 12; E vs E + SB1, 676 / 676 logits hashes.
- TR:evalsuite.md: the attempts log (the store plan's adoption line).

Use of AI tools

The code, the experiments and the text of this paper were produced with substantial help from AI coding agents. The author directed the work and is responsible for its content. The numbers were re-derived from the raw run records named beside them, and the references were opened by hand or by script.

References

- Jacob Austin et al. "Program Synthesis with Large Language Models." arXiv:2108.07732, 2021.
- Stella Biderman et al. "Lessons from the Trenches on Reproducible Evaluation of Language Models." arXiv:2405.14782, 2024.
- Charlie Chen et al. "Accelerating Large Language Model Decoding with Speculative Sampling." arXiv:2302.01318, 2023.
- Mark Chen et al. "Evaluating Large Language Models Trained on Code." arXiv:2107.03374, 2021.
- Karl Cobbe et al. "Training Verifiers to Solve Math Word Problems." arXiv:2110.14168, 2021.
- Thomas G. Dietterich. "Approximate Statistical Tests for Comparing Supervised Classification Learning Algorithms." Neural Computation 10(7): 1895-1923, 1998. doi:10.1162/089976698300017197.
- Abhinav Dutta et al. "Accuracy is Not All You Need." arXiv:2407.09141, 2024.
- EleutherAI. lm-evaluation-harness (software; MIT). https://github.com/EleutherAI/lm-evaluation-harness
- ggml-org. llama.cpp (software; MIT), tools/perplexity and its KL-divergence mode. https://github.com/ggml-org/llama.cpp
- Horace He and Thinking Machines Lab. "Defeating Nondeterminism in LLM Inference." Thinking Machines Lab: Connectionism, 10 Sep 2025. doi:10.64434/tml.20250910. https://thinkingmachines.ai/blog/defeating-nondeterminism-in-llm-inference/
- Dan Hendrycks et al. "Measuring Mathematical Problem Solving With the MATH Dataset." arXiv:2103.03874, 2021.
- Naman Jain et al. "LiveCodeBench: Holistic and Contamination Free Evaluation of Large Language Models for Code." arXiv:2403.07974, 2024.
- Eldar Kurtic et al. ""Give Me BF16 or Give Me Death"? Accuracy-Performance Trade-Offs in LLM Quantization." arXiv:2411.02355, 2024.
- Yaniv Leviathan, Matan Kalman and Yossi Matias. "Fast Inference from Transformers via Speculative Decoding." arXiv:2211.17192, 2022. ICML 2023.
- Percy Liang et al. "Holistic Evaluation of Language Models." arXiv:2211.09110, 2022.
- Hunter Lightman et al. "Let's Verify Step by Step." arXiv:2305.20050, 2023.

- Jiawei Liu et al. "Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation." arXiv:2305.01210, 2023.
- Quinn McNemar. "Note on the sampling error of the difference between correlated proportions or percentages." Psychometrika 12(2): 153-157, 1947. doi:10.1007/BF02295996.
- OpenAI. simple-evals (software; MIT). https://github.com/openai/simple-evals
- Valentina Pyatkin et al. "Generalizing Verifiable Instruction Following." arXiv:2507.02833, 2025.
- Qwen Team. Qwen3.8-Flash-Next (model weights; Qwen Community License 1.0). https://huggingface.co/Qwen/Qwen3.8-Flash-Next
- David Rein et al. "GPQA: A Graduate-Level Google-Proof Q&A Benchmark." arXiv:2311.12022, 2023.
- Donald J. Schuirmann. "A comparison of the two one-sided tests procedure and the power approach for assessing the equivalence of average bioavailability." Journal of Pharmacokinetics and Biopharmaceutics 15(6): 657-680, 1987. doi:10.1007/BF01068419.
- Jeffrey Zhou et al. "Instruction-Following Evaluation for Large Language Models." arXiv:2311.07911, 2023.

Companion papers:

- [KERNELS] Kamen Levi. Batch-invariant kernels on Apple's M5 GPU and its matrix units. 2026. A companion paper of this set.
- [MAIN] Kamen Levi. What It Takes to Run a 125B LLM Fast on a 64 GB Mac, and How to Go Faster. 2026. A companion paper of this set.
- [RELEASE] Kamen Levi. Running a 125B mixture-of-experts model on a 64 GB Mac: the engine package. 2026. A companion paper of this set.
- [SMALL-RAM] Kamen Levi. A 125B mixture-of-experts model in 32, 24 and 12 GB of memory on one Mac. Forthcoming (by 11 November 2026).

Drafting for a 125B mixture-of-experts model that does not fit in RAM: training its own MTP head, one drafter per text type, and the levers that failed

Kamen Levi
Independent researcher

9 November 2026

Abstract

We study the drafter side of exact speculative decoding for Qwen3.8-Flash-Next, a 125B mixture-of-experts model (48 layers, 512 routed experts per layer, 10 per token) whose experts stream from the internal SSD of one MacBook Pro (M5 Pro, 64 GB) ([MAIN]). The engine verifies up to 16 drafted rows per check and keeps the output bit-identical to one-token greedy decoding, so a drafter can change speed but never the output.

We compare two drafters: a 5-layer block drafter that proposes 16 positions in one pass (DFlash2-style), and the model's own multi-token-prediction (MTP) layer, run as a chain, as MTPLX does. Three findings:

- **Training the MTP layer the way it is used**, over its own chain (EAGLE-3's training-time test), raises its code kept tokens per 16-row check from 7.78 to 10.98 (x1.41) [K]. That is above the block drafter's 10.44 on the same answers. Chat moves by only 2-3 %, not significant.
- **The chained MTP head beats the block drafter on chat and agent text** (x1.08-1.32 tokens per check [K]) and loses on code (x0.75). So the engine routes by text type. On 19 fresh chat prompts the switch gave:
  - whole decode 23.1 -> 28.8 tok/s (+24.5 %);
  - whole request 18.6 -> 22.0 tok/s (+18.3 %);
  - first token 1.9 s in both arms;
  - lifetime peak 37.9 GB;
  - every token identical.
- **Most other levers failed.** Of 83 drafter-side items we tried (Appendix A), 41 ended in a KILL (by a pre-registered line, or at the desk or on paper), and 27 ended between lines, parked or as information.

Where we lose is chat. We keep 2.3-2.5 tokens per chat check, and the check's fixed cost is ~75-85 % of a chat check. So a drafter that never missed would still reach only ~105-115 tok/s of chat decode at today's check cost [P]. Against Strata on the same machine, our chat decode is about level, and Strata's first token wins every whole-request number up to 256 generated tokens. We also continued the chat head overnight on the model's own chat features. A memory guard stopped capture at 0.10M tokens, and the head kept x1.014 on held-out chat: a KILL by its own line, at 8 % of its planned data [K].

We also hit a training pitfall. MLX runs fp32 matmuls as TF32 unless told otherwise, which moved our drafter's training losses by 1.3-3.5 % against a float32 port.

Marks: [K] measured, [P] priced (arithmetic or a CPU replay on measured data), [G] judgement. Each number carries a source key; Appendix B maps the keys to files. GB = `10^9` bytes of the lifetime peak footprint (ri_ lifetime_max_phys_footprint, as the run reports print it); the 45 GB line is 41.9 GiB, and a memory line is the engine's own footprint on a 64 GB machine, not an OS limit; a 7.45 GB expert tier held as purgeable memory is not counted in it ([MAIN] section 2).

1. Setting

The model and its MTP layer.

- The model's MTP layer is one transformer layer with its own 512 experts (2.61 B parameters, [MAIN] section 3.1).
- It reads the trunk's widened hyper-connection stream (4 x 2560 = 10,240 wide) at position u plus the embedding of token u+1, and predicts token u+2.

- It shares the model's 248,320-token output layer.
- Chained, step j feeds the layer its own output state and its own previous token. The layer was trained for one step, so the stock chain decays: per-depth rate 0.98 at depth 1, 0.70 by depth 15 on code [K, R:'D-22 b3 REAL-RUNTIME RESULT'].

The engine ([MAIN], [KERNELS]) drafts up to 15 tokens per check and verifies them in one pass of 16 rows. Kept per check = accepted drafts + the bonus token. The check is exact to the engine's reference, so drafts never change the output; every live arm below reports token equality across arms.

Drafter A, the block drafter (L1-20).

- 5 layers, 1.21 GB, DFlash2-style: one pass drafts 16 positions from 5 taps of the trunk.
- Fine-tuned from Inco AI's public DFlash2 drafter for this model, incoai/Qwen3.8-Flash-Next-DFlash2 (Hugging Face; revision a22a174 of 19 Oct 2026; Apache-2.0).
- Recipe: forward KL to the model's top-32, block 32, prediction mask 0.5, lr 2e-5, on 99 windows of the model's own code and math answers [K, R:'P-CAP RESULT'; R:'E1 COVERAGE RESULT'].
- It drafts code, math and agent text in the shipped engine.

Drafter B, the MTP chain.

- The native MTP layer chained to K15.
- Cut where the running product of its top-1 probabilities falls below 0.3, with early exit.
- Run through MTPLX by Youssof Altoukhi (Apache-2.0; github.com/youssofal/mtplx), which first ran this model's own MTP head as a drafter on Apple Silicon. Our engine builds on its model code for this architecture.
- It drafts chat in the shipped engine.

Two kinds of measurement.

- **Teacher-forced walks:** the drafter at every position, or at the engine loop's own anchors, of fixed answers of the model. They give exact kept counts for that text, with no speed.
- **Live A/Bs** in the engine. Speeds follow the shared definitions ([MAIN]): whole decode, whole request, first token (median), lifetime peak, pooled over the set.
- The live drafter A/Bs ran on engine versions of 31 Oct - 2 Nov, before the current reference (NAX6-N). Their exactness is token equality across arms, plus equality to that version's own reference where the rows existed.

2. How we measured, and what went wrong in measuring

- **Lines first.** Every screen had a PASS / KILL line written before its run. The verdicts below are those lines' own.
- **Arm order.** On an SSD-streamed model, back-to-back arms on one prompt favour the second arm: it verifies on the experts the first arm just loaded. We measured the factor at x1.12-1.21 per type [P, R:'D-23c' CORRECTION']. One of our PASS reads (+10.6 %) fell to about +8.7 % after the correction, so it was withdrawn. From then on each arm ran its own pass over the set.
- **Retrain noise.** Retraining the block drafter with another seed moves its code kept by 0.16 (9.57 -> 9.73) and math by 0.01 [K, R:'SEED: L1-20 retrain noise']. Single-run code deltas under ~0.3 can't be told from noise. That turned one of our results (+0.11) into "no effect".
- **In-sample prompts**, caught twice:
  - the MTP head's chat windows included 3 of the IFEval prompts of a live A/B (R:'D-23c' CORRECTION');
  - the planned on-policy eval set for the overnight head was inside the old head's training data, and was replaced before the run (R:'F2A night', amendment 1).
- **Alignment.** Four cross-drafter read-outs read the MTP rows one position late. They were corrected the same day, and one lead was withdrawn (R:'ALIGNMENT FIX').
- **TF32 in MLX** (section 3.8).

3. Results

3.1 Training the native MTP head the way it chains

The first attempt failed.

- It trained each depth DeepSeek-style, over a fresh cache of the previous depth's outputs, on 15 agent transcripts.
- Agentic tokens per 16-row pass fell 4.04 -> 3.85 (-4.6 %, 90 % CI -8.4 .. -0.6), and general text -7.5 % [K, R:'The 125B MTP head fine-tune'].
- Depth 1, where training equals inference, rose (0.846 -> 0.858). Depths 2-4 fell, although their training loss fell ~45 %. At inference the chain attends to the true-pair history plus its own entries, not to a fresh cache of its outputs.
- The same read-out showed that putting a tuned head back into the stock format (4 bits per weight) keeps only 12-23 % of the change in those layers. 8-bit with group 64 keeps 99.9 % for ~29 MB more, so tuned heads ship at 8 bits or in bf16 [K, same section].
- Limit: this run's raw read-out files are not in our data tree; the report section holds its numbers.

D-22: training-time test inside the head's attention (EAGLE-3's idea, applied to the native layer).

- Depth j of anchor s takes the head's own depth-(j-1) output and token s+j. Its query sits at s+j-1, over keys = the true pairs up to s plus that anchor's own chain entries: exactly the MTP cache at inference.
- Loss: KL to the model's top-32 on assistant targets.
- Trained in fp32: the head's input projections, attention, router and shared expert. Frozen: its experts, hyper-connection mixers, norms and the output layer. 69.1M trainable parameters [K, R:'D-22 MTPTTT: train the native MTP head'; R:'F2A night', smoke].
- Results in MTPLX's own runtime on 32 held-out prompts, every answer token-identical to the stock-head run:

head	data	code K15 (8 answers)	gsm K15	IFEval K15	multilingual K15	source
stock	-	7.78	6.07	2.43	2.92	R:'D-22 b3 REAL-RUNTIME RESULT'
base (step 76)	99 code + math windows, depths 1-8	9.83 (x1.263, CI 1.185-1.366)	x1.048	-	-	R:'D-22 REAL-RUNTIME RESULT'; TL:data/ d22r_base76_vs_ stock.txt
b3	+ 73 IFEval windows, depths 1-15, one cosine epoch	**10.98** (x1.411, CI 1.287-1.565)	6.86 (x1.131)	2.49 (x1.024, n.s.)	3.00 (x1.028, n.s.)	R:'D-22 b3 REAL-RUNTIME RESULT'; TL:data/ d22r_b3_vs_stock.txt
b4	twice the chat windows (147)	10.65 (x1.368)	7.04 (x1.161)	2.51 (x1.031, n.s.)	3.03 (x1.038, CI 1.013-1.060)	R:'D-22 b4 REAL-RUNTIME RESULT'; TL:data/ d22r_b4_vs_stock.txt
block drafter (L1-20), same answers	-	10.44	6.60	2.24	2.22	R:'D-17 RESULT'

All rows are [K], tokens per 16-row pass (accepted + 1).

- **On code the trained chain becomes target-class:** ~0.97 per depth at every depth 1-15, against stock 0.98 -> 0.70. This took 172 training steps on one GPU [K].
- **Chat barely moves.** Its wall is depth-1 accuracy: IFEval 0.72, multilingual 0.80, code 0.98 [K, R:'D-22 b4 REAL-RUNTIME RESULT']. Twice the chat windows added ~1 %.
- **Stopping each anchor's loss at its chain's first failure (b5, AUF)** gave +0-3 %, significant on math only (MGSM x1.030, CI 1.018-1.042) [K, R:'D-22 b5 (Spec-AUF) RESULT']. b5 became the all-round head that ships for chat.
- **Under the engine's adaptive-width rule the tuned head paid nothing on code** (-8.9 % tokens per pass). The rule had been fitted to the stock head's confidences, and the tuned head is less sure where it is right. In a fixed 7-draft chain the same head gave code +12.5 % tok/s [K, R:'D-22 shipped-line A-B-A redo'].
- **At 24 GB it kept +5-8 % per pass at flat speed**, because decode there is SSD-bound [K, same section].
- **A 3-bit trunk under the head does no harm:** kept per 8-row check 4.02 -> 4.09 [K, R:'MTP3BIT'].

3.2 One drafter per text type

Teacher-forced, same answers [K, R:'D-17 RESULT']. Ratio of tokens per pass, MTP K15 / block drafter (90 % CI):

- code 0.746 (0.703-0.791);
- gsm 0.919;
- IFEval 1.084 (1.032-1.176);
- multilingual 1.315 (1.184-1.464);
- agent 1.295 (1.208-1.372).

A per-pass oracle over the two drafters would add +11 % on code and +43 % on agent text. A cheap router on the model's bonus-token probability got none of it.

Live, in four steps. All arms token-identical.

step	engine (date)	arms	whole decode	whole request	first token (median)	peak	verdict	source
D-20b	REF2b, no confidence cut, C 224 (1 Nov), 17 new prompts	block drafter / switch (MTP K7 on chat, math, agent)	15.95 / 20.30 (+27.3 %); chat 11.38 / 15.07	7.11 / 6.42; chat 9.18 / 11.61 (+26.5 %)	3.94 / 3.94 s	not logged	PASS	R:'D-20b SWITCH45 confirmation'; R:'PAPER SPEED'
D-23	the next default: the block drafter verifies only rows with running confidence >= 0.3 (1 Nov)	block drafter / the same switch	29.43 / 28.65 (-2.7 %)	7.09 / 7.04	3.80 / 3.82 s	42.4 GB	KILL	R:'D-23 SWITCHDEF RESULT'; R:'PAPER SPEED'
D-23c'	the same default, MTP K15 + the 0.3 cut + early exit, C 192, 18 prompts, arms back to back	block drafter / switch	+10.6 % by the letter, about +8.7 % order-corrected [P]	-	-	-	BETWEEN (a PASS withdrawn)	R:'D-23c' CORRECTION'
D-23d	the same, 23 fresh prompts, each arm its own pass (2 Nov)	block drafter / MTP on chat	chat 23.11 / 28.78 (+24.5 %; geo x1.223, 19 of 19 faster)	chat 18.56 / 21.95 (+18.3 %)	1.91 / 1.88 s	37.9 GB	PASS: chat ships on the MTP head	R:'D-23d CHATSWITCH RESULT'; R:'PAPER SPEED'; TL:data/d23d_read.txt

All speed cells are [K], tok/s, from the runs' own rows (R:'PAPER SPEED', TL:data/paper_speed.txt).

Reading.

- **The switch survived the next default only on chat.** That default made the block drafter verify fewer rows (3.9 per chat check), which the fixed 8-row MTP arm could not match. The MTP chain still kept more on chat (3.07 vs 2.71 per check): the loss was the verify width, not the head.
- **Under the same 0.3 cut, the MTP chain wins chat again.** The win is largest where the block drafter is weakest, in languages far from its code and math data. Kept per check on the 16 multilingual prompts went 2.09 -> 2.65. Gains ranged from +5-10 % (French) to +49-50 % (Bulgarian).
- **Agent text stays on the block drafter.** The MTP chain keeps far more per check there (4.80 -> 8.20). But its longer chains verify more rows, which touch more cold experts, so speed was flat.
- **Whole request barely sees the drafter on whole sets.** Long agent contexts took 62-89 s to prefill (median first token), and the prefill doesn't use the drafter. On short chat prompts, three quarters of the decode gain reaches whole request.

3.3 The block drafter: data, capacity, targets

All rows are teacher-forced walks, kept per 16-row check on held-out answers [K].

lever	code	math	verdict	source
data: 1/4, 1/2, all of the base windows	9.37 / 9.46 / 9.57	6.65 / 6.82 / 7.00	slope ~+0.13 per doubling (code)	R:'D-09 RESULT'

lever	code	math	verdict	source
+ the model's own MBPP answers, x2 / x4	9.79 / 9.85	6.99 / 7.06	PASS by its line, flattening; 5-point slope code +0.130, math +0.100 per doubling	R:'E2 / D2 RESULT (x2 + x4)'; TL:data/e2_read.txt
retrain with another seed	9.73	6.99	noise: 0.16 / 0.01	R:'SEED: L1-20 retrain noise'
twice the layers (10, 2.09 GB)	9.75	6.92	inside noise	R:'P-CAP RESULT'
off-policy code (public answers prefilled through the model)	9.68	-	no effect after SEED	R:'P-OFF 60K RESULT'

Killed by their own lines (Appendix A3 has the sources):

- mode-seeking targets (sharpened or total variation, x0.991-0.999);
- a little or ~11x more of the model's own text per language (multilingual kept 2.07 -> 2.06; 2.06 -> 2.05);
- the drafter borrowing the model's own experts;
- loss only at uncertain positions (accuracy there 0.8428 -> 0.8351);
- the MTP head's input stream fed to the block drafter;
- early-layer taps only;
- a second drafter trained to be wrong elsewhere (q = P(wrong given the first is wrong) 0.890-0.893 vs 0.901 for a plain twin);
- two block drafters, the more confident per depth (x1.026).

Reading. The block drafter's gaps are not cheap to close. More of the model's own code is the only lever that held, at ~+0.13 kept per doubling, and code >= 13 kept would need ~23 more doublings [P, R:'E2 / D2 RESULT (x2 + x4)'].

3.4 Combining drafters and picking candidates

- **The room is large.**
  - A perfect top-3 pick among the block drafter's 16 candidates keeps 11.57 on code (x1.209), all 16 12.37 (x1.292) [K oracle, R:'RERANK-0 RESULT'].
  - At the first miss, the model's token is the drafter's rank 2 in 54.7 % of checks.
  - At the model's mid-confidence rows its token is in the drafter's top-3 at 0.888 [K, R:'KWALL (a) RESULT'].
- **Nothing we built captured it.**
  - A per-depth pick between the two drafters keeps x1.073 on code. The MTP chain's ~33 ms per cycle costs more than that, so it is a KILL on paper [K kept, P speed, R:'MTPPICK aligned'].
  - A learned certificate head on the drafter's features: KILL.
  - The MTP head as a second opinion in a learned stop head: KILL (code -1.1 %).
  - Past-only bandit switches: KILL.
  - A per-type confidence cut: within 0-2 % of the shipped 0.3.
- **Trees.**
  - Leaf rows at the MTP chain's low-confidence depths price at -7 % .. +0.9 % on every set [P, R:'D-24 MTPTOP3']. A leaf row costs 7.8 ms, and a rescued chat token is worth ~34 ms.
  - Wider trees need a tree verify for the model's linear-attention layers, which we did not build.
  - 32-row checks keep 12.60 on HumanEval+ and 6.10 on MBPP, under their lines of 14 and 7.5 [K, R:'DEPTH32 RESULT'].

3.5 Copying from the context

- **Copying pays on code and ships there.** Overriding the draft with the continuation of the longest context match keeps x1.038 more per check on 24 HumanEval+ answers in the walk [K, R:'MATCH RESULT']. In the engine, on 12 HumanEval, it keeps x1.08 (geo) / x1.11 (pooled: 11.16 vs 10.01) more per check [K, KS], and whole decode moves x1.077, pooled (125.2 vs 116.2 tok/s) [K, BM:data/r57]. HumanEval answers copy their prompt's docstring, so this flatters: the same drafter keeps 5.94 per check on MBPP against 9.57 on HumanEval+ [K, KS].
- **On chat, copying loses.** Prompt lookup or suffix drafting alone gives x0.44-0.59 tok/s. With the MTP chain it gives x0.89-0.996, and even an oracle choice between the two is x1.012 [P on measured check costs, R:'CHAT150'].
- **Other lookups failed:**
  - the model's own n-gram table as a predictor;
  - computed calculator tokens in math (2.28 % of tokens);

  - schema filling for tool calls (2.2 % of agent first misses fillable);
  - a reduced draft vocabulary: KILL (Appendix A5).

3.6 What a draft step costs

- **The output layer dominates the step.** One MTP draft step takes ~2.2 ms; the 248,320-token output layer is 1.47 ms of it. That runs at 242 GB/s, 85-94 % of the MacBook Pro's streaming bandwidth [K, R:'HEADBENCH']. Measured in the engine's own path, a drafted chat row costs 2.59 ms [K fit, R:'CHAT150'].
- **The output layer can't be cut.** At 3 bits it loses 5 % of kept on IFEval, at 2 bits 14 % [K, R:'D1: the chat draft step'].
- **The head's trained parts at 4 bits** are neutral on the exact config: whole decode 36.6 -> 36.9 tok/s, 12 / 12 equal to one-token decoding. On the lossy config they lose: whole decode 43.80 -> 40.22 tok/s (-8.2 %), draft per check 7.9 -> 13.4 ms pooled (8.7 -> 14.9 as the mean over prompts), lifetime peak 42.37 vs 42.28 GB [K, R:'D1 v1b in the engine'; BM:data/chs/chs4_report.txt]. The draft step stays in bf16.
  - Limit: whole request and first token were not in that run's per-arm report.
- **Getting the draft off the critical path.**
  - Pre-drafting the next block during the check is exact: 281 of 281 pre-drafts were identical [K, R:'DA-1 RESULT'].
  - But it pays only with ~20-33 ms of idle GPU per check, and today's 45 GB check leaves <= ~2.5 ms [P, R:'DA-1 re-priced'].
  - Drafting on the CPU: the output layer alone takes 20.0 ms in fp32, 38.7 ms beside a busy GPU, against 1.47 ms on the GPU: KILL [K, R:'CPUMTP'; TL:data/cpu_head_bench.txt].
  - The MTP layer as a one-pass block drafter can't work. In one pass, rows 2-15 have no known token and no chain state: KILL by analysis [R:'MTP-BLOCK'].

3.7 Chat: the check-cost wall

- **Chat's check cost, fitted on the engine's own per-check records** (12 IFEval, 45 GB) [K, R:'CHAT150']:
  - exact config: 47.0 ms + 3.25 ms per verified row + 2.59 ms per drafted row. Estimator: least squares of the cycle wall minus its measured draft time, on rows, over checks >= 2 (TL:sticky/chat_ngram.py). A plain least-squares fit of the check time alone on the same checks gives 44.9 + 4.00 ms per row (the estimator of our independent re-derivation).
  - lossy config: 33.1 ms + 4.35 ms per row;
  - the fixed part is ~75-85 % of a chat check at 2-4 rows under either fit.
- **A drafter that never missed** would reach ~105-115 tok/s at 16 rows with the serial MTP draft (the two fits; both extrapolate from checks of mostly 2-4 rows), or ~135-150 with a one-pass draft [P]. 150 tok/s of chat decode needs the check's fixed part cut ~2.5x and a head ~4x faster.
- **Chat acceptance itself.** The shipped head keeps 2.34 (IFEval) / 2.54 (multilingual) per check [K replay]. The levers that remain on the drafter side are small:
  - a re-tuned cut: x1.015;
  - a 2x / 4x faster head: x1.095 / x1.138;
  - tree leaves, which need a tree verify: x1.009-1.044.
- **F2a, an EAGLE-3-style continuation of the chat head** on the model's own features. Text: UltraChat-200k + Aya; labels: the model's own top-32; the D-22 trainer, 2 epochs, from b5.
  - Before the run it was priced at x1.1-1.3 kept on IFEval [G, R:'F2A feasibility'].
  - Measured [K, R:'F2A night RESULT'; TL:data/f2a/f2a_eval.txt], kept per check, b5 -> F2a:
    - held-out IFEval 2.301 -> 2.334 (x1.014);
    - held-out multilingual 2.507 -> 2.540 (x1.013);
    - on-policy IFEval (24 fresh greedy answers) 2.444 -> 2.450 (x1.002).
  - KILL by its pre-registered line (held-out < x1.03).
  - Limit: the swap guard stopped capture after 27 min at 0.10M tokens, 8 % of the planned ~1.3M. So this kills F2a at 0.1M tokens, not the data-rich plan.
  - Capture ran at 76-80 tok/s [K]. 1.3M tokens therefore need ~4.6 h of quiet machine [P].

3.8 TF32 in MLX

- **What happened.** We ported the block drafter's MLX trainer to PyTorch for free cloud GPUs. The port matched MLX's step-1 loss to 1e-4 but sat 1.5-3.8 % above the logged losses at steps 2-5.
- **The cause** [K, R:'PORT steps FAIL: ROOT CAUSE FOUND']:
  - MLX 0.32.2 runs fp32 matmuls as TF32 unless MLX_ENABLE_TF32=0 (its utils.h default).
  - fp32 matmul error against float64 is 4.15e-4 by default, 6.5e-7 with TF32 off.
  - A 1e-5 relative weight change moves the output ~51x more than it should.
- **The check.** With TF32 off, MLX's own trainer matches the port within 1.53e-3. TF32 on vs off moves the loss -1.3 .. -3.5 % at steps 2-5 [K, R:'PORT full-scale confirmation'].
- **Consequence.** Every "fp32 master weights" MLX run of ours trained under TF32. The logs are deterministic but are not float32 arithmetic.
- Prior reports: MLX documents the default (its docs, "Environment variables": MLX_ENABLE_TF32, "The default is 1"; and "Precision"), and MLX issue #4595 (29 Sep 2026) reports its cost for fp32 convolutions on the M5 since 0.32. We claim only that we hit it.

4. Early headlines that were wrong

The set's withdrawn headlines ([MAIN]): "12 GB, 40.8 tok/s 'exact'" (a FAST arm; withdrawn); "45 GB FAST stack, 157.9 tok/s" (included CD16, which failed MBPP; withdrawn); "180.3 tok/s" was the registered steady metric, which is 142.8 on whole decode, pooled (metric corrected 7 Nov; the correction note's 142.1 is a geo mean). On the drafter track:

- **"The MTP chain doesn't beat the block drafter on chat"** (a desk check on depths 1-3). Overturned by measurement: x1.08 IFEval, x1.32 multilingual (R:'D-17 RESULT').
- **"The tuned head is +10-33 % on the shipped line."** It always ran second, on experts the stock arm had just loaded (R:'D-22 on the shipped 45 GB line, first A/B').
- **"The MTP chat switch passes, +10.6 %."** Withdrawn the same morning for arm order and in-sample prompts; BETWEEN (R:'D-23c' CORRECTION').
- **"The two drafters are wrong at different places (q 0.301)."** An off-by-one read. Aligned, q is 0.436, and four read-outs were corrected (R:'ALIGNMENT FIX').
- **"Off-policy data adds +0.11 kept."** No effect, once retrain noise (0.16) was measured (R:'SEED').
- **"One epoch over 20M tokens of cloud training in ~2-3 h."** About 15x too low: ~35-45 h on 2 x T4 [P, TL:notes/PORT_PLAN.md].
- **A wrong claim in our own write-up** ("the chat head was trained on code and math only"). 5 of 8 IFEval keys of the next test were in its training. Caught before that test ran.
- **CD16**, keeping the drafter's comment text unchecked: x1.20 kept on HumanEval, withdrawn after MBPP's tests failed ([MAIN] section 4; KS).

5. Where we lose

- **Against Strata.** Strata (MIT; github.com/Niko1221/Strata) ran on the same machine and the same 48 prompts. Our decode is ahead on code and math and about level on chat, our exact chat decode is behind, and Strata's first token (0.5-1.0 s) beats ours (about 3-4.6 s). So up to 256 generated tokens Strata wins or ties every whole-request number. Source: F:2026-11-08_1630_strata-head-to-head ([MAIN] has the table).
- **On chat, from the drafter's side:**
  - we keep 2.3-2.5 tokens per check;
  - the MTP head's next-token accuracy on IFEval is 0.73-0.76, against 0.98 on code;
  - Strata's own MTP drafting accepts 87 % of drafted tokens and gives it +53 % decode (25.3 -> 38.7 tok/s pooled) [K, our run of Strata, same source]. Its draft length per step is not in our records.
- **Data.** Capturing the model's own features runs at 76-80 tok/s here [K], so a night gives ~1-1.3M tokens at best; our one overnight run got 0.10M before a memory guard stopped it. EAGLE-3 trains its chat heads on far more: ShareGPT and UltraChat-200K, about 68K and 464K conversations, with the responses regenerated by the target model (arXiv 2503.01840; it gives no token count; self-reported).
- **Code gains are HumanEval-flavoured.** MBPP keeps 5.94 per check against HumanEval+'s 9.57 with the same drafter (KS).

6. Related work

Every entry below was checked against its source by the set's related-work check (Mon 9 Nov; arXiv ids against their titles). What each does, and what we took:

- **Speculative decoding** (Leviathan et al. 2023, arXiv 2211.17192): the exact accept rule our check implements.
- **MTPLX** by Youssof Altoukhi (Apache-2.0; github.com/youssofal/mtplx): ran this model's MTP head as a chained drafter on Apple Silicon. We use its model code and its chain; our trained heads load into it.
- **DeepSeek-V3's MTP and FastMTP** (2412.19437, 2509.18362): train MTP modules for multi-step drafting. FastMTP's recursion training is the closest prior to D-22.
- **EAGLE, EAGLE-2, EAGLE-3** (2401.15077, 2406.16858, 2503.01840): feature-level drafters. EAGLE-3's training-time test is D-22's training scheme, applied inside the native layer's attention.
- **Training the chain the way it runs:** AngelSpec (Liu et al., 2607.25852), one training framework for MTP and block drafters; and Badarayani et al., "Match the Distribution, Not the Compute: Post-Training Multi-Token Prediction Heads" (2610.00888), which post-trains chained MTP heads on a frozen model's own outputs.
- **DFlash** (2602.06036), its DFlash2 drafters, and DFlare (2606.02091): block drafters. Our block drafter is a fine-tuned DFlash2.
- **Medusa, SpecInfer, Sequoia, SpecExec, EAGLE-2's dynamic trees** (2401.10774, 2305.09781, 2402.12374, 2406.02532, 2406.16858): token trees. Our priced trees use their costs.
- **FR-Spec** (2502.14856): reduced draft vocabularies (killed here).
- **Prompt lookup, SuffixDecoding, SAM-Decoding** (SuffixDecoding 2411.04975; SAM-Decoding 2411.10666): copying (MATCH ships on code).
- **ToolSpec** (2604.13519): schema-guided tool drafting (killed here).
- **SpecDec++ and DISCO** (2405.19715, 2405.04304): learned acceptance and stop heads (killed here).
- **SP-MoE and Specter** (2510.10302, 2511.14102): expert prefetch inside speculative decoding (Appendix A7).
- **Ghidorah, DuoDecoding, PEARL** (2505.23219, 2503.00784, 2408.11850): parallel or CPU-side drafting (killed here).
- **WhiFlash** (2606.07710): switches between an autoregressive and a diffusion drafter token by token; our switch picks one drafter per task ([MAIN] cites it).
- **Strata** (MIT): MTP drafting in a streamed-expert engine (section 5).

7. What we left on the table

Measured room we did not use:

- **A better pick.** x1.209 (top-3) / x1.292 (all 16) on code with a perfect pick among the block drafter's candidates [K oracle]. The model's token is in the drafter's top-3 at 0.888 of mid rows [K].
- **A per-pass choice of drafter.** Oracle: +11.3 % code, +43.1 % agent, +18.6 % IFEval, +43.4 % multilingual [K, R:'D-17 RESULT'].
- **Width on chat.** An oracle width of 2 / 3 at every depth keeps 3.6 / 4.05 tokens per pass against 2.3-2.5 cut [K, R:'D-24 MTPTOP3']. It needs a cheap tree verify for the linear-attention layers.
- **Draft-ahead.** Exact (281 / 281); it pays once the check leaves >= ~20-33 ms of GPU idle [K / P].
- **Data.** +0.13 kept per doubling of the model's own code answers, still rising [K]. The data-rich chat-head run (~1.3M tokens, ~4.6 h of capture [P]) and its data-scaling arm did not run before the release.
- **Lossy acceptance on chat.** Near-tie acceptance gives +8 .. +23 % tokens per pass at rho 0.25 on stored chat answers [K replay, lossy, ungated, R:'R-61N RESULT']. The relaxed rules that passed a quality gate are in [EVAL].
- **The chat check's fixed part.** 75-85 % of a chat check [K]: not drafter room, but the drafter's ceiling.

8. Reproducibility

- Training: TL:sticky/d22_ttt.py, the D-22 heads; TL:sticky/f2a_*.py, the overnight chat head; runners in TL:turns/.
- Walks and replays: TL:sticky/d02b_walk.py, d17_*.py, d24_*.py, chat_ngram.py, paper_speed.py.

- Heads: TL:ckpt/d22_b5.safetensors (the shipped chat head; not in the public snapshot) and its training state.
- Raw data: TL:data/.
- The engine loads the chat head named by R57_MTP_HEAD (default b5) and checks its md5 [K, BM:scripts/r57_screen_fast.py].
- Software: MLX pinned to 0.32.2 with the pad-to-64 fix for MLX issue #4632 ([KERNELS]); 0.32.4 changes tokens against our exact references.
- Code: GPL-3 with the engine ([RELEASE]).

Appendix A. Every drafter-track item, its verdict and its source

Verdicts come from each item's own line, written before its run, unless the row says desk / on paper (a replay or a cost model, no live run). "R:" = a report section found by the words given. Rows marked (log) have no report section; their verdict is from the session log and carries no number here.

A1. Training the native MTP head

item	what	verdict	source
mtp125n_d4	DeepSeek-style depth training (each depth on a fresh cache), 15 agent transcripts	KILL	R:'The 125B MTP head fine-tune'
re-store at 4 bits	put a tuned head back into the stock format (4 bits per weight)	KILL (store tuned heads at 8-bit or bf16)	R:'The 125B MTP head fine-tune'
D-22 base	training-time test inside the head's attention (EAGLE-3 style), depths 1-8, code + math windows	PASS	R:'D-22 REAL-RUNTIME RESULT'
D-22 2 epochs	the same, longer	KILL	R:'D-22 interim'
D-22 b3	depths 1-15 + 73 IFEval windows + one cosine epoch	PASS	R:'D-22 b3 REAL-RUNTIME RESULT'
b3 in the shipped adaptive-width line	the tuned head inside a width rule fitted to the stock head	KILL as is (pays in a fixed chain)	R:'D-22 shipped-line A-B-A redo'
b3 at 24 GB	the tuned head where decode is SSD-bound	KILL for speed (kept +5-8 %)	R:'D-22 shipped-line A-B-A redo'
D-22 b4	twice the chat windows	BETWEEN	R:'D-22 b4 REAL-RUNTIME RESULT'
D-22 b5	loss stopped at each chain's first failure (AUF)	BETWEEN (kept as the all-round head)	R:'D-22 b5 (Spec-AUF) RESULT'
MTP3BIT	the head over a 3-bit trunk	no harm	R:'MTP3BIT'
F2a	the head continued on the model's own chat features (0.10M tokens; 1.3M planned)	KILL by its line, at 8 % of the planned data	R:'F2A night RESULT'

A2. Which drafter for which text

item	what	verdict	source
D-12	desk check: MTP chain vs block drafter on chat, depths 1-3	KILL at the desk, overturned by D-17	(log)
D-17	the MTP chain to K15 vs the block drafter, same answers	PASS (chat / agent)	R:'D-17 RESULT'
D-17 router	MTP or block per pass from the model's bonus-token p	KILL	R:'D-17 RESULT'
D-17 oracle / 2-branch tree	both chains in one check	PARKED (no selector; rows cost bytes)	R:'D-17 RESULT'
D-18	the MTP head's first guess revealed to the block drafter	KILL (desk)	R:'D-18 MTP1'
D-19	past-only switch rules (bandits) vs a text-type rule	KILL; the type rule PASS	R:'D-19 RESULT'
D-05	MTP drafts the near depths, the block drafter the rest	BETWEEN (not worth a build)	R:'D-05 MTPHYB + D-08 CALC walk'
D-20	type switch live, REF2b, C 160	PASS	R:'D-20 SWITCH45 RESULT'
D-20b	the same at C 224 on new prompts	PASS	R:'D-20b SWITCH45 confirmation'
D-23	the same switch on the newer default (confidence cut on the block drafter)	KILL	R:'D-23 SWITCHDEF RESULT'

item	what	verdict	source
D-23b	CPU replay: the MTP chain under the same cut	PARKED, measured by D-23c'	R:'D-23b MTP TRUNC'
D-23c'	MTP K15 + cut + early exit, live	BETWEEN (a PASS withdrawn: arm order + in-sample prompts)	R:'D-23c' CORRECTION'
D-23d	chat switch on 23 fresh prompts, arms in separate passes	PASS (shipped: chat on the MTP head)	R:'D-23d CHATSWITCH RESULT'
D-23e	the chat switch at C 224	DROPPED before any run	R:'D-23e DROPPED'
D-23x	the first check after a long prefill, read-traced	measured (a cold pool after the prefill)	R:'D-23x RESULT'

A3. The block drafter: data, capacity, targets, inputs

item	what	verdict	source
D-02a	the block drafter on chat; how sure the model is at its first misses	measurement	R:'D-02a RESULT'
D-09	1/4, 1/2, all of the base data	BETWEEN (a slow lever)	R:'D-09 RESULT'
D-10	soft KD targets	withdrawn: the recipe already uses forward KL to the model's top-32	(log)
D-13	mode-seeking targets (sharpened, total variation)	KILL	R:'D-13 RESULT'
D-14 C	a longer drafter window for agent text	KILL (stored data)	R:'D-14 probes'
D-15a	a little of the model's own text per language	KILL	R:'D-15a RESULT'
D-16	the MTP head's input (the widened hyper-connection stream) fed to the block drafter	KILL at the rapid protocol (may under-read)	R:'D-16 RESULT'
D-21	a second refinement step per block	KILL (measured earlier)	(log)
E1	~18K of the model's own answer tokens in each of 3 languages	KILL	R:'E1 COVERAGE RESULT'
E5	the block drafter borrows the model's own experts	KILL	R:'E5 DFlash-X RESULT'
HPD-1	2 more epochs with the loss only at uncertain positions	KILL	R:'HPD-1 RESULT'
E2 / D2	one and two more doublings of code data (the model's own MBPP answers)	PASS (a flattening slope)	R:'E2 / D2 RESULT (x2 + x4)'
EARLYTAP	read only early layers' features (so drafting can overlap the check)	KILL	R:'EARLYTAP RESULT'
DIVERSE2	a second drafter trained to be wrong elsewhere	KILL	R:'DIVERSE2 RESULT'
DUALDRAFT	two block drafters, the more confident per depth	KILL	R:'DUALDRAFT RESULT'
P-CAP	twice the layers (block expansion)	BETWEEN (inside retrain noise)	R:'P-CAP RESULT'
P-OFF	off-policy code answers prefilled through the model	ALIVE as data, no effect on kept	R:'P-OFF diagnostic RESULT'; R:'P-OFF 60K RESULT'
SEED	the base recipe rerun with another seed	measurement (retrain noise)	R:'SEED'
#20 BIGDRAFT	a block drafter 4-8x larger	PARKED (desk; needs paid compute)	R:'#20 BIGDRAFT'
TTS1	test-time LoRA on the block drafter after every check	KILL	R:'TTS1'
D7B-1	the block drafter at 4 bits vs bf16	information	R:'D7B-1'
MTP-BLOCK	the MTP layer as a one-pass block drafter	KILL by analysis, no run	R:'MTP-BLOCK'

A4. Combining drafters, picking candidates, trees, cuts

item	what	verdict	source
D-24 leaves	rank-1 / rank-2 leaf rows at the MTP chain's low-confidence depths	KILL (cost model on measured ranks)	R:'D-24 MTPTOP3'
D-24 K15 + cut	K15 vs K7 under the 0.3 cut	PASS on paper (then D-23c')	R:'D-24 MTPTOP3'
D-24 cut	retune the 0.3 cut for the MTP chain	KILL (keep 0.3)	R:'D-24 MTPTOP3'
R-61N near-tie	accept the drafter's token if its p >= rho x the model's top-1 p (lossy)	BETWEEN (ungated)	R:'R-61N RESULT'
R-61N top-3	a 2-3 way branch at the block drafter's first misses	PARKED (a lead)	R:'R-61N RESULT'

item	what	verdict	source
K-HEAD	a learned certificate head on the drafter's features	KILL	R:'K-HEAD RESULT'
KWALL (a)	is the model's token among the drafter's candidates at mid rows?	PASS (a selector problem)	R:'KWALL (a) RESULT'
KWALL (b)	the MTP head's depth-1 accuracy by the model's confidence class	BETWEEN	R:'KWALL (b) RESULT'
MTPPICK	the more confident of the two drafters per depth, one chain	KILL on paper (aligned)	R:'MTPPICK aligned'
MTPSCREEN	the same pick with a screened (cheaper) MTP head	PASS by its line, but the pick is small	R:'MTPSCREEN RESULT'
TOP3-USE	trees of top-3, a learned picker, or MTPPICK, priced	KILL on paper	R:'TOP3-USE'
RERANK-0	the ceiling of a perfect pick among the 16 candidates	PASS at its edge (a ceiling, not a lever)	R:'RERANK-0 RESULT'
SELGAP	the block drafter's own selector on vs off	OPEN, not queued	R:'SELGAP'
CUTSWEEP	a per-type cut for the block drafter	no knob (within 0-2 %)	R:'CUTSWEEP'
DEPTH32	32-row checks with both drafters	KILL	R:'DEPTH32 RESULT'
M-23b	the MTP head as a second opinion in a learned stop head	KILL	R:'M-23b FIRSTMISS-MTP RESULT'
ANCHORCOV	pre-drafting the next block from 3 anchors	KILL by its rule	R:'ANCHORCOV RESULT'
TREE3 + STAGED	aligned top-3 tree data, priced for staged verification	data + price	R:'TREE3 + STAGED'

A5. Copying from the context and other lookups

item	what	verdict	source
D-03	the model's own n-gram table as a next-token predictor	KILL	R:'D-03 NGRAM'
D-08	compute calculator-determined math tokens exactly	KILL	R:'D-05 MTPHYB + D-08 CALC walk'
MATCH	the longest context match overrides the draft	BETWEEN (walk); shipped in the engine's code profile	R:'MATCH RESULT'; KS
COPYGATE	MATCH behind a precision gate	BETWEEN	R:'COPYGATE RESULT'
TOOLSPEC (a)	schema-FSM fill of agent first misses	KILL	R:'TOOLSPEC'
FR-Spec vocab	draft over a reduced vocabulary	KILL (desk)	R:'FR-Spec-style reduced draft vocab'
CHAT150 lookup	prompt lookup / suffix drafting on chat, alone and with the MTP chain	KILL	R:'CHAT150'
T16-HE	comment rows kept when the model ranks them top 3 (lossy, code)	KILL (kept +3 %, cycle +1.9 ms)	R:'T16-HE'

A6. The draft step's cost, and getting it off the critical path

item	what	verdict	source
HEADBENCH	what the MTP step's output layer costs	KILL for a kernel (near bandwidth)	R:'HEADBENCH'
D1	the step's anatomy; output layer at 3 / 2 bits; trained parts at 4 bits; no-sync chain	output-layer cuts KILL; parts at 4 bits small	R:'D1: the chat draft step'
D1 v1 / v1b	the trained parts at 4 bits, in the engine	neutral exact, KILL lossy	R:'D1 v1b in the engine'
CPUMTP	MTP draft steps on the CPU during GPU waits	KILL	R:'CPUMTP'
DA-0 / DA-1	pre-draft the next block during the check	BETWEEN / PASS (exact), but it pays only with idle GPU	R:'DA-0 RESULT'; R:'DA-1 RESULT'
DA-1 re-priced	on the GPU-bound 45 GB check	slower (no idle GPU)	R:'DA-1 re-priced'
DRAFTOFF	what it takes to take drafting off the critical path	estimate	R:'DRAFTOFF estimate'
DRAFT17 / D17-1	a 17th draft position for the bonus row	answered (cost) / information	R:'DRAFT17 RESULT'; R:'D17-1'
CH3	a 2x / 4x faster MTP step, priced on chat	x1.095 / x1.138 [P]	R:'CHAT150'

A7. Adjacent (the SSD prefetch side, run on drafter data)

item	what	verdict	source
ROUTEPRED	a learned next-check expert-route predictor	KILL on paper	R:'ROUTEPRED'
LEAD1CORR	a learned next-layer forecast	information	R:'LEAD1CORR'
S2	a cheap chain one cycle early as a route predictor	PASS by the letter, small in speed	R:'S2'
SAMELAYER-T	a ridge-trained same-layer forecast router	screened, then fit and delivered	R:'PR160 read check + SAMELAYER-T'; R:'SAMELAYER-T FIT RESULT'

A8. Methods

item	what	verdict	source
PORT	a PyTorch port of the block drafter's MLX trainer (for free cloud GPUs)	validated; found MLX's TF32 default	R:'PORT steps FAIL: ROOT CAUSE FOUND'; R:'PORT full-scale confirmation'
DLOG	log the model's own answers from the live exact loop as training data	a tool	R:'E5 DFlash-X' (its DLOG paragraph)
PAPER SPEED	whole request and first token from the live A/Bs' rows	derivation	R:'PAPER SPEED'

Appendix C. Headline numbers and their sources

One row per headline number in the text; every row was re-read from its source on Sun 11-08.

#	result	number	mark	config	source
1	Native MTP head trained with the training-time test (b3), MTP runtime, 8 code answers	tokens per 16-row pass 7.78 -> **10.98** (x1.411, 90 % CI 1.287-1.565); per-depth rate ~0.97 at every depth 1-15 (stock 0.98 -> 0.70); the block drafter on the same answers: 10.44	[K]	drafts only; every answer token-identical to the stock-head run	R:'D-22 b3 REAL-RUNTIME RESULT'; TL:data/d22r_b3_vs_stock.txt
2	Same head (b3), other sets	gsm 6.07 -> 6.86 (x1.131, CI 1.093-1.180); IFEval 2.43 -> 2.49 (x1.024, n.s.); multilingual 2.92 -> 3.00 (x1.028, n.s.)	[K]	same	same
3	Twice the chat windows (b4)	IFEval 2.51 (x1.031, n.s.), multi 3.03 (x1.038, CI 1.013-1.060), code 10.65 (x1.368). Chat's wall is depth-1 accuracy: IFEval 0.72, multi 0.80, code 0.98	[K]	same	R:'D-22 b4 REAL-RUNTIME RESULT'; TL:data/d22r_b4_vs_stock.txt
4	Loss stopped at each chain's first failure (b5, AUF)	+0-3 %, significant on math only (MGSM K15 x1.030, CI 1.018-1.042); code and chat flat	[K] teacher-forced	drafter only	R:'D-22 b5 (Spec-AUF) RESULT'; TL:data/d22_b5_vs_b4.txt
5	The earlier DeepSeek-style tune (each depth on a fresh cache)	agentic tokens per 16-row pass 4.04 -> 3.85 (-4.6 %, 90 % CI -8.4..-0.6); general text -7.5 %; depth 1 rose (0.846 -> 0.858), depths 2-4 fell. KILL	[K]	greedy output token-identical to the stock run	R:'The 125B MTP head fine-tune (exact-safe ...): killed'; raw read-out files not in our data tree (stated limit)

#	result	number	mark	config	source
6	Drafter by text type, teacher-forced, same answers	MTP K15 / block drafter, tokens per pass: code 0.746 (0.703-0.791), gsm 0.919, IFEval 1.084 (1.032-1.176), multi 1.315 (1.184-1.464), agent 1.295 (1.208-1.372)	[K]	drafter only	R:'D-17 RESULT'; TL:data/d17_route.txt, d17_ci.txt
7	The type switch live, no confidence cut (REF2b, C 224, 17 new prompts)	whole decode, whole set 15.95 -> 20.30 tok/s (+27.3 %); pooled per type, block drafter -> MTP K7: chat +32.4 % (11.38 -> 15.07; faster on 6 of 6 prompts), math +19.8 %, agent +42.0 %, code -14.7 % (so code stays on the block drafter). Whole request: chat 9.18 -> 11.61 (+26.5 %); whole set 7.11 vs 6.42, because 3 agent prompts' first tokens (median 62-66 s) dominate and their prefills ran longer in the MTP arm. First token median 3.94 s both. Peak not logged (stated limit)	[K]	exact (all arms' tokens equal)	R:'D-20b SWITCH45 confirmation'; R:'PAPER SPEED'; TL:data/d20_ runs.jsonl, data/paper_ speed.txt
8	The same switch on the next default (block drafter verifies only rows with running confidence >= 0.3)	whole set 29.4 -> 28.6 tok/s (-2.7 %): KILL; peak 42.4 GB. The MTP arm still kept more on chat (3.07 vs 2.71 per check) but verified a fixed 8 rows against 3.9. Whole request 7.09 vs 7.04; first token median 3.80 / 3.82 s	[K]	exact	R:'D-23 SWITCHDEF RESULT'; R:'PAPER SPEED'; TL:data/d23_ runs.jsonl, data/paper_ speed.txt
9	MTP K15 + the same 0.3 cut + early exit (D-23c', 18 prompts, C 192)	+10.6 % whole set by the letter, then BETWEEN after the order correction: about +8.7 % (a model with a constant w per type)	[K] raw, [P] corrected	exact	R:'D-23c' at C 192 FINAL'; R:'D-23c' CORRECTION'; TL:data/d23c_ mtptrunc.txt
10	Chat switch on 23 fresh prompts, each arm its own pass (D-23d, C 192)	chat time-weighted +24.5 %, geo x1.223, faster on 19 of 19; multi +27.4 % (kept per check 2.09 -> 2.65); IFEval +1.4 % (n = 3); agent -0.5 % (stays on the block drafter). Chat whole decode 23.11 -> 28.78 tok/s; whole request 18.56 -> 21.95 (+18.3 %); first token median 1.91 / 1.88 s; lifetime peak 37.9 GB	[K]	exact (every token equal across arms)	R:'D-23d CHATSWITCH RESULT'; R:'PAPER SPEED'; TL:data/d23d_ read.txt, data/paper_ speed.txt, logs/d23d.out
11	Block drafter, data slope	code kept per check 9.37 / 9.46 / 9.57 at 1/4, 1/2, all of the base data; + the model's own MBPP answers x2 9.79, x4 9.85. 5-point slope: code +0.130, math +0.100 per doubling	[K] teacher-forced	drafter only	R:'D-09 RESULT'; R:'E2 / D2 RESULT (x2 + x4)'; TL:data/e2_read.txt

#	result	number	mark	config	source
12	Retrain noise	one reseed: code 9.57 -> 9.73 (0.16), math 7.00 -> 6.99	[K]	drafter only	R:'SEED: L1-20 retrain noise' (its RESULT)
13	Capacity, 10 layers (2.09 GB) vs 5 (1.21 GB)	code 9.57 -> 9.75, math 7.00 -> 6.92: inside retrain noise	[K]	drafter only	R:'P-CAP RESULT'
14	Off-policy code data	+0.11 code: no effect once retrain noise was known	[K]	drafter only	R:'P-OFF 60K RESULT'
15	A perfect pick among the block drafter's candidates (ceiling, not a lever)	code 9.57 -> top-3 11.57 (x1.209), all 16 12.37 (x1.292); at the first miss the model's token is the drafter's rank 2 in 54.7 %	[K] oracle	drafter only	R:'RERANK-0 RESULT'; TL:data/rerank0.txt
16	Per-depth pick between the two drafters (aligned)	code 9.58 -> 10.28 kept (x1.073), but the MTP chain's ~33 ms per cycle costs more: KILL	[K] kept, [P] speed	drafter only	R:'MTPPICK aligned'; TL:data/mtppick_sim.txt
17	Leaf trees on the MTP chain	-7 % .. +0.9 % on every set; a leaf row costs 7.8 ms, a rescued chat token is worth ~34 ms: KILL	[P] on measured ranks	needs a tree verify (not built)	R:'D-24 MTPTOP3'; TL:data/d24_read.txt
18	32-row checks	best realizable arm 12.60 HumanEval+ / 6.10 MBPP kept per check, lines 14 / 7.5: KILL	[K] walk	drafter only	R:'DEPTH32 RESULT'; TL:data/d23h_read.txt
19	Copying the context's continuation over the draft (MATCH, 16 rows)	kept per check: walk x1.038 on 24 HumanEval+; engine x1.08 geo / x1.11 pooled (11.16 vs 10.01; 1.114 unrounded) on 12 HumanEval. Whole decode x1.077 pooled (125.19 vs 116.23 tok/s; geo x1.056). Shipped in the code profile	[K]	the lever keeps exactness; the engine figure is on the lossy code profile	R:'MATCH RESULT'; KS; BM:data/fday, data/sat; BM:data/r57/st{b,bb,s,sb}_224_runs.json
20	Prompt lookup / suffix drafting on chat	alone x0.44-0.59 tok/s, hybrids with the MTP chain x0.89-0.996, an oracle selector x1.012: KILL	[P] replay on measured check costs	exact	R:'CHAT150'; TL:data/chat150_read.txt
21	What one MTP draft step costs	the 248K-vocabulary output layer is 1.47 ms of ~2.2 ms, at 242 GB/s = 85-94 % of the MacBook Pro's streaming bandwidth	[K]	-	R:'HEADBENCH'; TL:data/head_bench.txt
22	The trained head's bf16 parts quantized to 4 bits, in the engine (12 IFEval, C 256)	exact config neutral (whole decode 36.61 -> 36.94 tok/s); lossy config KILL twice: v1 draft per check 8.6 -> 14.4 ms pooled (9.1 -> 16.2 mean over prompts); v1b whole decode 43.80 -> 40.22 tok/s (-8.2 %), draft 7.9 -> 13.4 ms pooled (8.7 -> 14.9 mean over prompts); lifetime peak 42.37 (L) vs 42.28 GB (L + v1b)	[K]	exact (12 / 12 equal to one-token decoding) / lossy ungated	R:'D1 v1b in the engine'; BM:data/chs/chs4_report.txt (arms y0, y0p4); whole request and first token not in that run's per-arm report (stated limit)

#	result	number	mark	config	source
23	Chat check cost and the drafter ceiling	exact: 47.0 ms + 3.25 ms per row + 2.59 ms per drafted row (least squares of wall minus draft on rows, checks >= 2; a plain fit of the check time gives 44.9 + 4.00 per row); lossy 33.1 + 4.35 per row; the fixed part is ~75-85 % of a chat check at 2-4 rows. A drafter that never misses: ~105-115 tok/s at 16 rows with the serial draft, ~135-150 with a one-pass draft	[K] fit, [P] ceiling	exact / lossy	R:'CHAT150'; TL:data/chat150_read.txt; BM:data/r57/nie, *nil*_256_checks.json
24	TF32 in MLX 0.32.2	fp32 matmuls run as TF32 by default: error vs float64 4.15e-4 (6.5e-7 with MLX_ENABLE_TF32=0). With it off, MLX's trainer matches our PyTorch port within 1.53e-3; on vs off moves the training loss -1.3..-3.5 % at steps 2-5	[K]	-	R:'PORT steps FAIL: ROOT CAUSE FOUND'; R:'PORT full-scale confirmation'; TL:port/test_mlx_tf32.py, data/port/
25	A 3-bit trunk under the MTP head	kept per 8-row check 4.02 -> 4.09: no harm	[K]	lossy trunk	R:'MTP3BIT'; TL:data/mtp_3bit_vs_4bit.txt
26	Guarded comment-row acceptance on HumanEval (T16)	kept x1.031 (geo); whole decode 173.2 vs 173.2 tok/s: KILL; peak 42.9 GB	[K]	the certified lossy code config (v1' + wired, [EVAL]); the T16 arm itself was never gated	R:'T16-HE' (its RESULT); BM: data commit a03e25d (scripts/t16he_ab.sh 7bc901e)
27	Chat head continued on 0.10M tokens of the model's own chat features (F2a; the swap guard stopped capture at 8 % of plan)	held-out kept x1.014 (IFEval 2.301 -> 2.334, multi 2.507 -> 2.540); on-policy x1.002 (2.444 -> 2.450): KILL	[K]	drafter only	R:'F2A night RESULT'; TL:data/f2a/f2a_eval.txt, capture.log
28	Competitor drafting: Strata's MTP layer on the same model, from our own runs	+53 % decode (25.3 -> 38.7 tok/s pooled, all sets), 87 % of drafted tokens accepted	[K]	its lossy default (Q8_0 hyper-connections)	F:2026-11-08_1630_strata-head-to-head

Appendix B. Sources

Paths are relative to the root of the public snapshot, github.com/kamenlevi/350-tps; commit hashes are those of each folder's own repository, and the snapshot holds the files as last committed.

key	file
R:'*words*'	a section of `TrainLab/REPORT.md`, found by the words of its heading (line numbers move)
TL:*path*	a file under `TrainLab/` (raw data `data/`, scripts `sticky/`, runners `turns/`, notes `notes/`, heads `ckpt/`)
BM:*path*	a file under `BigMoE/` (the engine's own run records and scripts)
KS	`TrainLab/notes/K_SCOREBOARD.md` (the engine's kept-per-check table, from BM:data/fday and BM:data/sat)
F:*slug*	a note in `FableGod/findings/`
[MAIN], [KERNELS], [EVAL], [RELEASE]	the companion papers (see References)

Raw files behind the headline rows:

- TL:data/d22r_base76_vs_stock.txt, d22r_b3_vs_stock.txt, d22r_b4_vs_stock.txt, d22_b5_vs_b4.txt (section 3.1);

- TL:data/d17_route.txt, d17_ci.txt, d20_runs.jsonl, d20b_summary.txt, d23_runs.jsonl, d23_switchdef.txt, d23c_runs.jsonl, d23c_mtptrunc.txt, d23d_read.txt, paper_speed.txt, logs/d23d.out (3.2);
- TL:data/d02b_walk*.txt, e2_read.txt, e15_read.txt, dd_read.txt, dv_read.txt, d13_ratio.txt, d15_read.txt (3.3);
- TL:data/rerank0.txt, kw_read.txt, mtppick_sim.txt, d24_read.txt, d23h_read.txt, cut_sweep.txt, m23b.txt (3.4);
- TL:data/match_read.txt, gate_read.txt, chat150_read.txt (3.5);
- TL:data/head_bench.txt, mtp_step_bench*.txt, d1_variants.txt, da_check.txt, cpu_head_bench.txt; BM:data/chs/chs4_report.txt (3.6);
- TL:data/chat150_read.txt, f2a_headroom.txt, f2a/f2a_eval.txt (3.7);
- TL:port/test_mlx_tf32.py, data/port/ (3.8).

Use of AI tools

The code, the experiments and the text of this paper were produced with substantial help from AI coding agents. The author directed the work and is responsible for its content. The numbers were re-derived from the raw run records named beside them, and the references were opened by hand or by script.

References

- Youssof Altoukhi. MTPLX (software, v2.12.2, commit 9882703f; Apache-2.0). https://github.com/youssofal/mtplx
- Prachi Badarayani et al. "Match the Distribution, Not the Compute: Post-Training Multi-Token Prediction Heads." arXiv:2610.00888, 2026.
- Tianle Cai et al. "Medusa: Simple LLM Inference Acceleration Framework with Multiple Decoding Heads." arXiv:2401.10774, 2024.
- Yuxuan Cai et al. "FastMTP: Accelerating LLM Inference with Enhanced Multi-Token Prediction." arXiv:2509.18362, 2025.
- Jian Chen, Yesheng Liang and Zhijian Liu. "DFlash: Block Diffusion for Flash Speculative Decoding." arXiv:2602.06036, 2026. ICML 2026. Code: https://github.com/z-lab/dflash (MIT).
- Liangkun Chen et al. "SP-MoE: Speculative Decoding and Prefetching for Accelerating MoE-based Model Inference." arXiv:2510.10302, 2025.
- Zhuoming Chen et al. "Sequoia: Scalable, Robust, and Hardware-aware Speculative Decoding." arXiv:2402.12374, 2024.
- DeepSeek-AI. "DeepSeek-V3 Technical Report." arXiv:2412.19437, 2024.
- Awni Hannun, Jagrit Digani, Angelos Katharopoulos and Ronan Collobert. "MLX: Efficient and flexible machine learning on Apple silicon" (software, 2023; MIT). https://github.com/ml-explore/mlx
- Yuxuan Hu et al. "SAM Decoding: Speculative Decoding via Suffix Automaton." arXiv:2411.10666, 2024.
- Kaixuan Huang, Xudong Guo and Mengdi Wang. "SpecDec++: Boosting Speculative Decoding via Adaptive Candidate Lengths." arXiv:2405.19715, 2024.
- Inco AI. "DFlash 2: Keep Drafting Parallel." Blog, Aug 2026. https://inco.ai/blog/dflash2
- Inco AI. Qwen3.8-Flash-Next-DFlash2 (drafter checkpoint, revision a22a174, 19 Oct 2026; Apache-2.0). https://huggingface.co/incoai/Qwen3.8-Flash-Next-DFlash2
- Young D. Kwon et al. "WhiFlash: Accelerating Speculative Decoding with Token-Level Cross-Paradigm Routing." arXiv:2606.07710, 2026.
- Yaniv Leviathan, Matan Kalman and Yossi Matias. "Fast Inference from Transformers via Speculative Decoding." arXiv:2211.17192, 2022. ICML 2023.
- Yuhui Li et al. "EAGLE-2: Faster Inference of Language Models with Dynamic Draft Trees." arXiv:2406.16858, 2024.
- Yuhui Li et al. "EAGLE-3: Scaling up Inference Acceleration of Large Language Models via Training-Time Test." arXiv:2503.01840, 2025.
- Yuhui Li et al. "EAGLE: Speculative Sampling Requires Rethinking Feature Uncertainty." arXiv:2401.15077, 2024.
- Hong Liu et al. "AngelSpec: Towards Real-World High Performance Inference with Speculative Decoding." arXiv:2607.25852, 2026.

- Tianyu Liu et al. "PEARL: Parallel Speculative Decoding with Adaptive Draft Length." arXiv:2408.11850, 2024.
- Kai Lv et al. "DuoDecoding: Hardware-aware Heterogeneous Speculative Decoding with Dynamic Multi-Sequence Drafting." arXiv:2503.00784, 2025.
- Jonathan Mamou et al. "Dynamic Speculation Lookahead Accelerates Speculative Decoding of Large Language Models." arXiv:2405.04304, 2024.
- Xupeng Miao et al. "SpecInfer: Accelerating Generative Large Language Model Serving with Tree-based Speculative Inference and Verification." arXiv:2305.09781, 2023.
- MLX documentation: "Environment Variables", "Numerical Precision" and "Custom Metal Kernels". https://ml-explore.github.io/mlx/build/html/
- MLX issue #4595, "Option to opt out of TF32 for fp32 conv: small-kD conv3d is ~5000× less precise on M5 since 0.32", 29 Sep 2026. https://github.com/ml-explore/mlx/issues/4595
- MLX issue #4632, "[BUG] gather_qmm(sorted_indices=True) returns wrong results on M5 Max (macOS 27) when rows > 32768 and not a multiple of 64", 5 Oct 2026. https://github.com/ml-explore/mlx/issues/4632
- Niko1221 and the Strata contributors. Strata (software, version 0.1.58-metal, commit 146df3c; MIT). https://github.com/Niko1221/Strata
- Gabriele Oliaro et al. "SuffixDecoding: Extreme Speculative Decoding for Emerging AI Applications." arXiv:2411.04975, 2024.
- Qwen Team. Qwen3.8-Flash-Next (model weights; Qwen Community License 1.0). https://huggingface.co/Qwen/Qwen3.8-Flash-Next
- Apoorv Saxena. "Prompt Lookup Decoding" (software), Nov 2023. https://github.com/apoorvumang/prompt-lookup-decoding
- Ruslan Svirschevski et al. "SpecExec: Massively Parallel Speculative Decoding for Interactive LLM Inference on Consumer Devices." arXiv:2406.02532, 2024.
- Wenfeng Wang et al. "Specter: Fast MoE Inference via Quantization-Enabled Multi-Step Speculative Offloading." arXiv:2511.14102, 2025.
- Jinhui Wei et al. "Ghidorah: Fast LLM Inference on Edge with Speculative Decoding and Hetero-Core Parallelism." arXiv:2505.23219, 2025.
- Heming Xia et al. "ToolSpec: Accelerating Tool Calling via Schema-Aware and Retrieval-Augmented Speculative Decoding." arXiv:2604.13519, 2026.
- Jiebin Zhang et al. "DFlare: Scaling Up Draft Capacity for Block Diffusion Speculative Decoding." arXiv:2606.02091, 2026.
- Weilin Zhao et al. "FR-Spec: Accelerating Large-Vocabulary Language Models via Frequency-Ranked Speculative Sampling." arXiv:2502.14856, 2025.

Companion papers:

- [EVAL] Kamen Levi. Certifying lossy decoding on the model's known benchmarks: same-scores gates for a 125B mixture-of-experts model on one 64 GB Mac. 2026. A companion paper of this set.
- [KERNELS] Kamen Levi. Batch-invariant kernels on Apple's M5 GPU and its matrix units. 2026. A companion paper of this set.
- [MAIN] Kamen Levi. What It Takes to Run a 125B LLM Fast on a 64 GB Mac, and How to Go Faster. 2026. A companion paper of this set.
- [RELEASE] Kamen Levi. Running a 125B mixture-of-experts model on a 64 GB Mac: the engine package. 2026. A companion paper of this set.

Running a 125B mixture-of-experts model on a 64 GB Mac: the engine package

Kamen Levi
Independent researcher

9 November 2026

Abstract

This paper documents the engine package behind [MAIN]. The engine runs Qwen3.8-Flash-Next, a 125B mixture-of-experts model, on one MacBook Pro (M5 Pro, 64 GB): the experts are stored once on the internal SSD, and each layer keeps a fixed pool of them in memory. It installs from a clean clone (tested on one machine) and ships six configurations: exact, lossy and chat, each at a 45 GB and a 32 GB memory line, labelled as [MAIN] section 2 defines. At 45 GB, the exact configuration, bit-identical to one-token greedy decoding under our reference NAX6-N, decodes 37.1-101.4 tok/s; the shipped lossy one, which scored the same as the exact reference on the model card's IFBench, GPQA Diamond and LiveCodeBench v6 in both orders, decodes 45.2-164.0 tok/s; both stay at a lifetime peak of 41.7-43.2 GB, plus a 7.45 GB expert tier that macOS may reclaim [K, section 2]. Where we lose: on the same machine and prompts, Strata's first token (0.5-1.0 s) beats ours (2.7-4.6 s; 2.26 s on chat as shipped), so up to 256 generated tokens Strata wins or ties every whole-request number [K, section 2]. The code is GPL-3; the model's weights keep their own licence (section 7).

Every number has a mark ([K] measured, [P] priced, [G] judgement) and a source key (section 8; keys as in [MAIN] Appendix C). Code: GPL-3.

1. What this is

An inference engine that runs Qwen3.8-Flash-Next, a 125B mixture-of-experts model (48 layers, 512 routed experts per layer, 10 per token), on one MacBook Pro (M5 Pro, 64 GB unified memory, internal SSD). The experts are stored once on the SSD at about 3.2 bits per expert weight; each layer keeps a fixed pool of experts in memory and reads the rest on demand. The engine's lifetime peak memory footprint is the memory line it is named for (45 GB or 32 GB): the engine's own footprint on a 64 GB machine, not an OS limit ("GB" = `10^9` bytes of the process's lifetime peak phys_footprint). Besides it, every configuration holds a 7.45 GB volatile expert tier as purgeable memory, which macOS reclaims before it compresses or swaps and which phys_footprint does not count [K, BM:data/rel/rli_160.txt line 7].

It ships three kinds of configuration, labelled as [MAIN] section 2 defines them:

- **exact (to NAX6-N):** output tokens and the logits behind every kept token are bit-identical to one-token greedy decoding under our reference NAX6-N.
- **lossy, same scores on :** passed a pre-registered same-scores gate against the exact reference on the named known benchmarks ([EVAL]).
- **lossy, ungated:** faster, not yet gated; shipped switched off, labelled.

2. Results you should expect (pooled; 12 prompts per set, 256 new tokens, greedy; [MAIN] has the full tables)

Whole decode / whole request (tok/s) / first token (median, s) / lifetime peak (GB). Source rows: section 8.

line	config	label	HumanEval	MBPP	IFEval (chat)	math
45 GB	E (+ bundle)	exact (NAX6-N)	101.4 / 32.6 / 4.41, 42.24 [K]	70.4 / 21.7 / 3.80, 41.92 [K]	37.1 / 20.8 / 2.89, 41.70 [K]	77.9 / 38.9 / 2.73, 42.50 [K]

line	config	label	HumanEval	MBPP	IFEval (chat)	math
45 GB	lossy as shipped: L + COH	lossy, same scores on IFBench, GPQA Diamond and LiveCodeBench v6 (the card gate, both orders, vs NAX6-N; EV:RESULT_card_ lcoh_2026-11-09; the package runs the gated code and env, section 4). The certificate (next row's label) does not carry: it was earned without the bundle and without COH, and COH changes the one-call path's reads. L alone is run-to-run nondeterministic on chat; L + COH repeated itself: a second IFEval pass in its own process, same prompts in the same order, gave the same tokens on 12 / 12 prompts [K]. One pass on HumanEval, MBPP and math; IFEval pools the two passes	164.0 / 36.2 / 4.60, 42.94 [K]	112.7 / 25.3 / 3.72, 42.60 [K]	45.2 / 22.4 / 3.01, 42.42 [K]	78.9 / 37.9 / 3.20, 43.17 [K]
45 GB	L = the certified flags (CERT: v1' + wired) + the exact-safe bundle; row 27 without COH; not shipped	lossy, not gated by itself. The certificate (same scores on HumanEval+, MBPP, math and IFEval, both orders; EV:RESULT_cert_ 2026-11-08) was earned by CERT, without the bundle. CERT's own rows (S5' B): HumanEval 171.0 / 36.0 / 4.56, 42.91 [K]; MBPP 114.5 / 25.8 / 3.79, 42.55 [K]. The bundle is bitwise on the exact path; on this lossy path it is not shown to keep tokens	181.3 / 36.6 / 4.51, 42.91 [K]	119.8 / 25.3 / 3.76, 42.57 [K]	43.4 / 21.1 / 3.00, 42.37 [K]	86.9 / 39.6 / 2.96, 43.10 [K]
45 GB	M1-B + COH (chat)	lossy, same scores on IFEval (both orders; EV:RESULT_chat_ m1b_2026-11-08) and on IFBench, GPQA Diamond and LiveCodeBench v6 (the card gate, both orders, vs NAX6-N; EV:RESULT_card_ m1b_2026-11-09); the multilingual set is information only	-	-	49.8 / 24.4 / 2.92, 42.38 [K]	-
45 GB	chat as shipped: zs (M1-B + COH + SB1) + PF-D v4 + read-ahead	as M1-B + COH: the same tokens on all 12 prompts [K]; PF-D v4 is bitwise to zs (logits hashes 721/721) and SB1 only reorders work (logits hashes 676/676 on E) [K]	-	-	50.7 / 28.8 / 2.26, 43.69 [K] (one pass)	-
32 GB	E at C 160	exact (NAX6-N)	not measured	not measured	33.8 / 19.8 / 2.83, 30.62 [K]	not measured

line	config	label	HumanEval	MBPP	IFEval (chat)	math
32 GB	L + COH at C 160 (lossy-32)	lossy, not gated (no gate ran at C 160); C30's PASS (L at C 160 before COH; HumanEval+ only, EV:RESULT_c30_ 2026-11-07) does not carry: COH changes tokens on the one-call path; the card gate (row 27) ran at C 256 only. On IFEval, C 160 gives the same tokens as C 256 (12 / 12) [K]	C30, before COH: 150.7 / 36.0 / 4.43, 31.78 [K]	C30, before COH: 111.0 / 26.8 / 3.69, 31.44 [K]	39.0 / 21.1 / 2.90, 31.30 [K] (one pass)	-
32 GB	zs32 = M1-B + COH + SB1 at C 160	lossy, not gated: its card gate did not fit in Monday's gate window (EV:PREREG_card_ gates_2026-11-08, amendment 11); row 29's IFEval pass was at C 256	-	-	51.3 / 22.5 / 2.99, 31.22 [K]	-

- The shipped lossy config's card gate (L + COH against the exact reference, both orders): IFBench 40.7 against 44.0 % (-3.3 points), GPQA Diamond 47.0 against 44.0 % (+3.0), LiveCodeBench v6 30.5 / 29.0 against 32.1 % (-1.5 / -3.1). All are inside the pre-registered rule (a delta above -5 points and an exact McNemar p of at least 0.05), so it passes. A pass bounds the loss at these sizes (150 / 100 / 131 questions); it does not show there is none [K, EV:RESULT_card_lcoh_2026-11-09].
- The chat config's card gate (M1-B + COH against the exact reference, forward / reverse order): IFBench 44.0 / 41.3 against 44.0 % (0.0 / -2.7 points), GPQA Diamond 42.0 / 42.0 against 44.0 % (-2.0 / -2.0), LiveCodeBench v6 29.0 / 32.1 against 32.1 % (-3.1 / 0.0). All six are inside the same rule, so it passes; the same bound applies [K, EV:RESULT_card_m1b_2026-11-09].
- The shipped lossy config decodes slower than L (row 28) on code and math: HumanEval 164.0 against 181.3 tok/s, MBPP 112.7 against 119.8, math 78.9 against 86.9; on IFEval it is faster, 45.2 against 43.4 [K]. This is not an A/B: the runs are from different days (8 and 9 Nov), L + COH has one pass on those three sets against L's two, and COH changes tokens, so acceptance differs too. The cause is not measured.
- Where we lose: against Strata (MIT; github.com/Niko1221/Strata), run by us on the same machine and prompts (every Strata number here is our measurement, none self-reported), our decode is ahead on code and math and about level on chat, exact chat decode is behind, and Strata's first token (0.5-1.0 s) beats ours (2.7-4.6 s; 2.26 s on chat as shipped, with PF-D), so up to 256 generated tokens Strata wins or ties every whole-request number. Chat as shipped (2.26 s, 28.8 tok/s whole request, peak 43.7 GB) against Strata's full-cache arms (0.83 s, 33.0-33.7 tok/s; peaks 40.8-41.2 GB): first token 2.7x slower, Strata ahead on whole request, our chat decode ahead (50.7 vs 40.6-41.8 tok/s) [K, SE:h2h/results_ arena33/{B1,D1}.requests.jsonl].
- Long prompts: the default prefill re-reads each layer's experts per 1,024-token chunk. PF-A (layer-major) and PF-B (layer split
  - read-ahead; exact since the seg-per-chunk fix) prefill a 7.9K-token agent prompt at 433 and 500 tok/s [K] (exact E; lifetime peak 45.09 / 46.23 GB [K], above the 45.0 line on that prompt).
- Chat first token: PF-D builds a chat prompt's pool-resident experts on the GPU from memory instead of re-reading them from the SSD; v4 also reads the next layer's misses during this layer, as Strata's prompt path does (section 7). It is exact: tokens and logits are bitwise equal, and every staged row equals the reference rebuild (87,912 rows on E) [K]. On the chat config the first token falls from 2.91 to 2.26 s (median), whole request rises from 24.6 to 28.8 tok/s and whole decode is unchanged (50.3 / 50.7) [K, 12 IFEval, 256 tokens, one pass each]. On exact E an earlier version cut the first token from 2.89 to 2.26 s (64 tokens) [K]. Default: on for chat at 45 GB. It is off for the other configs, whose rows above were measured without it, and it was not measured at 32 GB, where its 1.0-1.3 GB of extra peak matters. An earlier run's decode gap (40.0 against 49.6 tok/s) did not reproduce when the PF-D arm ran first; its cause is unknown [K]. A lossy switch (absent experts' side parts skipped: 3-bit weights) reaches 1.83 s and 30.5 tok/s but changes tokens on 10 of 12 prompts; it is ungated and ships off.

3. Requirements

- Apple silicon Mac with 64 GB unified memory (tested: MacBook Pro, M5 Pro, 64 GB); tested on macOS 26.5 (25F71) only.
- Python 3.12; MLX pinned to 0.32.2 with the pad-to-64 fix for MLX issue #4632 (sorted gather_qmm leaves rows unwritten past 32,767 when M is not a multiple of 64; 0.32.4 fixes it but changes tokens against our exact references); mlx-lm 0.31.3; MTPLX 2.12.2 from a pinned commit (section 4); CMake + a C++ toolchain for the extension.
- The model weights: Qwen3.8-Flash-Next by Qwen (Qwen Community License 1.0, section 7), as MTPLX's flat 4-bit MLX pack (Hugging Face: Youssofal/Qwen3.8-Flash-Next-MTPLX-Bare-Speed). The package ships no weights.
- Disk: the pack as downloaded, without its vision tower: 105.4 GB [K] (routed experts 67.9 GB; the n-gram table, 32.0 GB, which the model code streams from the SSD at run time; dense text weights 3.8 GB; MTP head 1.7 GB), plus the converted expert store, 48 x 512 experts x 2.79 MB = 68.5 GB [K] (records3, native+p11+pad layout): about 174 GB in all [K].

4. Install and run

The package is the frozen import closure of real runs, not a rewrite: the engine's frozen snapshot (the decode loop and every module it loaded, taken read-only from the commit that holds the snapshot), the union of all 174 module manifests the engine's runs wrote (43 modules from the kernel, drafter and keep-set trees), the engine package, the extension source, the harness package and the published env lines. The manifests miss three kernel files that the lossy and chat runs read from another research folder (section 8); the package already held byte-identical copies, and the clean-clone smoke found that the engine still looked for them in that folder (below). A build script renames the trees (engine/, kernels/, third_party/, drafters/, keepset/, harness/), rewrites every path into the clone, scrubs comments by the public snapshot's rules and refuses to commit on a leak, a syntax error or a path left pointing at the research trees. Its research-path check did not cover that folder until the smoke: the two engine lines that name it are now rewritten to the package's copies, and the build lists the 43 other home-folder paths still named in the code (records of a run's source, benchmark outputs, optional self-checks, switches the harness refuses) in its manifest. The build from the PF-D snapshot: 218 files, 6.2 MB, 0 leaks, 0 syntax errors, 0 research paths left by the check of that day; from the provisional release freeze rel0 after the smoke's fixes (9 Nov 07:20): 222 files, 6.3 MB, 0 / 0 / 0 [K, LT:'ENGINE PACKAGE v0']; the release build from the engine's final snapshot rel1 (frozen 9 Nov 07:19, identical to rel0 apart from its name), 9 Nov 09:57: 225 files, 6.3 MB, 0 / 0 / 0, which are rel0's files plus the conversion driver and the two lossy references [K, LP:data/release/rel1_build.txt].

Install (Python 3.12, CMake, the Xcode command-line tools):

```
bash install.sh --store <model dir with records3/> --weights <dir with l120_ft.safetensors and
d22_b5.safetensors>
```

It writes the clone's path into the code, creates a venv with the pinned packages (requirements.txt: MLX 0.32.2, mlx-lm 0.31.3, MTPLX 2.12.2 at its GitHub commit, and the rest of the measured venv), builds the extension with the vendored nanobind 2.15.0, and links the store and the two drafter files: l120_ft.safetensors (1.14 GB, our finetune of DFlash2) and d22_b5.safetensors (277 MB, the MTP head). Neither is in the repository; how they are published is decided with the release. The block drafter's base checkpoint (DFlash2, section 7) comes from the Hugging Face cache at the revision our runs loaded: install.sh checks for it, and run.py pins it (the drafter's own loader would take the cache's default branch) and stops with a one-line fetch command when it is missing. A cache that holds only that revision resolves through run.py and fails without the pin [K, LP:data/release/drafter_pin_test.txt].

Run:

```
.venv/bin/python run.py --config exact|lossy|chat --mem 45|32 --prompt "..." [--max-tokens 256]
```

run.py runs the decode loop unchanged, the way the research runner ran it (one arm, one repetition, one discarded warm-up prompt), with the config's env line from configs/-.env. It only turns a text prompt into a prompt item with the harness's chat template (non-thinking) and prints the answer, whole decode, whole request, first token and the lifetime peak. `-keys ifeval/1220,...` runs benchmark items instead. The six configs: exact = E (exact to NAX6-N) at C 256 / 160; lossy = L + COH; chat = zs (M1-B + COH + SB1, bitwise equal to M1-B + COH), at 45 GB with PF-D v4 and its read-ahead (R57_PFD=1, R57_PFD_V=4, R57_PFD_AHEAD=1; the same tokens, section 8), at 32 GB without; PF-D is off in the other four; their labels are section 1's. Each env line is the published one (parse-checked against the engine's run scripts) [K,

LT:'ENGINE PACKAGE v0']. The lossy configs run what the card gate ran: against the card snapshot the engine package, the extension source and every module the lossy runs import are the same, the decode loop differs only by the PF-D switch (off in them), and the env line differs only by the name of the card's own code benchmark in the QOM sets (run.py names a code prompt `humaneval`, which is in them, and a chat prompt `multi`, which is not) [K, LP:data/release/lossy_card_equiv.txt].

Conversion. The store is built in three steps: (a) safetensors -> records/ (4-bit pieces; repack_records.py), (b) records/ -> records3 row-major, MSB-3 + LSB-1 parts (make_records3.py, with a verifier on every expert: rebuilt 4-bit == the originals, s3 / 2 == s), (c) row-major -> native+p11+pad (convert_records_p11.py --pad); format: DESIGN_3bit.md "Record file". convert/convert_store.sh runs the three steps layer by layer into a store (a layer's 4-bit pieces are deleted after the split; finished layers are skipped when it is run again). Run from the tested package clone on all 48 layers of the safetensors (9 Nov 16:07-16:17), it built the full store in 9.5 min (11-14 s per layer), and the result equals the store that every run in this paper used, byte for byte: all 48 layer files (1,426,063,360 B each) and index.json [K, LP:data/release/convert_full.txt]. Earlier runs on layers 0 and 47 alone gave the same bytes, at 19-39 s per layer with other jobs running [K, LP:data/release/pfd_convert_report.txt, LP:data/release/convert_store_test.txt]. The release runs used the existing store.

Clean-clone test (passed on one machine). `tools/clean_clone_test.sh` clones the package, installs it offline (no network: the wheels are rebuilt from the measured venv's installed files; MTPLX's 321 source files are identical to the venv's), builds the extension, then runs every config on two IFEval prompts (up to 256 tokens each, the references' cap) with HOME set to an empty directory, so any path left pointing outside the clone fails instead of reading the research trees. It checks the tokens against the shipped references (the exact configs as a prefix of one-token decoding; chat and lossy against the engine's own runs of the same config: for lossy, its IFEval runs of L + COH on rel1 at C 256 and C 160, which a repeat in its own process matched on 12 / 12 prompts and which equal each other on 12 / 12 [K, BM:data/rel/rel_rows_report.txt]; a reference of a config never repeated would ship as information only), that every module the engine imported lies in the clone or its venv, and that every file Python opened lies in the clone, its venv, the store, the drafter files, the Hugging Face cache or the system's folders. The lossy and chat configs' tokens depend on what the process decoded before (their relaxed acceptance takes the drafter's proposals, and the drafter's state carries across prompts), so a prompt of theirs is checked only when the earlier prompts ran as in the reference run: the same prompts, in the same order, to the same length. Without the GPU (8 Nov): install rc 0 in 5-7 s, dry runs of all six configs, and all 51 shipped modules import from the clone under the empty HOME with none from outside it [K, LP:data/release/pfd_prep_report.txt]; the rel0 build again: rc 0 in 7 s, six dry runs [K, LP:data/release/rel0_prep_report.txt]. The GPU smoke on rel0, 9 Nov 07:00-07:32 [K, LP:data/release/smoke_0700.txt]:

- 07:01: the exact configs' tokens equal their references on both prompts, but the four lossy and chat configs and the free-text prompt stopped at import: the engine still looked for the three kernel files in the other research folder (section 8), and the closure check flagged the drafter files because it followed their links without allowing them (a fault of the check). Both fixed in the build: the two path strings now point at the shipped copies, and the check allows the linked drafter files.
- 07:13, at a 64-token cap: all six configs and the free-text prompt ran, with every module and file inside the allowed folders; exact 2 / 2; chat matched prompt 1 but not prompt 2 (from token 4 at 45 GB, token 60 at 32 GB), because the cap cut prompt 1 at 64 tokens where the reference run let it end at 192 / 184, so prompt 2 followed a different history.
- 07:28, at cap 256 with the history rule above: CLEAN-CLONE PASS. Exact 2 / 2; chat 2 / 2 to the references' full length (192 + 235 and 184 + 256 tokens); lossy reported (no reference yet); every module and every opened file inside the allowed folders on all six configs; the free-text prompt gave a complete answer.

Before the final run, the 07:28 smoke's lossy tokens (rel0, the same code) were compared on the CPU with the L + COH references: 2 / 2 prompts equal to their full length (154 + 139 tokens) at 45 and 32 GB [K, LP:data/release/lossy_precheck.txt]. The final run on the release package (rel1, its clone prepared 9 Nov 09:58), with those references as checks, ran 9 Nov 12:10-12:14: CLEAN-CLONE PASS [K, LP:data/release/final_1210.txt]. All six configs gave their references' tokens on both prompts (exact as a prefix of one-token decoding; lossy and chat to the references' full length), every module and every opened file on all six lay inside the allowed folders (834-849 modules and 3,268-3,353 files per config, none outside), and the free-text prompt gave the same complete 87-token answer as at 07:28. Read on the CPU afterwards, its tokens equal the 07:28 smoke's on all 12 prompt runs. On the lossy and chat configs each prompt took the same checks and kept the same tokens per check as the engine's own run of it, at the same pace: over the two prompts, whole decode x0.996-1.020 and whole request x0.996-1.012 of those runs, median first token within 0.11 s (information only: two prompts per config) [K, LP:data/release/final_1210.txt].

5. Known limits

- First token on short prompts at 45 GB: 2.7-4.6 s; 2.26 s on chat as shipped, with PF-D [K]. Strata's is 0.5-1.0 s (0.83 s on chat at its full cache) [K].
- PF-D was measured on chat-length prompts (37-101 tokens) only; long prompts take the grouped prefill path without it.
- Lifetime peaks above 45.0 GB on 7.9K-token prompts (45.1-46.4 GB) [K].
- At 32 GB the exact config was measured on chat (IFEval) only: its HumanEval, MBPP and math speeds were not measured. The 32 GB lossy config has IFEval rows of its own (L + COH) and code rows only from L before COH (C30).
- MLX version sensitivity (0.32.4 changes tokens against the exact references).
- Lossy configs are gated on the named sets only; the multilingual chat set fails the blunder line for M1-B (information).
- The clean-clone test ran on one MacBook Pro (section 3's) with two IFEval prompts per config; other Macs and macOS versions are untested.

6. What we left on the table

- Chat prefill read-ahead: v4's next-layer prediction (layer L+1's router on layer L's input) supplied 71-75 % of the rows read from the SSD, and 19-31 % of what it staged went unused, so SSD bytes per prompt rose from 7.5 to 9.6 GB [K]. A better predictor, and PF-D measured on the other configs and at 32 GB, are left.
- SSD reads at ~7.3 GB/s [K] of a 12-14 GB/s drive; larger side pools; per-call Python overhead (~0.4 s per prompt) [K].

7. Credits and licences

- Strata (MIT; github.com/Niko1221/Strata; "Copyright (c) 2026 Niko1221 and the Strata contributors"): PF-D and its read-ahead follow Strata's prompt-path design (read only what memory lacks; read the next layer's misses during this layer). No Strata code is included; anything ported later keeps its notice.
- MTPLX by Youssof Altoukhi (Apache-2.0; github.com/youssofal/mtplx): our engine builds on its model code for this architecture, and it ran this model's own MTP head as a drafter on Apple Silicon before we did (its backend for the family is dated 29 Aug 2026). The package installs MTPLX from a pinned commit (it does not copy MTPLX's sources), ships its LICENSE and NOTICE (third_party/LICENSE-mtplx, NOTICE-mtplx), and, as the NOTICE requires, shows "Powered by MTPLX" and https://github.com/youssofal/mtplx in the CLI's startup banner and the README credits. Apache-2.0 is compatible with GPL-3.
- DFlash2 (Inco AI; Apache-2.0): every config drafts with the block drafter incoai/Qwen3.8-Flash-Next-DFlash2 (Hugging Face; a finetune of Qwen/Qwen3.8-Flash-Next; revision a22a174 of 19 Oct 2026, the copy our runs loaded), with our finetuned weights loaded over it. Inco AI, "DFlash 2: Keep Drafting Parallel" (blog, Aug 2026, inco.ai/blog/dflash2); the method: Chen, Liang and Liu, "DFlash: Block Diffusion for Flash Speculative Decoding" (ICML 2026; arXiv 2602.06036; github.com/z-lab/dflash, MIT). The package pins that revision (run.py passes it to the drafter's loader, section 4) and ships no weights; our finetuned weights (l120_ft.safetensors) are distributed separately, with Apache-2.0's licence text and a notice of our changes (third_party/LICENSE-dflash2, third_party/NOTICE-l120_ft).
- MLX (MIT), mlx-lm (MIT).
- The model: Qwen3.8-Flash-Next, "Copyright (c) 2026 Qwen", under the Qwen Community License 1.0. The weights and anything derived from them (the converted store too) stay under that licence, not the GPL-3. Its conditions bind whoever uses the weights: keep its notice; products or services with more than 100 million monthly active users or US$20 million monthly revenue must show the model's name; model-as-a-service and AI work assistant businesses need a separate licence from Qwen for commercial use. The 4-bit pack is MTPLX's (Hugging Face: Youssofal), under the same licence.

8. Sources (raw rows; the pooled values were re-derived from them; keys as in [MAIN] Appendix C)

Paths are relative to the root of the public snapshot, github.com/kamenlevi/350-tps: BM: = BigMoE/, LP: = Leap/, LT: = a section of Leap/TABLE_350.md, MS: = MoonShots/, EV: = EvalSuite/docs/, SE: =

StrataEval/. Commit hashes are those of each folder's own repository; the snapshot holds the files as last committed.

- 45 GB E / EB HumanEval, MBPP: BM:data/e5/ (E5 runs). E IFEval and math: BM:data/n45/ (N45, two passes; the shipped exact config, without SB1). L (= BM:data/stack/s5p_env.txt, CERT + the bundle): BM:data/s5p/ (S5' X passes: spx1-2, spmx1-2), BM:data/n45/ (IFEval, math: nil1-2, nml1-2; n45.sh reads s5p_env). CERT (row 28's label): S5' B passes (spb1-2, spmb1-2). M1-B + COH: CHS5 z0 / z0b (BM:data/chs/chs5_report.txt). 32 GB: BM:data/chs/c32_report.txt (e32, zs32). C30: BM:data/c30/ (f24_report.txt); its pooled rows from BM:data/r57/c30h1_160_runs.json + c30h2 (HumanEval) and c30m1 + c30m2 (MBPP12), pooled the way the re-derivation script ours_runs.py does (sum n / sum decode time; first token = pfs median). Disk sizes: the pack's safetensors headers (tensor bytes by name) and du of the converted store records3 (the converter's output; not in the public snapshot; 8 Nov 2026).
- Prefill: BM:data/pfd/pfd_report.txt at a6d5347 (part B; PF-A / PF-B), BM:data/pfb/pfb_report.txt (CT1). PF-D: BM:data/pfd/ (v3 run, snapshot 7d50712; E rows), BM:data/pfd/pfd_report_2355.txt (diagnostic, PF-D first, chat control; snapshot 366c452), BM:data/pfd/pfd_report_0009.txt (v4 + read-ahead, FAST, E + v4 CHECK; snapshot 36ee120). Release freeze rel0: BM aa1ca3b; rel1: BM 0b4df8c (no change to the engine code BM:bigmoe/ since aa1ca3b).
- L + COH (45 and 32 GB): BM:data/rel/rel_rows_report.txt (REL ROWS on rel1, 9 Nov 09:47-09:57, quiet lock; data commit BM 53dfff5). Raw rows: BM:data/r57/rlh_256, rlm_256, rli_256, rlt_256 (C 256), rli_160 (C 160) and rlr_256 (the IFEval repeat), each _runs.json. Keys: HumanEval and MBPP as E5's e5h1 / e5m1 runs, IFEval and math = BM:data/n45/keys.txt lines 1 / 2. The IFEval cell at 45 GB pools rli_256 + rlr_256. The same names' _tokens.json are the package's lossy references. No row hit the screen's 600 s budget. The fan controller's log shows no heat pause in the run's window.
- Files read outside the freeze: the lossy and chat runs load three kernel files from the tree they were written in, not from the snapshot, and the runs' module manifests do not list them: MS:patches/planes_engine/tierglu.py (committed 2aeb5ad), gdn_exact.py and gdn_exact_patch.py (83109c8), all of 21 Oct 2026 with no later change (no later commit, clean tree). They are byte-identical (md5 dc81729b, 3b835480, 936c2635) to the copies frozen with the snapshot (BM:data/r57/snap/frozen_rel0/Leap_third_party/ms_gdn/) and to the package's kernels/ms_gdn/, which the package loads [K, LP:data/release/smoke_0700.txt]; the package's 9 Nov 07:00 smoke found the gap. The L + COH rows ran only after checking that the three files equal the copies frozen with rel1, and the same check passed after the run [K, BM:data/rel/rel_rows_report.txt].
- Chat as shipped vs M1-B + COH: tokens BM:data/r57/pdz4a_256_tokens.json (db922b9) == BM:data/r57/z0_256_tokens.json and zs_256_tokens.json (CHS5, 5abc51a) on 12 / 12 prompts; logits hashes ("lh") BM:data/r57/pdz4a_256_runs.json vs pdz0_256_runs.json 721 / 721; SB1 on E: BM:data/r57/yeh_256_runs.json vs yesh_256_runs.json 676 / 676 (CHS5). CHS5's z0 / zs runs carry no logits hashes, so the link from zs to M1-B + COH is token-level.
- Strata: SE:h2h (the head-to-head records); F:2026-11-08_1630_strata-head-to-head.

Use of AI tools

The code, the experiments and the text of this paper were produced with substantial help from AI coding agents. The author directed the work and is responsible for its content. The numbers were re-derived from the raw run records named beside them, and the references were opened by hand or by script.

References

- Youssof Altoukhi. MTPLX (software, v2.12.2, commit 9882703f; Apache-2.0). https://github.com/youssofal/mtplx
- Jian Chen, Yesheng Liang and Zhijian Liu. "DFlash: Block Diffusion for Flash Speculative Decoding." arXiv:2602.06036, 2026. ICML 2026. Code: https://github.com/z-lab/dflash (MIT).
- Awni Hannun, Jagrit Digani, Angelos Katharopoulos and Ronan Collobert. "MLX: Efficient and flexible machine learning on Apple silicon" (software, 2023; MIT). https://github.com/ml-explore/mlx
- Inco AI. "DFlash 2: Keep Drafting Parallel." Blog, Aug 2026. https://inco.ai/blog/dflash2
- Inco AI. Qwen3.8-Flash-Next-DFlash2 (drafter checkpoint, revision a22a174, 19 Oct 2026; Apache-2.0). https://huggingface.co/incoai/Qwen3.8-Flash-Next-DFlash2
- Quinn McNemar. "Note on the sampling error of the difference between correlated proportions or percentages." Psychometrika 12(2): 153-157, 1947. doi:10.1007/BF02295996.

- MLX issue #4632, "[BUG] gather_qmm(sorted_indices=True) returns wrong results on M5 Max (macOS 27) when rows > 32768 and not a multiple of 64", 5 Oct 2026. https://github.com/ml-explore/mlx/issues/4632
- mlx-lm (software; MIT). https://github.com/ml-explore/mlx-lm
- Niko1221 and the Strata contributors. Strata (software, version 0.1.58-metal, commit 146df3c; MIT). https://github.com/Niko1221/Strata
- Qwen Team. Qwen3.8-Flash-Next (model weights; Qwen Community License 1.0). https://huggingface.co/Qwen/Qwen3.8-Flash-Next
- Youssofal. Qwen3.8-Flash-Next-MTPLX-Bare-Speed (4-bit weights; Qwen Community License 1.0). https://huggingface.co/Youssofal/Qwen3.8-Flash-Next-MTPLX-Bare-Speed

Companion papers:

- [EVAL] Kamen Levi. Certifying lossy decoding on the model's known benchmarks: same-scores gates for a 125B mixture-of-experts model on one 64 GB Mac. 2026. A companion paper of this set.
- [MAIN] Kamen Levi. What It Takes to Run a 125B LLM Fast on a 64 GB Mac, and How to Go Faster. 2026. A companion paper of this set.